

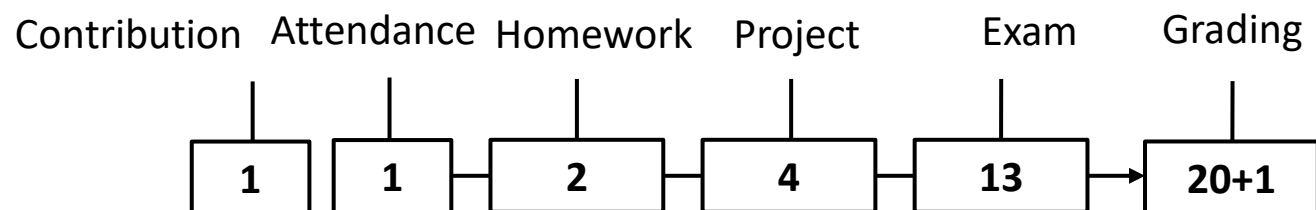
به نام خداوند جان و خرد

زبان مدل سازی یکپارچه



مهندسی نرم افزار ۲

ارزیابی



زبان مدل سازی یکپارچه (UML- Unified Modeling Language)

■ UML یک زبان گرافیکی است که اهداف آن :

■ مشخص کردن (specifying)

■ ساختن (constructing)

■ مصور سازی (visualizing)

■ مستند سازی (documenting)

اجزای یک سیستم نرم افزاری است.

■ UML استانداری تحت حمایت گروه مدیریت شی
(Object Management Group – OMG) است

■ UML بهترین ابزار برای مدل سازی شی گرا است.

مورد کاربرد (Use-Cases)

- رفتار دینامیک یک سیستم را مدل می کنند. برای موارد زیر به کار می روند:
- مدل کردن محتویات یک سیستم - شناسایی **Actor** ها (کاربرانی که از سیستم استفاده می کنند و سیستم های خارجی که با سیستم در ارتباط هستند.) و نقش آنها
- مدل سازی خواسته های سیستم - تعیین اینکه سیستم چیست مستقل از اینکه چگونه به آن می رسد (توصیف اینکه سیستم چه کاری می کند اما نه چگونه؟)
- یک **Use-case** عملیاتی که برای کاربر قابل مشاهده هستند را ضبط می کند.
- هر **use-case** هدف خاصی از کاربر را نمایش می دهد.

مدل سازی Use case

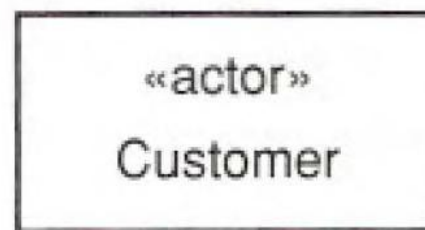
- مدل سازی use case شامل مراحل زیر است:
 - مرز سیستم را مشخص کنید.
 - Actor ها (بازیگران) را پیدا کنید.
 - حداقل یک کاربر می تواند به عنوان Actor انتخاب شود.
 - کمترین همپوشانی بین نقش ها وجود داشته باشد.
 - Use case ها را پیدا کنید.
- Use case کاری است که actor ها در سیستم انجام می دهد.
- از use case های خیلی بزرگ یا خیلی کوچک اجتناب کنید.
- تا زمانی که use case ها، Actor ها و مرز سیستم به حالت ایستا برسد این عملیات را ادامه دهید.

The subject (system boundary)

- موضوع توسط کسی یا چیزی که از سیستم استفاده می کند (برای مثال : **Actor** ها) تعریف می شود و مشخص می کند که مزایای اصلی که سیستم برای **Actor** ها فراهم می کند چیست (**use case** ها)
- **Subject** به صورت جعبه ای همراه با نام سیستم مشخص می شود.
- **Actor** ها خارج از مزر و **use case** ها داخل جعبه قرار دارند.

Actor

- مشخص کننده نقش موجودیت های خارجی هنگام تعامل آنها با سیستم است.
- یک نقش می تواند بازی شود توسط :
 - یک کاربر
 - سیستم دیگر
 - یک قطعه سخت افزاری



Actor مدل سازی یک فروشگاه آنلاین مدرن

■ بازیگران: (Actors)

1. مشتری: (Customer)

کاربری که به فروشگاه وارد شده، به جستجو، انتخاب، خرید محصولات و دریافت پیشنهادات هوشمندانه مشغول می شود.

2. مدیر فروشگاه: (Store Manager)

مسئول مدیریت محصولات، سفارش ها، موجودی کالا و نظارت بر عملکرد کلی سیستم.

3. سیستم هوش مصنوعی: (AI System)

این سیستم وظیفه تحلیل داده های مشتری، ارائه پیشنهادات شخصی سازی شده، پیش بینی روندهای خرید و بهبود تعامل با کاربر را دارد.

4. سیستم پرداخت: (Payment Gateway)

مسئول پردازش تراکنش های مالی به صورت امن و تأیید پرداخت ها.

5. سیستم مدیریت موجودی: (Inventory Management System)

سامانه ای برای کنترل و به روز رسانی موجودی کالاها، دریافت هشدارها در مورد کاهش موجودی و هماهنگی با بخش سفارش ها.

شناسایی actor

- بررسی اینکه چه کسی یا چه چیزی از سیستم استفاده می کند و آنها در تعامل با سیستم چه نقشی بازی می کنند.
- پرسش سوالات زیر برای شناسایی Actorها :
 - چه کسی یا چه چیزی از سیستم استفاده می کند؟
 - نقش آنها در سیستم چیست؟
 - چه کسی سیستم را نصب می کند؟
 - چه کسی سیستم را شروع یا پایان می دهد؟
 - چه کسی از سیستم نگهداری میکند؟
 - چه سیستم هایی با این سیستم در ارتباطند؟
 - چه کسی یا چه چیزی اطلاعات دریافت می کند یا اطلاعات برای سیستم فراهم میکند؟

مشخصه Actorها

- هر Actor به یک نام کوتاه نیاز دارد
- هر Actor باید یک توصیف اجمالی داشته باشد.

Actor name: Order Processing Clerk

Description: The Order Processing Clerk is responsible for processing sales orders, submitting reorder requests, requesting necessary deposits from members and scheduling the delivery of the goods to members.

Use case

- مشخصه ای از عملیات، شامل مجموعه ای از عملیات و خطاهایی که یک سیستم، زیر سیستم یا کلاس در تعامل با **Actor** خارجی انجام میدهد.
- همیشه توسط یک **Actor** شروع می شود.
- همیشه از دیدگاه **actor** نوشته می شود.



شناسایی use case ها

موارد استفاده: (Use Cases)

1. جستجو و نمایش محصولات:

- توضیح: مشتری از طریق موتور جستجو و فیلترهای پیشرفته، محصولات مورد نظر خود را جستجو می کند.
- نقش هوش مصنوعی: ارائه پیشنهادات اولیه بر اساس سابقه جستجو و خرید مشتری.

2. پیشنهاد محصولات شخصی سازی شده:

- توضیح: پس از ثبت نام یا ورود به سیستم، مشتری براساس تاریخچه فعالیت هایش، پیشنهادات ویژه دریافت می کند.
- نقش هوش مصنوعی: تحلیل داده های رفتاری مشتریان و استفاده از الگوریتم های یادگیری ماشین برای پیشنهاد محصولات مطابق با سلیقه فردی.

3. خرید و پرداخت آنلاین:

- توضیح: مشتری محصولات مورد نظر خود را به سبد خرید اضافه کرده و از طریق سیستم پرداخت، تراکنش را نهایی می کند.
- نقش سیستم پرداخت: تأمین امنیت تراکنش ها و پردازش پرداخت ها به صورت آنلاین.

4. مدیریت سفارشات و پیگیری وضعیت سفارش:

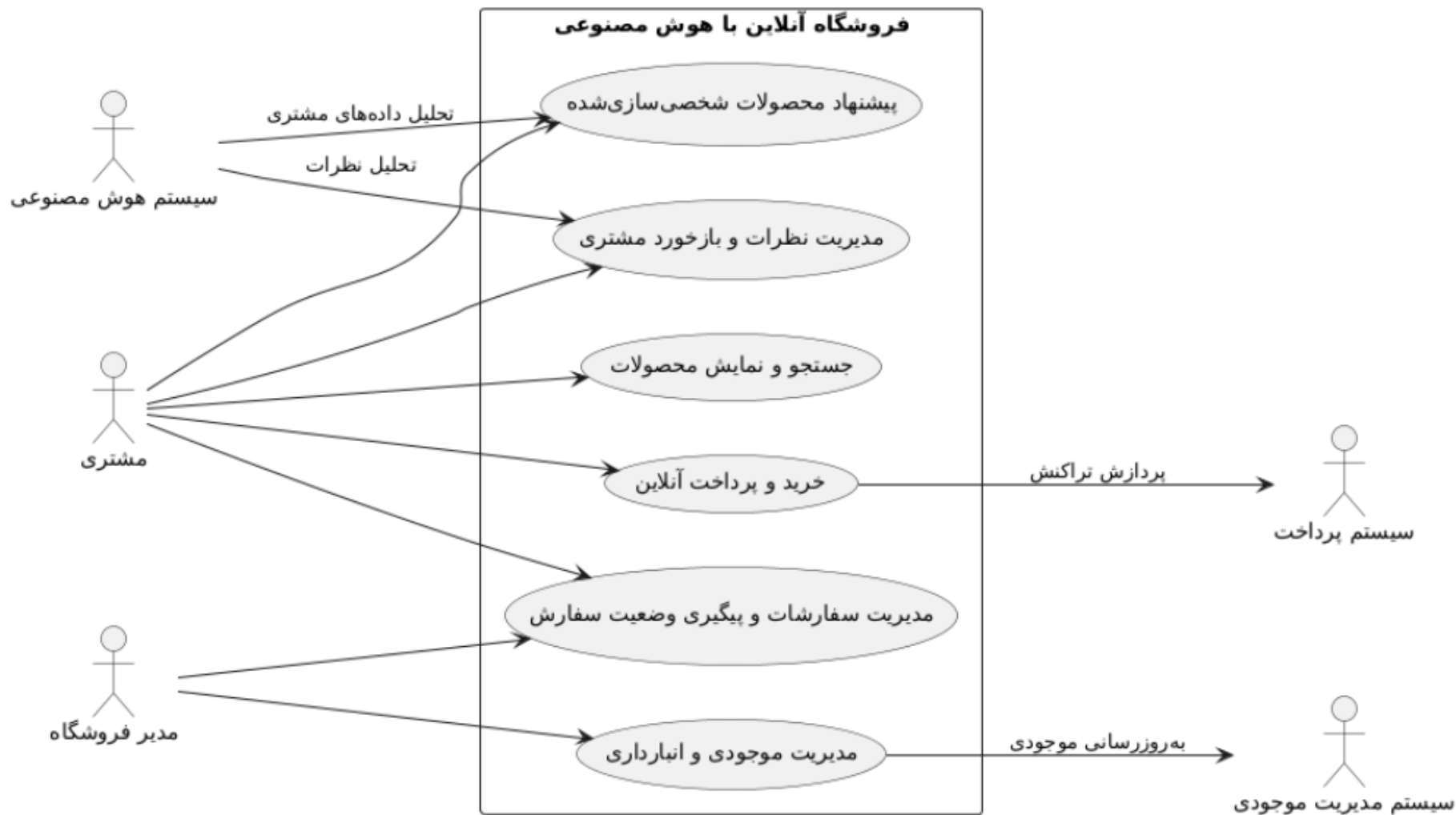
- توضیح: مشتری پس از خرید قادر به پیگیری وضعیت سفارش است؛ از ثبت سفارش تا تحویل نهایی.
- نقش مدیر فروشگاه: نظارت بر روند پردازش سفارش ها و هماهنگی با بخش های مختلف جهت تحویل به موقع.

5. مدیریت موجودی و انبارداری:

- توضیح: سیستم به صورت خودکار موجودی کالاها را به روزرسانی کرده و در صورت کاهش موجودی، هشدارهای لازم را به مدیر فروشگاه ارسال می کند.
- نقش سیستم مدیریت موجودی: کاهش خطاهای انسانی و بهبود روند سفارش دهی مجدد کالاها.

6. مدیریت نظرات و بازخورد مشتری:

- توضیح: مشتریان پس از خرید می توانند نظرات و امتیازات خود را ثبت کنند که به بهبود کیفیت محصولات و خدمات کمک می کند.
- نقش هوش مصنوعی: تحلیل بازخوردها برای شناسایی نقاط قوت و ضعف و ارائه گزارش های تحلیلی به مدیر فروشگاه.



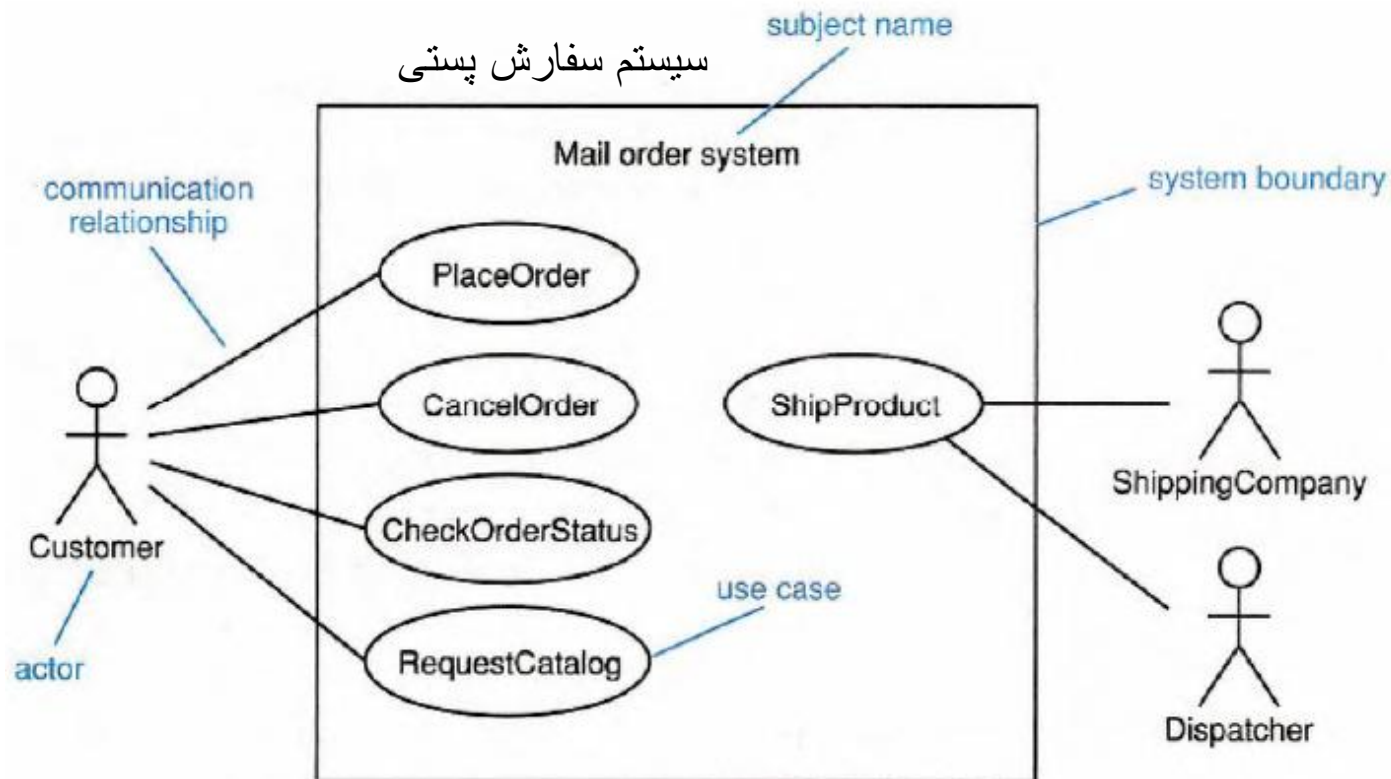
شناسایی use case ها

- بهترین راه برای شناسایی use case ها شروع با لیستی از Actor ها است و سپس بررسی اینکه چگونه هر Actor از سیستم استفاده می کند.
- هر use case با یک عبارت فعل ی مشخص می شود.
- شناسایی use case ها گاهی منجر به پیدا کردن Actor های جدید می شود.

شناسایی use case ها

- سوالاتی که در شناسایی use case ها به ما کمک می کنند :
 - چه عملکردی یک Actor از سیستم انتظار دارد؟
 - آیا سیستم اطلاعات را ذخیره و بازیابی می کند؟ اگر اینچنین است کدام Actor ها این عملیات را انجام می دهند؟
 - هنگامی که حالت سیستم تغییر می کند چه چیزی اتفاق می افتد ؟ (برای مثال شروع و پایان سیستم) آیا هیچ Actor ی نقش دارد؟
 - آیا رخدادهای خارجی هم روی سیستم تاثیر می گذارند؟
 - آیا سیستم با سیستم خارجی دیگری در تعامل است؟
 - آیا سیستم گزارشی تهیه می کند؟

Use case diagram



مشخصه use case

- نام use case: عبارت فعلی توصیفی و کوتاه
- شناسه use case
- توصیف اجمالی: یک پارگراف که مشخص کننده هدف use case است.
- Actorهایی که در use case هستند:
 - Primary actors: actorی که use case را راه اندازی می کند.
 - Secondary actors: بعد از اینکه use case شروع به کار کرد با آن در تعامل است.
- پیش شرط ها: چیزهایی که باید قبل از اینکه use case اجرا شود برقرار باشند – آنها محدودیت های روی حالت سیستم هستند.
- روند اصلی: مراحل use case
- پس شرط ها: چیزهایی که باید در پایان use case درست باشند.
- روند جایگزین: لیست جایگزین ها برای روند اصلی

مشخصه use case : مثال

use case name	Use case: PaySalesTax
use case identifier	ID: 1
brief description	Brief description: Pay Sales Tax to the Tax Authority at the end of the business quarter.
the actors involved in the use case	Primary actors: Time
	Secondary actors: TaxAuthority
the system state before the use case can begin	Preconditions: 1. It is the end of the business quarter.
the actual steps of the use case	Main flow: <i>implicit time actor</i> 1. The use case starts when it is the end of the business quarter. 2. The system determines the amount of Sales Tax owed to the Tax Authority. 3. The system sends an electronic payment to the Tax Authority.
the system state when the use case has finished	Postconditions: 1. The Tax Authority receives the correct amount of Sales Tax.
alternative flows	Alternative flows: None.

مشخصه use case : مثال

مورد کاربرد: فروش کتاب
شماره: ۶
توصیف اجمالی: فروشنده کتاب اطلاعات فروش انجام شده را وارد می کند و سامانه موجودی کتاب ها را بهنگام می کند.
عامل اصلی: فروشنده
عامل فرعی: ندارد
شرایط اولیه: فروشنده باید وارد سامانه شده باشد.
روند اصلی: ۱. این مورد کاربرد وقتی آغاز می شود که فروشنده بخواهد، اطلاعات یک فروش را در سامانه وارد کند. ۲. شامل: تهیه لیست ۳. سامانه تاریخ فروش انجام شده را از فروشنده می خواهد. ۴. فروشنده تاریخ فروش انجام شده را وارد می کند. ۵. فروشنده اطلاعات فروش را تأیید می کند. ۶. سامانه تعداد درخواست شده کتاب ها را با موجودی کتابفروشی مقایسه می کند. ۷. سامانه اطلاعات خرید را ثبت می کند و موجودی را به هنگام می کند.
شرایط نهایی: موجودی تعدادی از کتاب ها کاهش یافته اند.
روند جایگزین: نداشتن موجودی کافی

Use case : جریان ها

- مراحل در use case به صورت جریانی از رخدادها لیست می شود
- هر use case یک روند اصلی (main flow) دارد، که مراحل یک use case را وقتی همه چیز همانطور که انتظار داریم اتفاق می افتد و هیچ خطا، وقفه یا مشکلی وجود ندارد، نشان می دهد.
- روند فرعی (alternative flow): انحراف ها از جریان اصلی که منجر به خطا، شاخه های دیگر یا وقفه در روند اصلی می شوند.
- روند اصلی همیشه با یک عامل اصلی (primary actor) اتفاق می افتد

Use case : جریان های فرعی

- معمولاً به روند اصلی برنمی گردند زیرا اغلب به خطاها و استثنای روند اصلی رسیدگی کرده و پس شرط های متفاوتی دارند.
- بهتر است به طور جداگانه مستند شوند.
- ممکن است به سه حالت متخلف باشند :
- بجای روند اصلی : توسط **Actor** اصلی صدازده شده و جایگزین **use case** می شوند.
- بعد از یک مرحله خاص در روند اصلی اتفاق می افتد
- در هر زمانی در روند اصلی اتفاق می افتد.

Use case : مثال جریان های فرعی

روند جایگزین: فروش کتاب : نداشتن موجودی کافی کتاب

شماره: ۶.۱

توصیف اجمالی: سامانه به فروشنده اطلاع می دهد که تعدادی از کتاب های لیست مورد نظر به اندازه تقاضا شده موجودی ندارند و همچنین آن کتاب ها را به لیست کتاب های تقاضا شده اضافه می کند.

عامل اصلی: فروشنده

عامل فرعی: ندارد

شرایط اولیه: تعداد تقاضا شده حداقل یک کتاب در لیست از موجودی آن کتاب کمتر باشد.

روند جایگزین:

۱. روند جایگزین بعد از اتمام مرحله ۶ روند اصلی ، امکان وقوع دارد.
۲. سامانه به فروشنده کتاب هایی را که از آنها به تعداد مورد نیاز نداریم را اعلام می کند.
۳. سامانه کتاب های مورد تقاضا را ثبت می کند.

شرایط نهایی: ندارد

Use case : پیدا کردن جریان های فرعی

- جریان های فرعی را با بررسی جریان اصلی پیدا کنید. در هر مرحله در جریان اصلی موارد زیر را جستجو کنید:
 - انتخاب های ممکن برای روند اصلی
 - خطاهایی که ممکن است در روند اصلی اتفاق افتد.
 - وقفه هایی که ممکن است در یک نقطه خاص اتفاق افتد.
 - وقفه هایی که ممکن است هر لحظه اتفاق افتد.

Use case : مثال جریان های فرعی

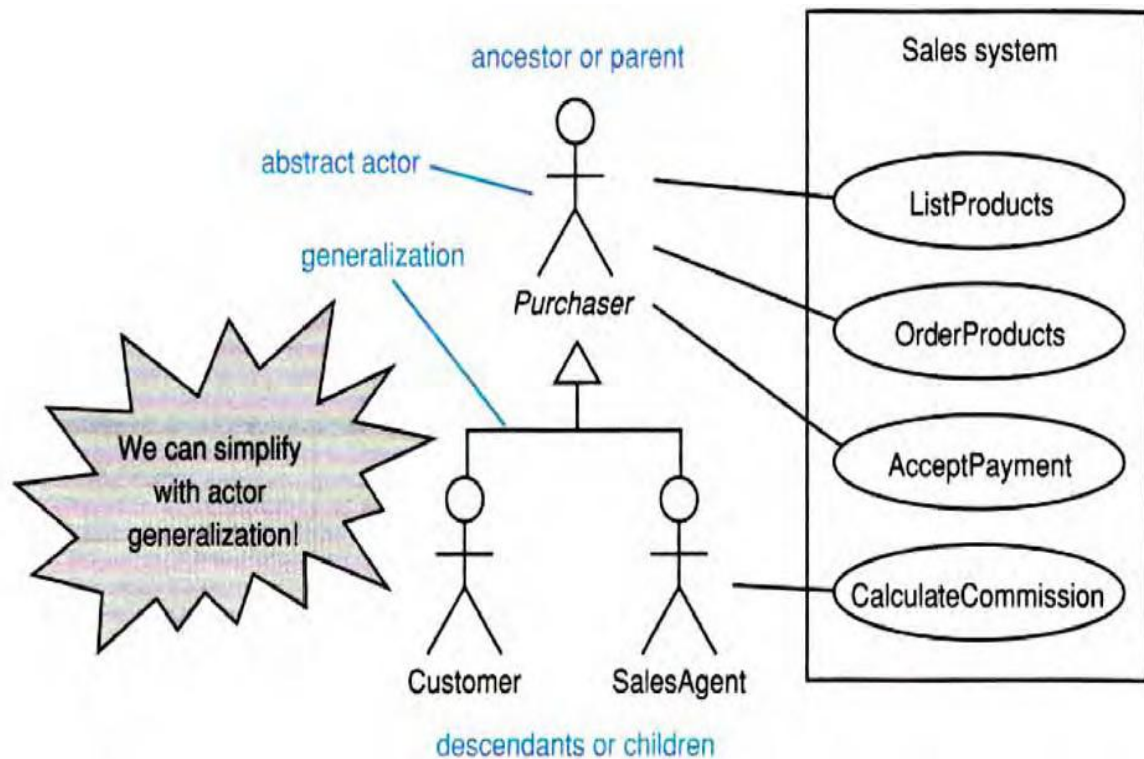
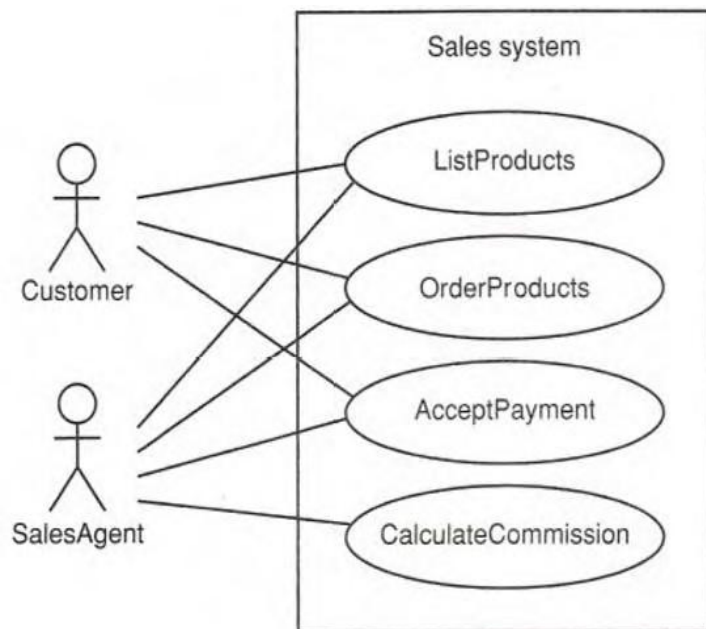
روند جایگزین: فروش کتاب : نداشتن موجودی کافی کتاب
شماره: ۶.۱
توصیف اجمالی: سامانه به فروشنده اطلاع می دهد که تعدادی از کتاب های لیست مورد نظر به اندازه تقاضا شده موجودی ندارند و همچنین آن کتاب ها را به لیست کتاب های تقاضا شده اضافه می کند.
عامل اصلی: فروشنده
عامل فرعی: ندارد
شرایط اولیه: تعداد تقاضا شده حداقل یک کتاب در لیست از موجودی آن کتاب کمتر باشد.
روند جایگزین: ۱. روند جایگزین بعد از اتمام مرحله ۶ روند اصلی ، امکان وقوع دارد. ۲. سامانه به فروشنده کتاب هایی را که از آنها به تعداد مورد نیاز نداریم را اعلام می کند. ۳. سامانه کتاب های مورد تقاضا را ثبت می کند.
شرایط نهایی: ندارد

Relationships

- **actor generalization**: رابطه بین یک **Actor** کلی و یک **Actor** جزئی
- **Use case generalization**: رابطه بین یک **use case** کلی و یک **use case** جزئی – **use-case** فرزند رفتار پدر را به ارث می برد.
- **<<include>>**: رابطه بین **use case** ها که یک **use case** از رفتار **use case** دیگر استفاده می کند.
- **<<extend>>**: رابطه بین **use case** ها که یک **use-case** (**base use-case**) به طور ضمنی از رفتار **use-case** دیگر استفاده می کند. **Base use-case** ممکن است مستقل باشد، اما تحت شرایط خاص ممکن است گسترش داده شود.

Actor generalization

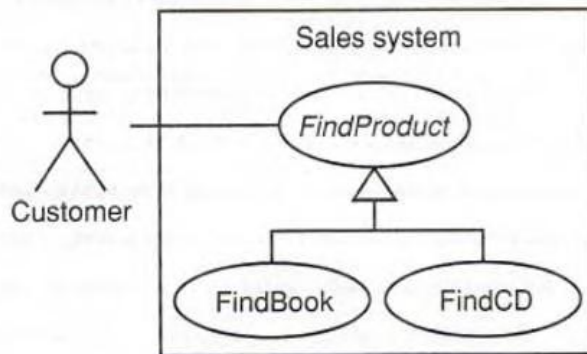
■ اگر دو Actor با هم از طریق مجموعه یکسانی از use case ها در ارتباط باشند



Use case generalization

- هنگامی استفاده می شود که یک یا چند use case حالت خاص یک نمونه کلی هستند.
- Use case های فرزند ممکن است:
 - ویژگی هایی را از use case پدر به ارث ببرند.
 - ویژگی های جدیدی اضافه کنند
 - ویژگی های ارث برده را تغییر دهند.

use case generalization مثال

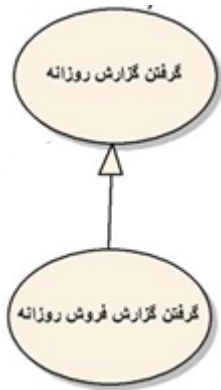


Use case: <i>FindProduct</i>
ID: 6
Brief description: The Customer searches for a product.
Primary actors: Customer
Secondary actors: None.
Preconditions: None.
Main flow: 1. The Customer selects "find product". 2. The system asks the Customer for search criteria. 3. The Customer enters the requested criteria. 4. The system searches for products that match the Customer's criteria. 5. If the system finds some matching products 5.1 The system displays a list of the matching products. 6. Else 6.1 The system tells the Customer that no matching products could be found.
Postconditions: None.
Alternative flows: None.

overridden
 overridden
 inherited without change
 overridden
 overridden
 added
 overridden and renumbered
 added
 added
 inherited without change
 added
 renumbered

Use case: FindBook
ID: 7
Parent ID: 6
Brief description: The Customer searches for a book.
Primary actors: Customer
Secondary actors: None.
Preconditions: None.
Main flow: 1. (o1.) The Customer selects "find book". 2. (o2.) The system asks the Customer for book search criteria comprising author, title, ISBN, or topic. 3. (3.) The Customer enters the requested criteria. 4. (o4.) The system searches for books that match the Customer's criteria. 5. (o5.) If the system finds some matching books 5.1 The system displays the current best seller. 5.2 (o5.1) The system displays details of a maximum of five books. 5.3 For each book on the page the system displays the title, author, price, and ISBN. 5.4 While there are more books, the system gives the Customer the option to display the next page of books. 6. (6.) Else 6.1 The system displays the current best seller. 6.2 (6.1) The system tells the Customer that no matching products could be found.
Postconditions: None.
Alternative flows: None.

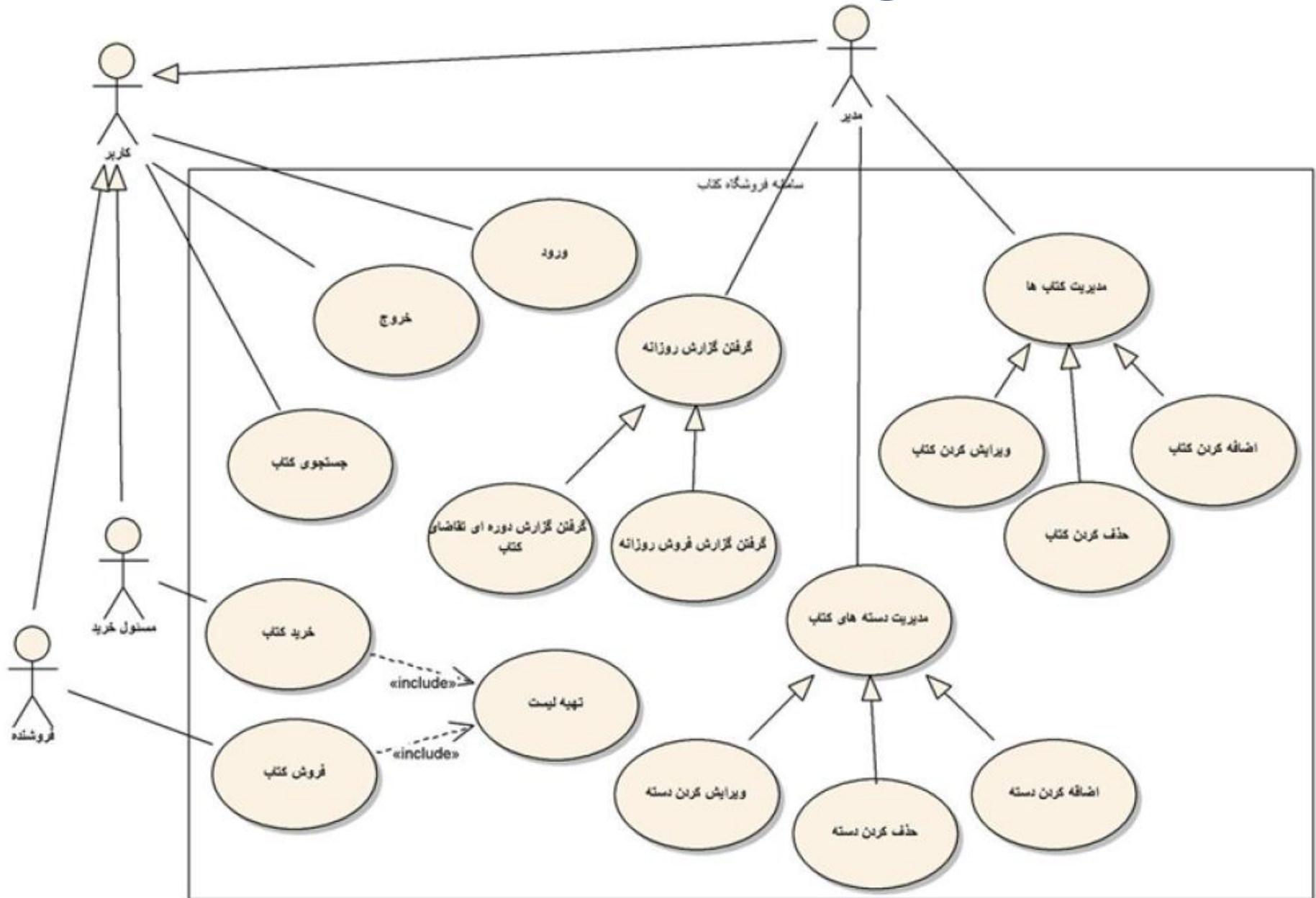
مثال use case generalization



مورد کاربرد: گرفتن گزارش روزانه
شماره: ۷
توصیف اجمالی: مدیر می تواند بر روی اطلاعات مشخصی در سامانه گزارش روزانه تهیه کند.
عامل اصلی: مدیر

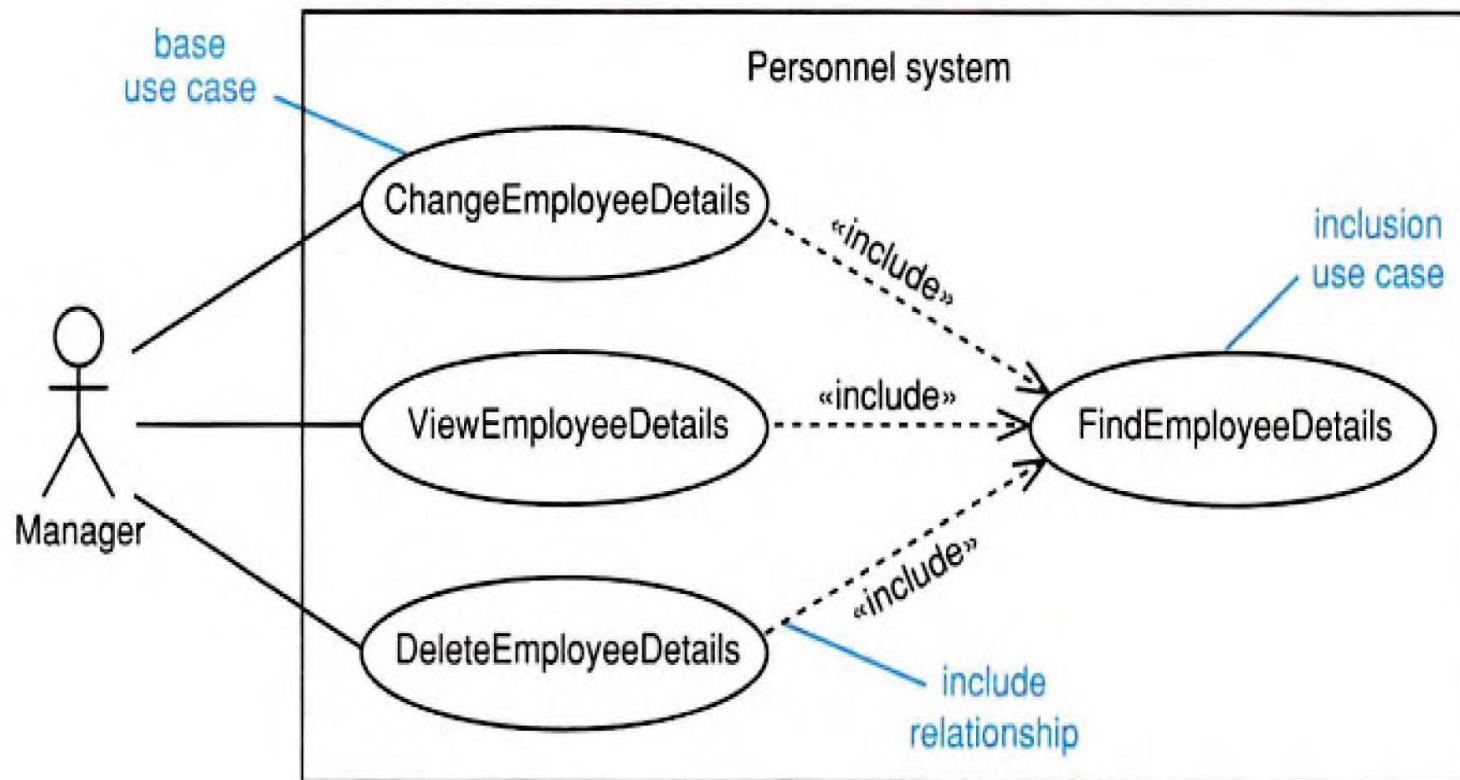
مورد کاربرد: گرفتن گزارش روزانه فروش	عامل فرعی: ندارد
شماره: ۸	شرایط اولیه: مدیر باید وارد سامانه شده باشد.
شماره پدر: ۷	روند اصلی:
توصیف اجمالی: مدیر می تواند بر روی اطلاعات فروش ثبت شده در سامانه گزارش روزانه تهیه کند.	۱. این مورد کاربرد وقتی آغاز می شود که مدیر بخواهد از سامانه گزارش روزانه بگیرد.
عامل اصلی: مدیر	۲. سامانه از مدیر، تاریخ جهت گزارش گیری را سوال می کند.
عامل فرعی: ندارد	۳. مدیر تاریخ مورد نظر خود را وارد می کند.
شرایط اولیه: مدیر باید وارد سامانه شده باشد.	۴. مدیر درخواست مشاهده گزارش را می دهد.
روند اصلی:	۵. سامانه گزارش مورد نظر مدیر را به او نشان می دهد.
۱. (01) این مورد کاربرد وقتی آغاز می شود که مدیر بخواهد، از سامانه گزارش روزانه فروش بگیرد.	شرایط نهایی: ندارد
۲. (۲) سامانه از مدیر، تاریخ جهت گزارش گیری را سوال می کند.	روند جایگزین: ندارد
۳. (۳) مدیر تاریخ مورد نظر خود را وارد می کند.	
۴. (۴) مدیر درخواست مشاهده گزارش را می دهد.	
۵. (05) سامانه گزارش فروش روزانه را به مدیر نشان می دهد.	
شرایط نهایی: ندارد	
روند جایگزین: ندارد	

مثال use case diagram



رابطه <<INCLUDE>>

- رابطه <<include>> بین use case ها به شما اجازه می دهد تا رفتار یک use case را به جریان use case دیگر اضافه نمایید.



رابطه <<INCLUDE>> (مشخصه)

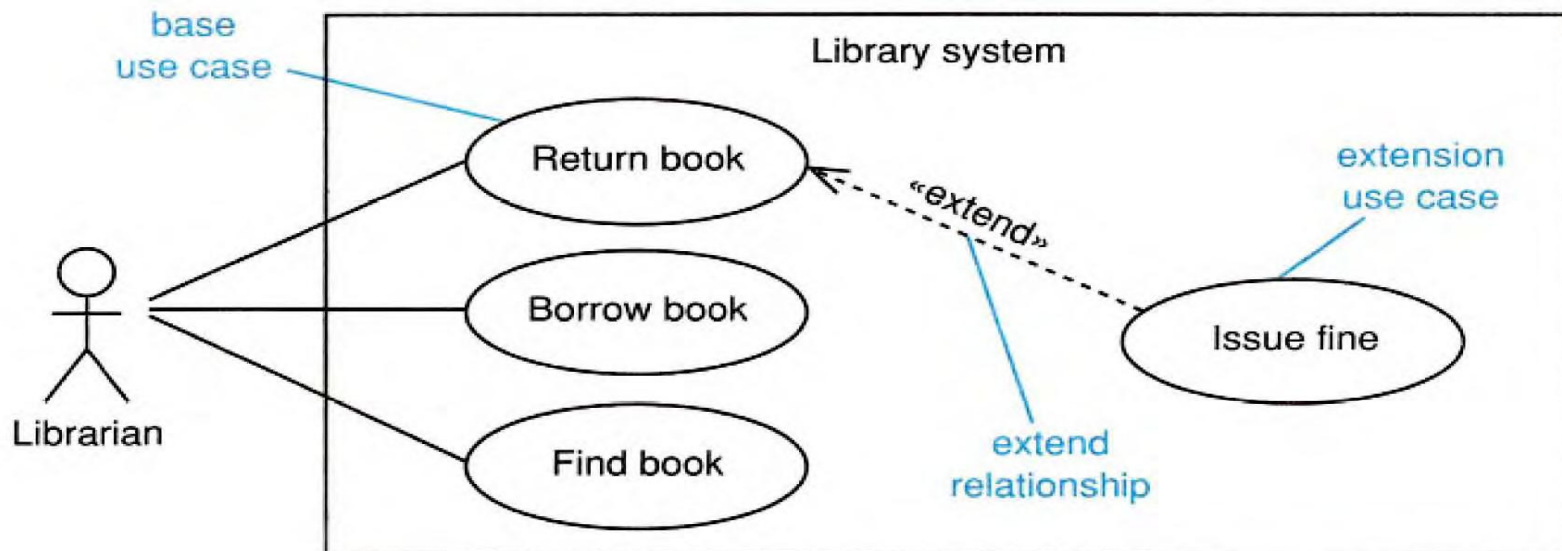
- Use case پایه بدون اضافه کردن use case دیگر کامل نیست. Use case include شده می تواند کامل باشد یا نباشد.

Use case: ChangeEmployeeDetails
ID: 1
Brief description: The Manager changes the employee details.
Primary actors: Manager
Secondary actors: None.
Preconditions: 1. The Manager is logged on to the system.
Main flow: 1. include(FindEmployeeDetails). 2. The system displays the employee details. 3. The Manager changes the employee details. ...
Postconditions: 1. The employee details have been changed.
Alternative flows: None.

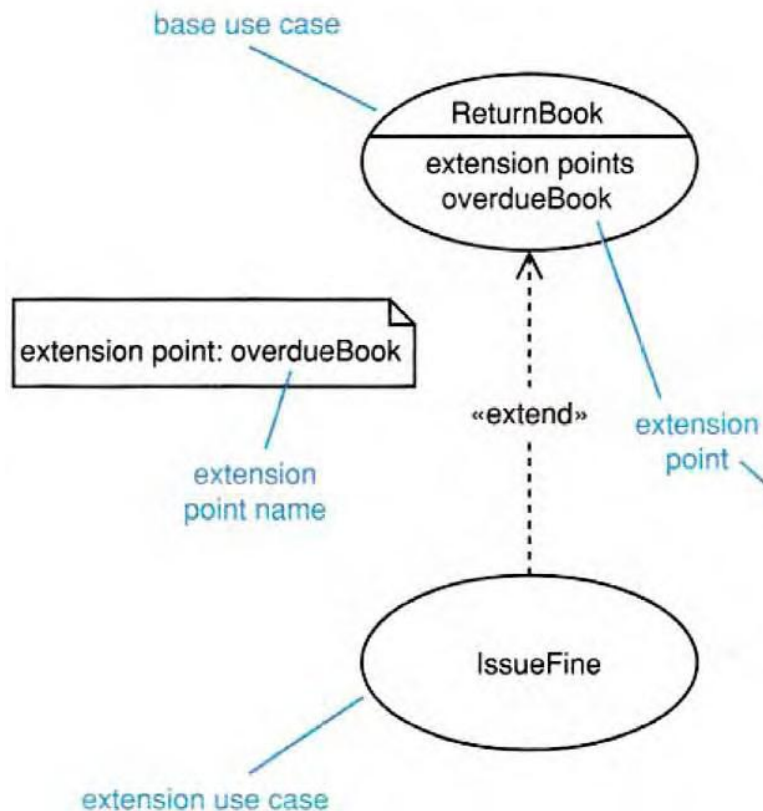
Use case: FindEmployeeDetails
ID: 4
Brief description: The Manager finds the employee details.
Primary actors: Manager
Secondary actors: None.
Preconditions: 1. The Manager is logged on to the system.
Main flow: 1. The Manager enters the employee's ID. 2. The system finds the employee details.
Postconditions: 1. The system has found the employee details.
Alternative flows: None.

رابطه <<extend>>

- راهی برای اضافه کردن رفتار جدید به use case موجود فراهم می کند.
- Use case پایه ، کامل است و use case اضافه شده مجموعه ای از قطعات اضافی است که می توانند به use case اضافه شوند.



رابطه <<extend>> (مثال)

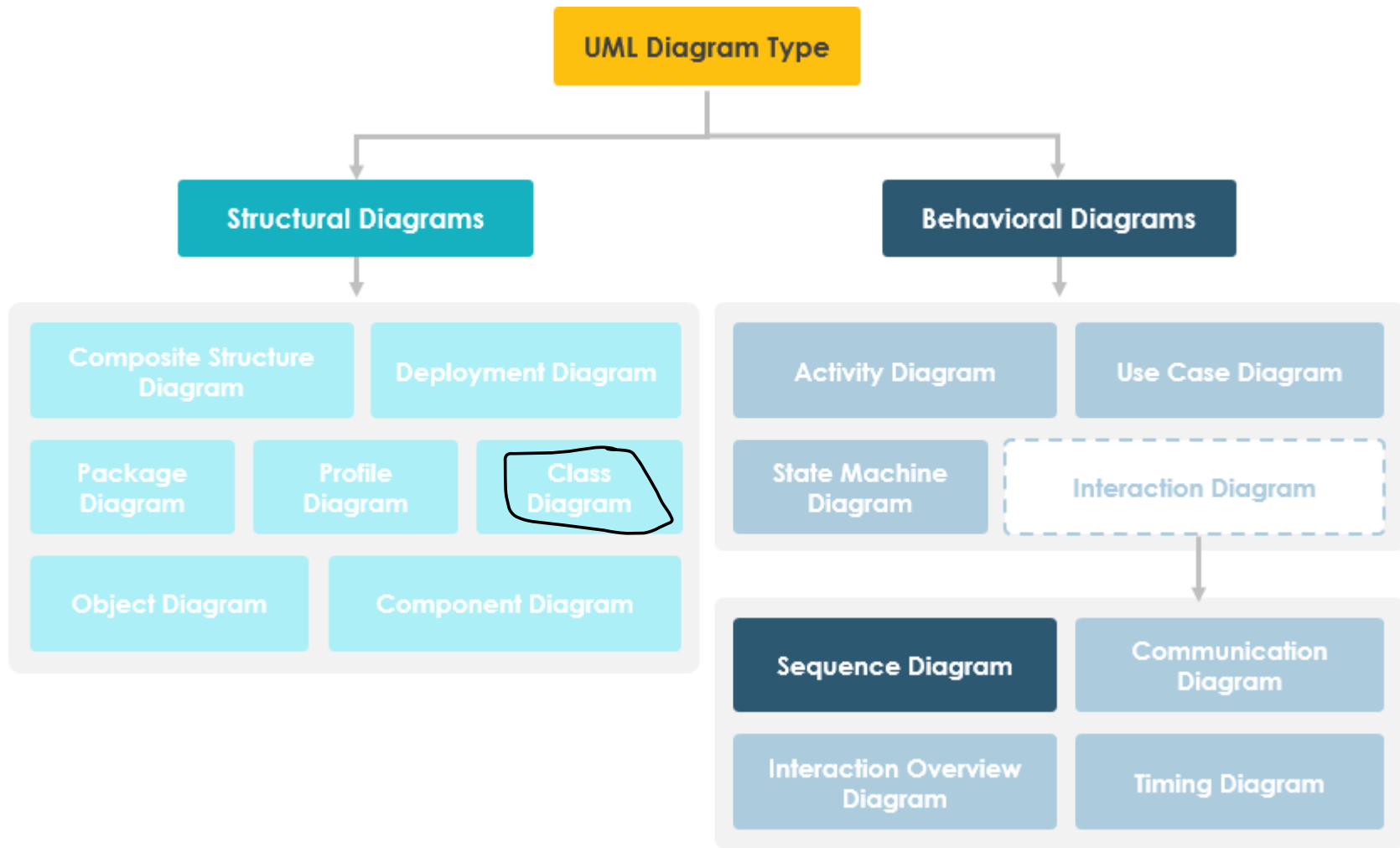


Use case: ReturnBook
ID: 9
Brief description: The Librarian returns a borrowed book.
Primary actors: Librarian
Secondary actors: None.
Preconditions: 1. The Librarian is logged on to the system.
Main flow: 1. The Librarian enters the borrower's ID number. 2. The system displays the borrower's details including the list of borrowed books. 3. The Librarian finds the book to be returned in the list of books. extension point: overdueBook 4. The Librarian returns the book. ...
Postconditions: 1. The book has been returned.
Alternative flows: None.

تفاوت‌های کلیدی بین روابط `<<include>>` و `<<extend>>`

Extending Use Case	Included Use Case	
خیر	بله	این use case اجباری است؟
بله	خیر	آیا use case اولیه بدون این use case کامل است؟
بله	خیر	آیا اجرای این use case شرطی است؟
بله	خیر	آیا این use case رفتار use case اولیه را تغییر می دهد؟

Class diagram ■



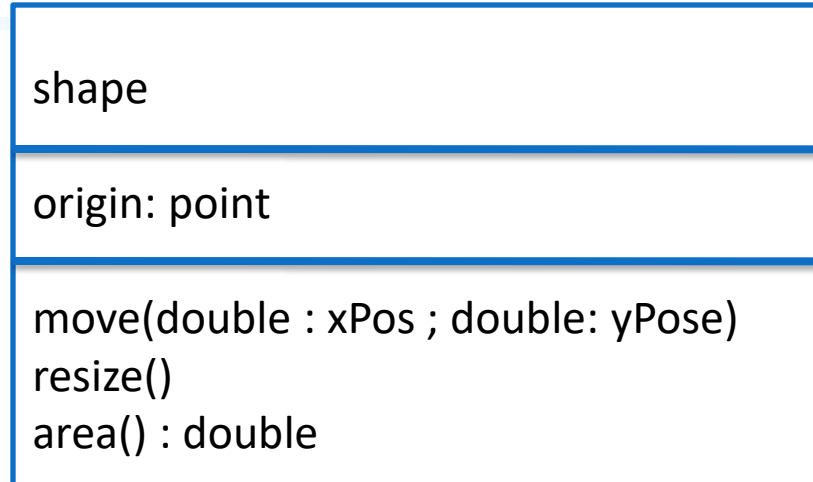
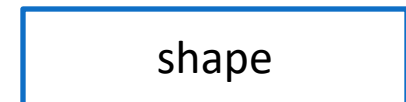
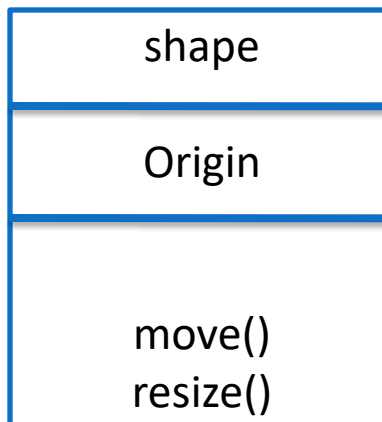
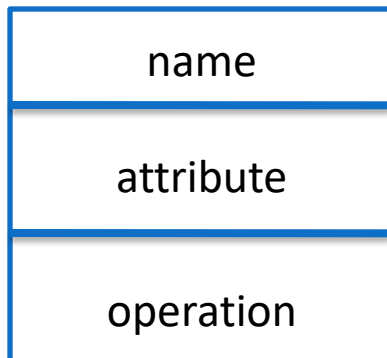
کلاس چیست؟

- اشیائی که ساختار و رفتار مشترک دارند در یک کلاس قرار می گیرند.
- کلاس ها انتزاعی از اشیائی هستند که در زمان و فضا وجود دارند. - کلاس ها و اشیا به یکدیگر متصل هستند.

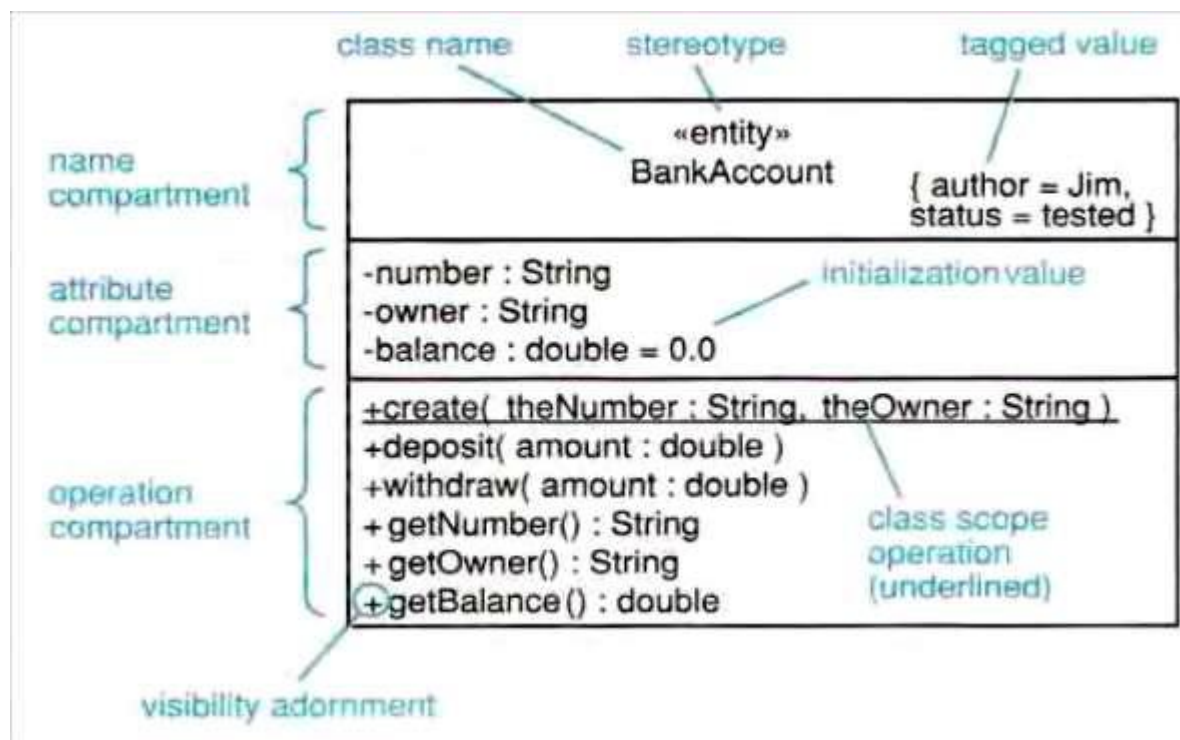
کلاس ها (ادامه...)

Semantic

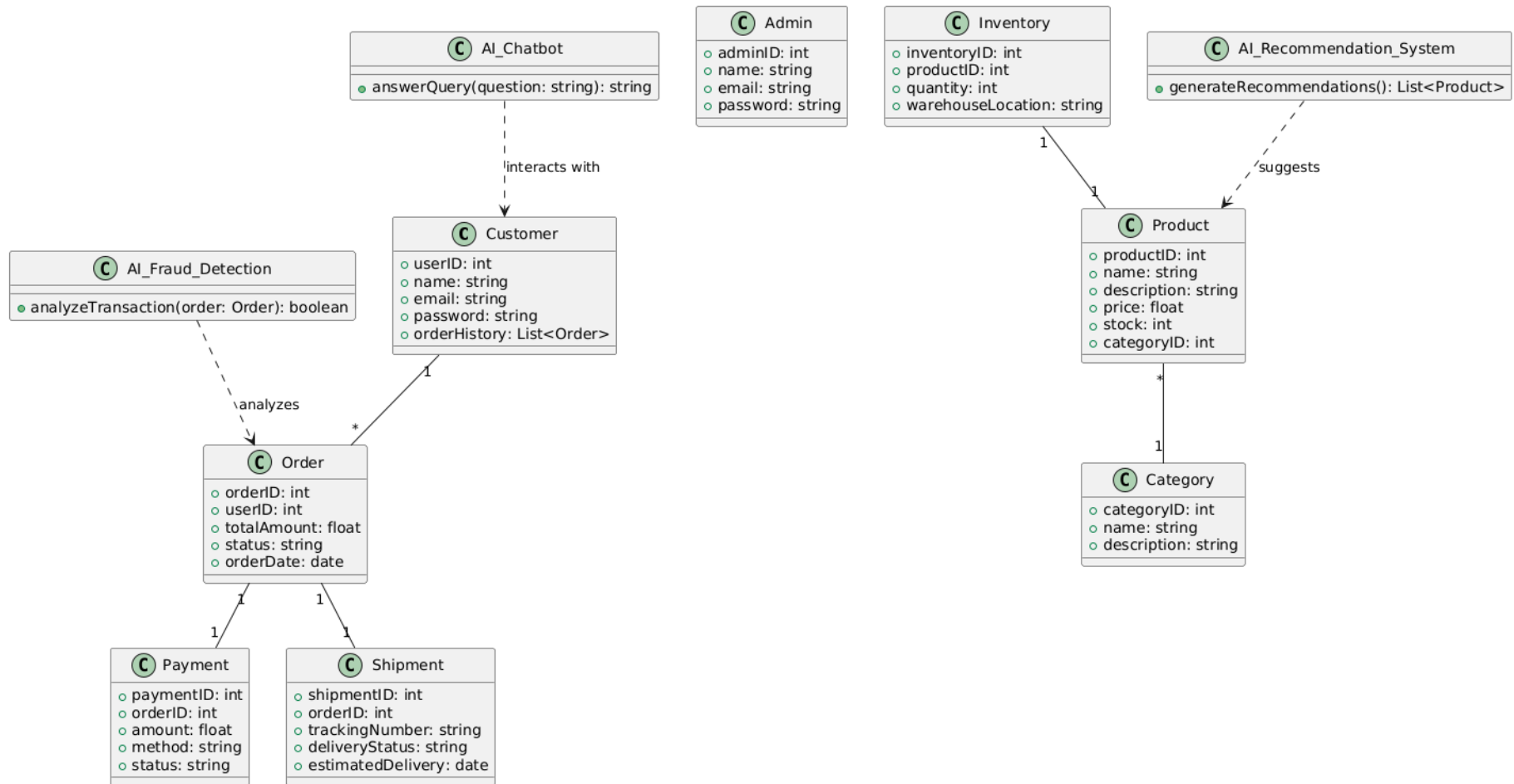
- Name
- Attributes
- operations
- نمادهای کلاس



نشانه گذاری کلاس در UML



online shop enhanced with AI features



روابط کلاس ها و اشیاء

■ رابطه بین دو شیء فرضیاتی است که هر کدام نسبت به هم دارند. عملیات و نتایج مورد انتظار بسیار مهم هستند.

■ روابط :

■ انجمنی (association)

■ وابستگی (dependency)

■ تعمیم / تخصصی کردن (generalization/specialization)

■ تجمعی (Aggregation)

رابطه انجمنی (association relationship)

- رابطه ای که در آن یک شی به شی دیگر متصل می شود.
- اگر بین دو شی لینکی وجود دارد باید یک رابطه Association یا Dependency بین کلاسهای این اشیا وجود داشته باشد.

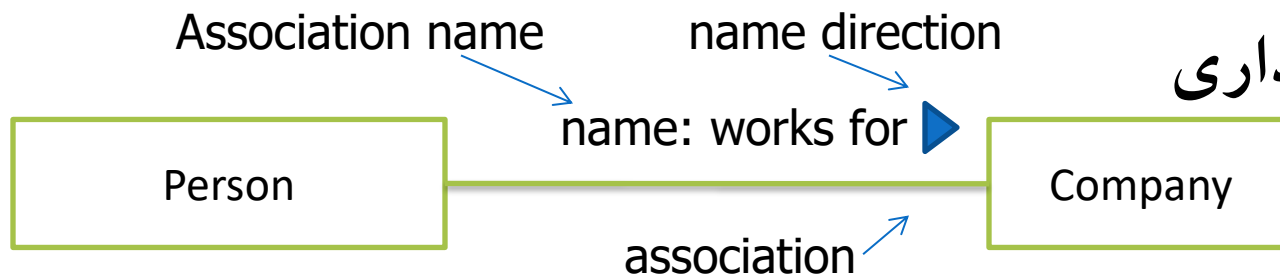
■ نام (name)

■ نقش (role)

■ چندگانگی (multiplicity): چندی رابطه بین دو شی را نشان می دهد. یک به یک ، یک به چند، چند به چند

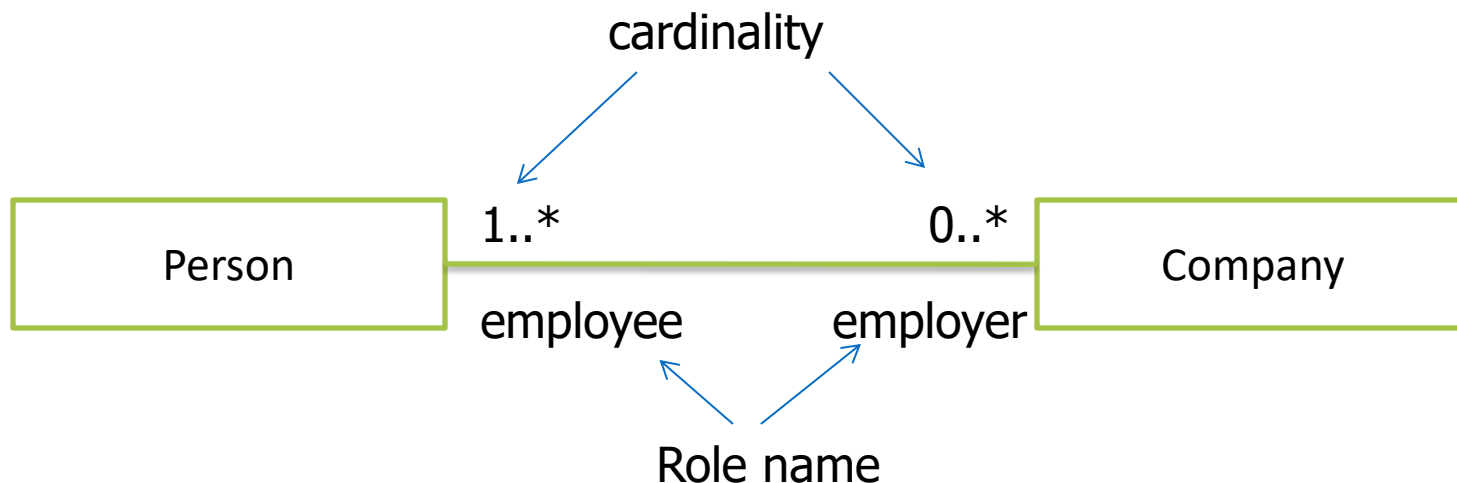
■ جهت (direction)

■ نمادگذاری



رابطه انجمنی (ادامه...)

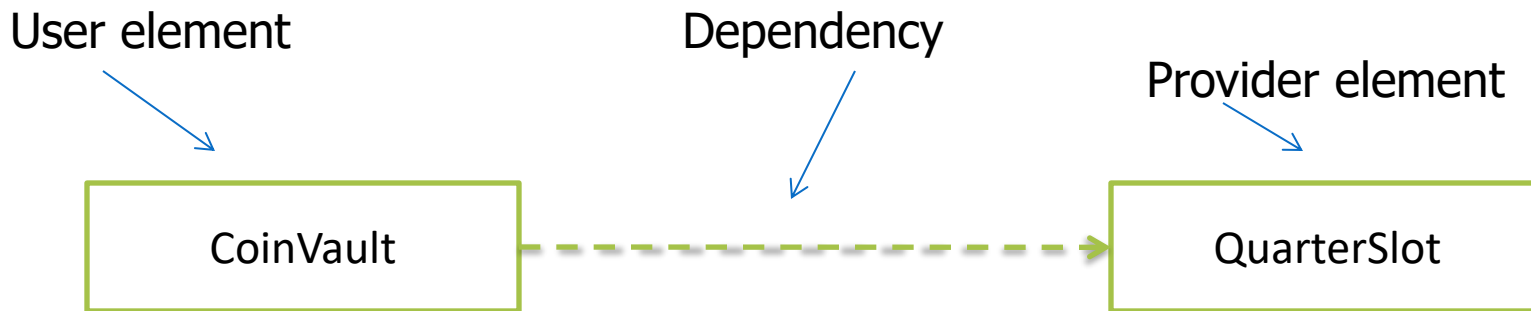
- نام **Association**: باید یک فعل یا عبارت باشد
- نام نقش :
- باید یک اسم یا عبارت اسمی باشد که توصیف کننده نقش است.
- چندی: نشان دهنده تعداد اشیائی است که می توانند در رابطه وجود داشته باشند.



رابطه وابستگی (Dependency Relationship)

- این رابطه نشان می دهد که یک تغییر در **provider** **element** نیاز به تغییری در **user element** دارد. معمولاً رابطه وابستگی نشان می دهد که یک کلاس (**user**) از یک کلاس دیگر (**provider**) به عنوان آرگومانی در امضای یکی از توابع اش استفاده می کند.

نماد:



CoinVault از QuarterSlot استفاده می کند

رابطه وابستگی (Dependency Relationship)

- این رابطه نشان می دهد که یک تغییر در **provider** **element** نیاز به تغییری در **user element** دارد. معمولاً رابطه وابستگی نشان می دهد که یک کلاس (**user**) از یک کلاس دیگر (**provider**) به عنوان آرگومانی در امضای یکی از توابع اش استفاده می کند.
- نماد:

رابطه وابستگی (Dependency Relationship)

```
class Document {
    String content;

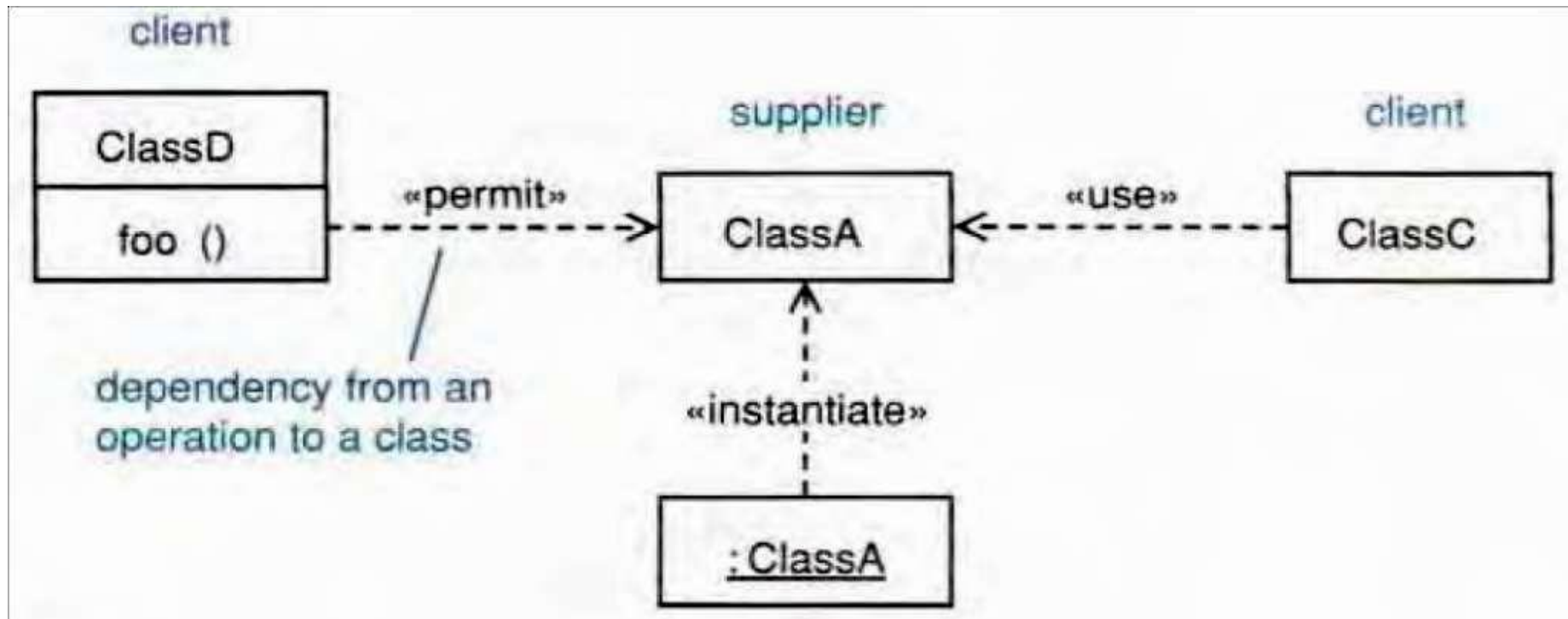
    Document(String content) {
        this.content = content;
    }

    String getContent() {
        return content;
    }
}

class Printer {
    void print(Document doc) {
        System.out.println("Printing: " + doc.getContent());
    }
}

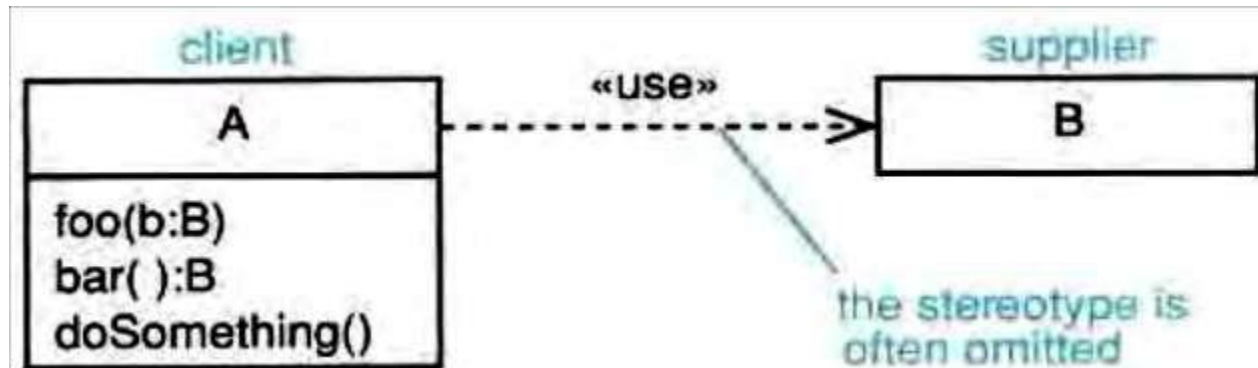
public class Main {
    public static void main(String[] args) {
        Document myDoc = new Document("Hello, UML!");
        Printer myPrinter = new Printer();
        myPrinter.print(myDoc);
    }
}
```

رابطه وابستگی (Dependency Relationship)

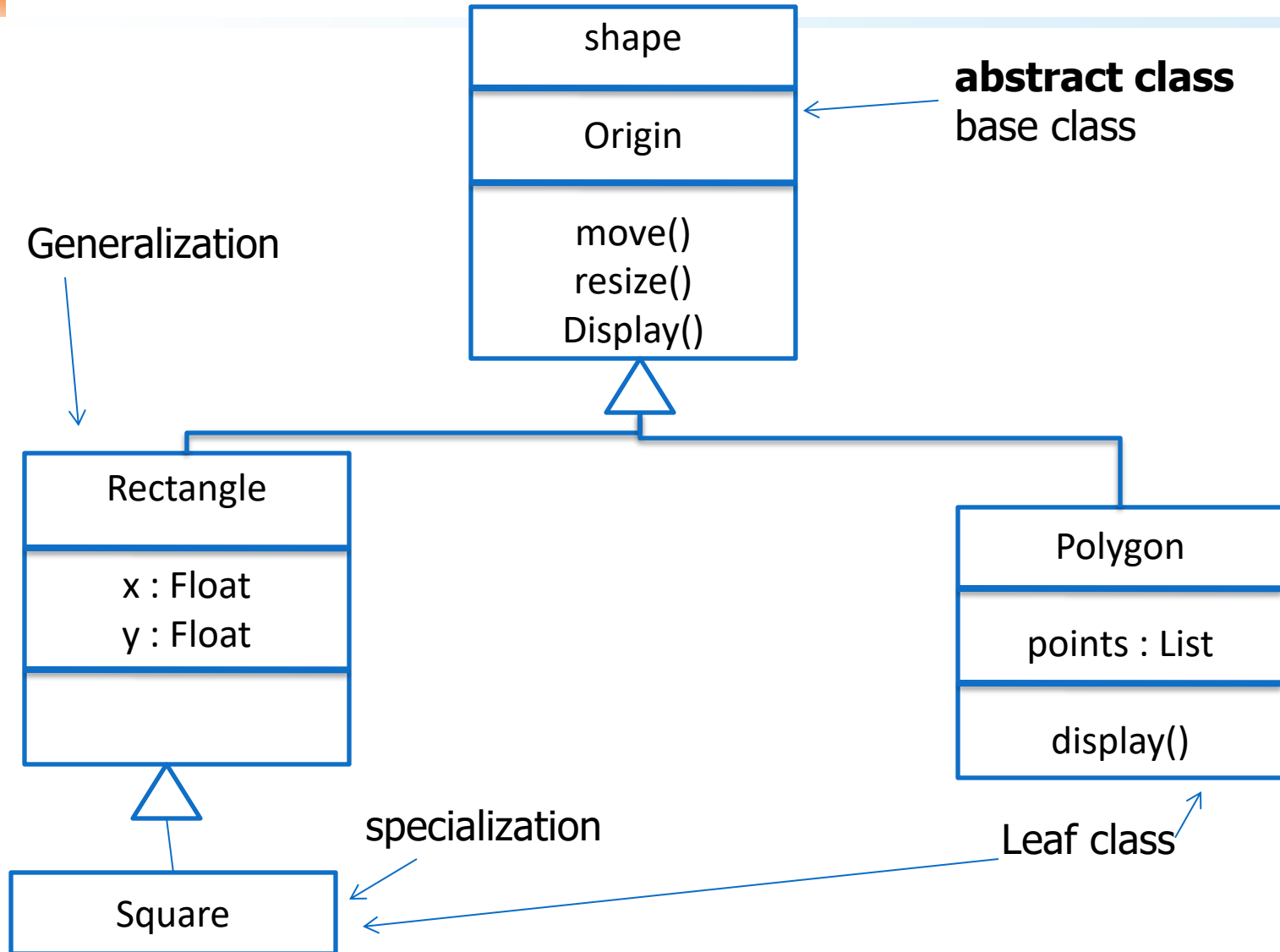


کاربرد وابستگی

- **<<use>>** - کلاس کلاینت از **supplier** استفاده می کند.
- **<<call>>** - تابع کلاینت ، تابع **supplier** را صدا میزند.
- **<<parameter>>** - **supplier** یک پارامتر یا مقدار برگشتی یکی از توابع کلاینت است.
- **<<send>>** - کلاینت یک سیگنال به **supplier** می فرستد.
- **<<instantiate>>** - کلاینت نمونه ای از **Supplier** است.



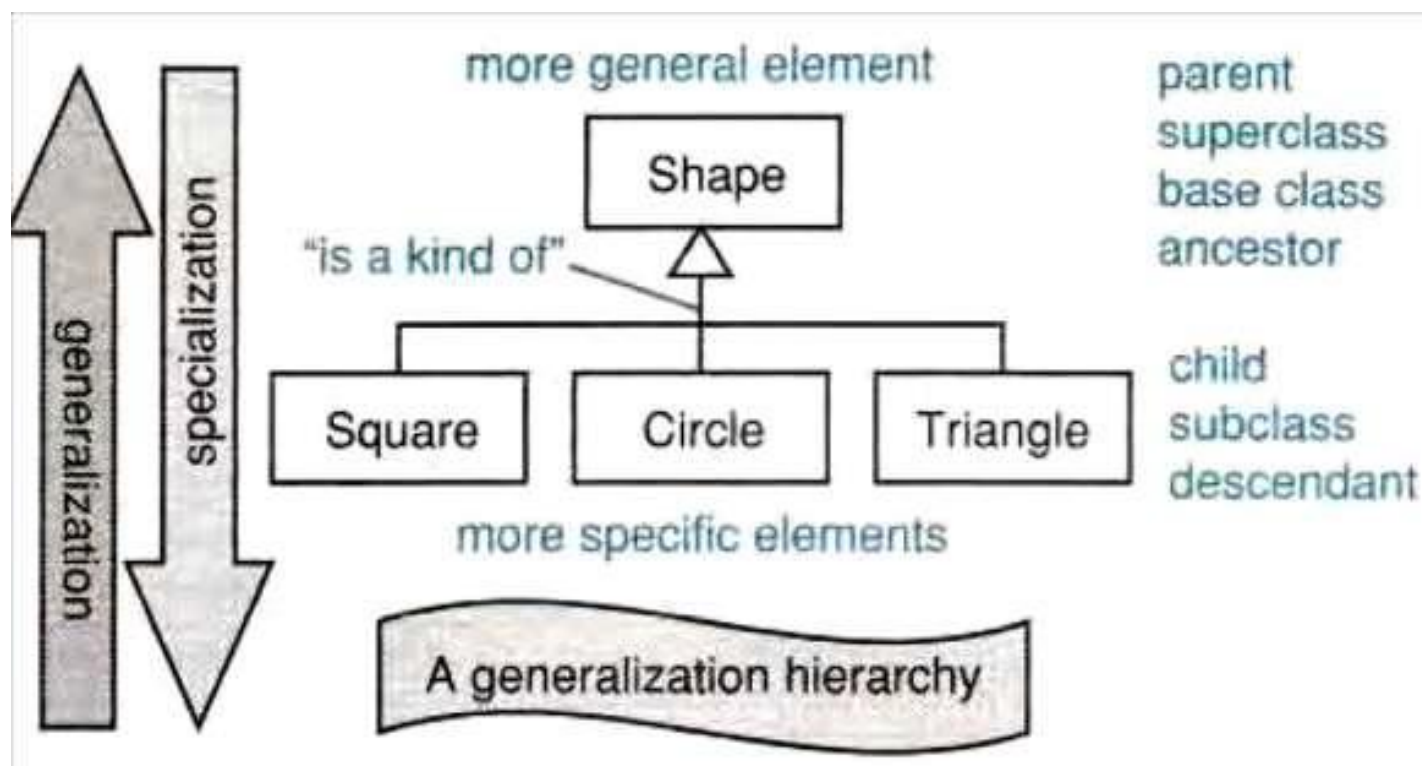
رابطه وارث یگانه (single inheritance)



Generalization/specialization

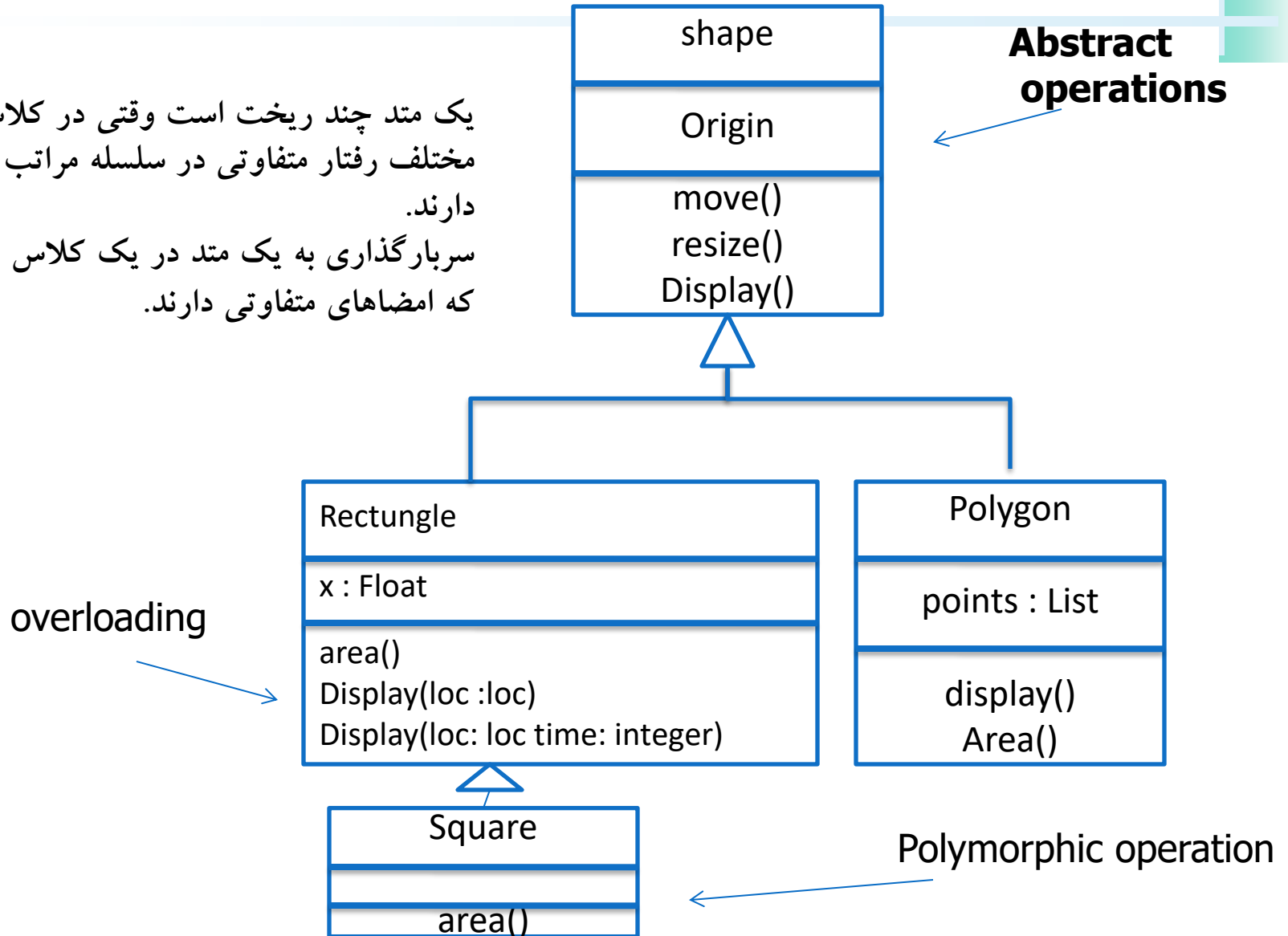
تعمیم (حرکت به سمت بالا): فرآیندی است که در آن ویژگی‌های مشترک دو یا چند کلاس استخراج شده و یک کلاس فوق‌العاده عمومی‌شده ایجاد می‌شود.

تخصص‌سازی (حرکت به سمت پایین): فرآیندی است که در آن زیرکلاس‌های جدید از یک کلاس موجود با افزودن ویژگی‌ها یا رفتارهای خاص ایجاد می‌شوند.

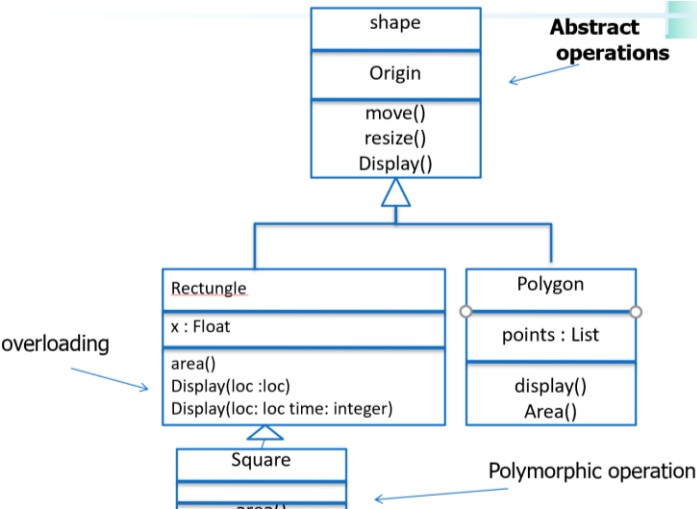


Polymorphism and Overloading

یک متد چند ریخت است وقتی در کلاسهای مختلف رفتار متفاوتی در سلسله مراتب کلاسها دارند.
سربارگذاری به یک متد در یک کلاس اشاره میکند که امضاهای متفاوتی دارند.



Polymorphism and Overloading



تفاوت چندریختی (Polymorphism) و تابع چندبارگی (Overloading)

1. چندریختی: (Polymorphism)

- وقتی کلاس‌های فرزند متدهای انتزاعی کلاس والد را **بازنویسی (Override)** می‌کنند و هر کدام رفتار مخصوص به خودشان را پیاده‌سازی می‌کنند. برای مثال، `area()` در مستطیل، چندضلعی یا مربع به شکل‌های متفاوتی پیاده‌سازی می‌شود؛ در نتیجه هنگام صدا زدن `area()` روی یک آبجکت از جنس **Shape**، بسته به نوع واقعی آن (`Rectangle`, `Polygon`, `Square`) پیاده‌سازی متفاوتی اجرا می‌شود.

2. چندبارگی توابع: (Overloading)

- وقتی در یک کلاس واحد چند متد با نام یکسان اما امضای متفاوت تعریف می‌شوند. در این شکل، `Display(loc : loc)` و `Display(loc : loc, time : integer)` نمونه‌هایی از **Overloading** در کلاس **Rectangle** هستند. هر دو نامشان `Display` است، اما پارامترهای متفاوتی دارند.

Polymorphism پلی مورفیزم - چند ریختی)

تعریف:

چند ریختی به این معناست که یک تابع، متد یا شیء می تواند چند شکل مختلف داشته باشد. به عبارت دیگر، یک اینترفیس واحد می تواند رفتارهای مختلفی بسته به نوع شیء واقعی ارائه دهد.

```
class Animal:  
    def speak(self):  
        print("Animal sound")
```

```
class Dog(Animal):  
    def speak(self):  
        print("Woof!")
```

```
class Cat(Animal):  
    def speak(self):  
        print("Meow!")
```

```
def make_sound(animal: Animal):  
    animal.speak()
```

```
make_sound(Dog()) # خروجی: Woof!  
make_sound(Cat()) # خروجی: Meow!
```

Overriding بازنویسی تابع

- بازنویسی یعنی یک کلاس فرزند متدی را که از کلاس پدر به ارث برده بازتعریف کند تا رفتار خاص خود را داشته باشد. این کار فقط زمانی انجام می شود که متد در کلاس پایه موجود باشد و در کلاس مشتق با همان نام و امضا دوباره تعریف شود.

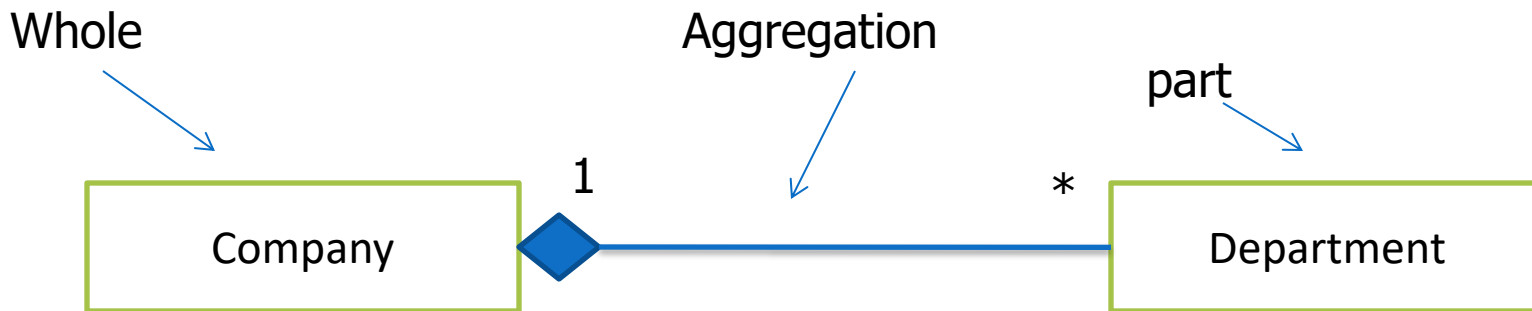
```
class Parent:  
    def greet(self):  
        print("Hello from Parent")
```

```
class Child(Parent):  
    def greet(self):  
        print("Hello from Child")
```

```
obj = Child()  
obj.greet() # خروجی: Hello from Child
```

رابطه تجمعی (Aggregation Relationship)

- معنایی : این رابطه کل (whole) را از بخش (part) جدا می کند. **aggregation** یک رابطه "has - a" است - یک شی از کل شامل اشیائی از بخش است.
- نماد:



نمودار کلاس (class diagram)

- مدل سازی واژگان (Vocabulary) و همکاری ها: (Collaborations)
این بخش به تعریف موجودیت ها و نحوه همکاری یا ارتباط آن ها می پردازد. به کمک نمودار کلاس می توانیم مفاهیم کلیدی (واژگان) و روابط متقابل (همکاری ها) را در سیستم شناسایی کنیم.
- ماهیت توصیفی نمودارهای کلاس:
این نمودارها برای مصورسازی (Visualization)، مشخص سازی (Specification)، مستندسازی (Documentation) و حتی ساخت سیستم های اجرایی از طریق فرایند مهندسی پیشرو (Forward Engineering) سودمند هستند. در مهندسی پیشرو، می توان بخشی از کد سیستم را مستقیماً از روی مدل ها تولید کرد.
- یک کلاس دیاگرام ممکن است به چند sub-class diagram تجزیه شود.
- کمک به سایر نمودارها:
نمودارهای کلاس پایه ای برای تهیه نمودارهای مؤلفه (Component Diagrams) و نمودارهای استقرار (Deployment Diagrams) نیز هستند و نقش مهمی در تکمیل دید کلی از سیستم ایفا می کنند.

کاربردهای نمودار کلاس (class diagram)

1. طراحی سیستم‌های نرم‌افزاری:

- دیاگرام کلاس به توسعه‌دهندگان کمک می‌کند تا ساختار سیستم را قبل از شروع کدنویسی طراحی کنند.
- مثلاً در طراحی یک سیستم مدیریت دانشگاه، کلاس‌هایی مانند "دانشجو"، "استاد"، و "درس" با روابط مشخص ایجاد می‌شوند.

2. درک روابط بین اجزا:

- با استفاده از دیاگرام کلاس، می‌توان روابط بین اجزای سیستم را به وضوح مشاهده کرد.
- مثلاً در یک سیستم بانکی، رابطه بین کلاس‌های "حساب بانکی" و "مشتري" مشخص می‌شود.

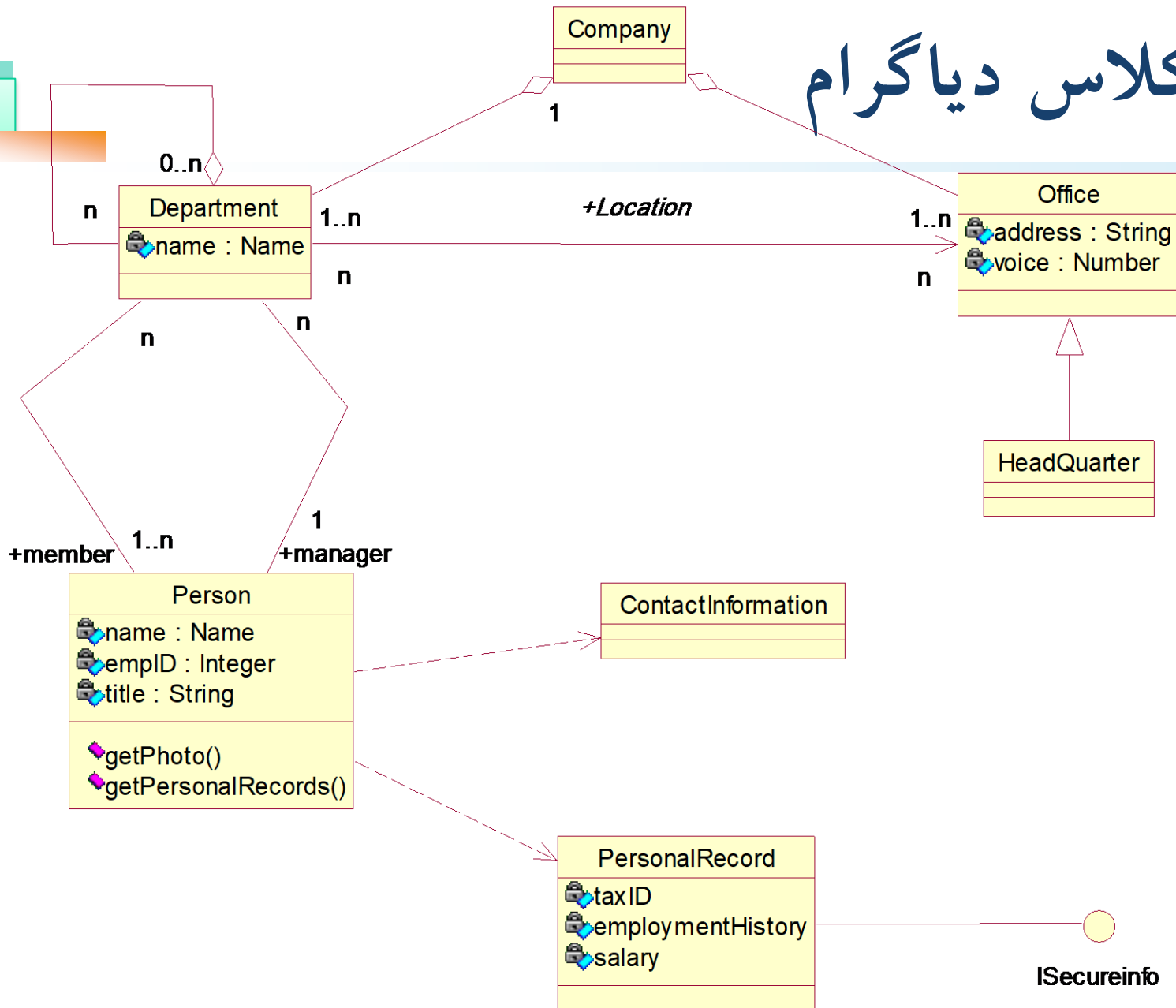
3. مدیریت پیچیدگی:

- در سیستم‌های بزرگ، دیاگرام کلاس به سازمان‌دهی کد و کاهش پیچیدگی کمک می‌کند.
- مثلاً در یک سیستم تجارت الکترونیک، کلاس‌های "محصول"، "سفارش"، و "پرداخت" به طور جداگانه طراحی می‌شوند.

4. برقراری ارتباط بین تیم‌ها:

- دیاگرام کلاس به عنوان یک زبان مشترک بین توسعه‌دهندگان، طراحان، و ذینفعان عمل می‌کند.
- مثلاً در یک پروژه تیمی، همه اعضا می‌توانند با نگاه کردن به دیاگرام کلاس، ساختار سیستم را درک کنند.

مثال کلاس دیاگرام



نمودار کلاس (ادامه...)

■ هر نمودار کلاس شامل :

Classes ■

Abstract class ■

Interfaces ■

Packages and subsystems ■

Association, dependency, generalization and
aggregation relationship ■

Notes ■

■ کلاس مجرد (abstract class) یک کلاس معمولی است
با این تفاوت که نمی توان از آن کلاس نمونه ای ایجاد کرد.

نماد کلاس

■ آیکن کلاس شامل سه بخش است:

- نام: نام باید منحصر به فرد باشد. به طور قراردادی، نام با حروف بزرگ شروع می شود و فاصله بین کلمات حذف می شود. به طور قراردادی، حرف اول صفت و نام های عملیات با حروف کوچک شروع می شوند.
- صفات : فرمت صفت :

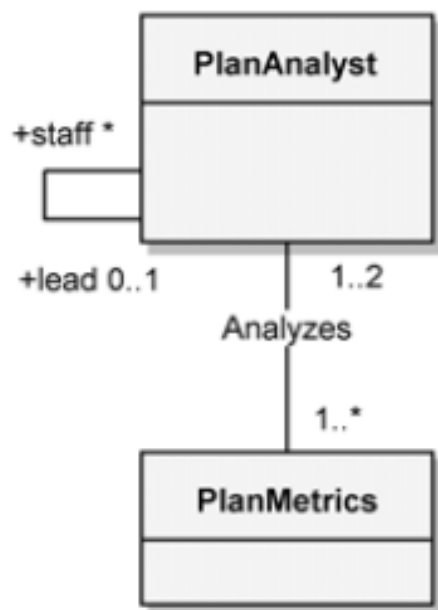
Visibility attributeName : Type [multiplicity] = ■
DefaultValue {property string}

■ عملیات: فرمت عملیات

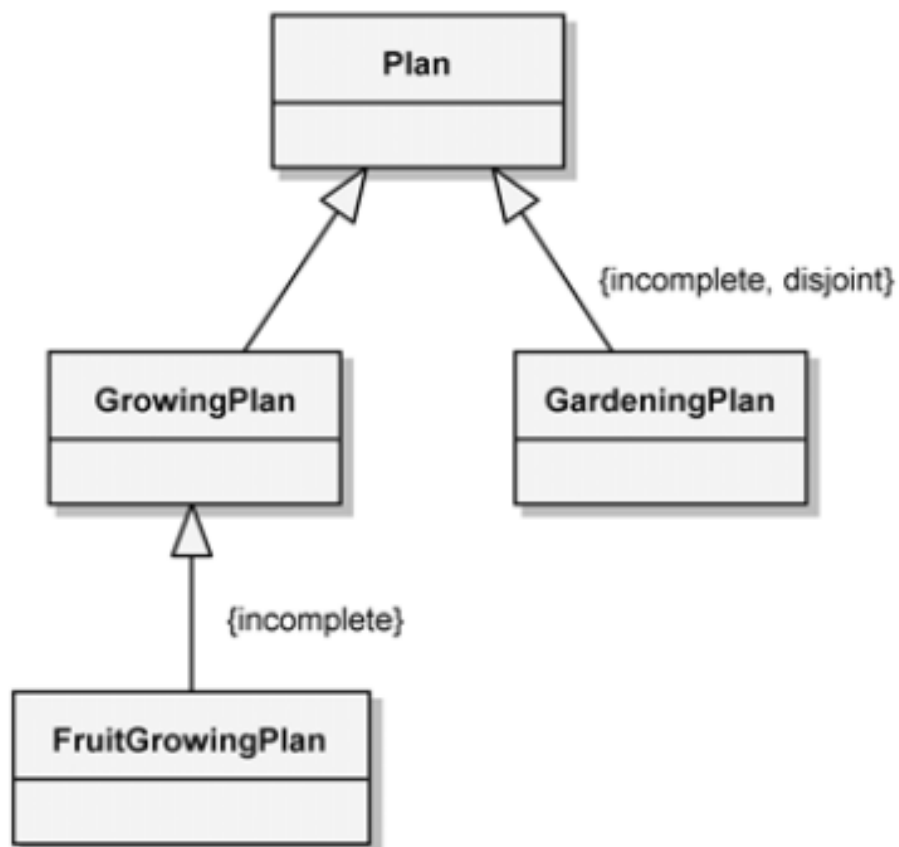
Visibility operationName (parameterName : Type) : ■
ReturnType {property string}

روابط کلاس ها

ASSOCIATION

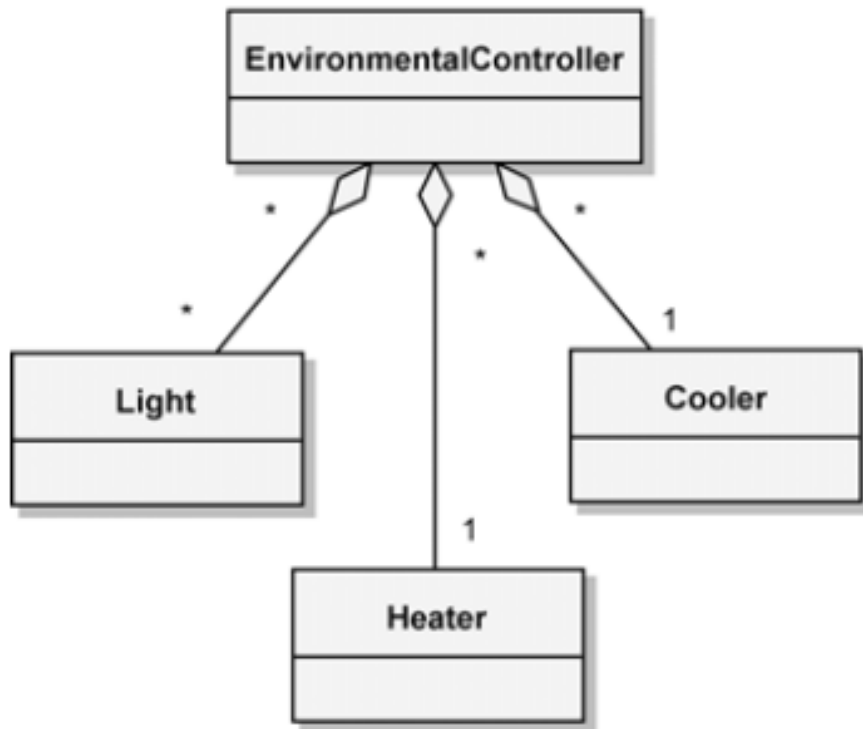


GENERALIZATION

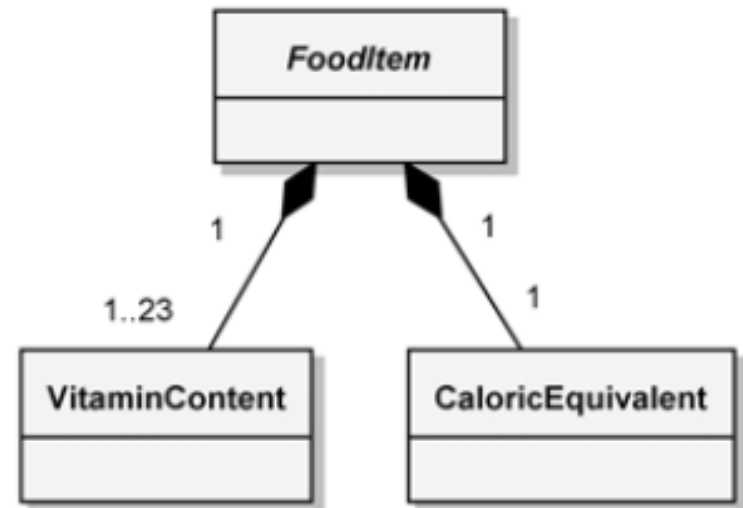


روابط کلاس ها (ادامه...)

AGGREGATION



COMPOSITION



تفاوت Composition و Aggregation

- **Aggregation** (خط تو خالی با الماس): رابطه "جزء-کل" ضعیف. جزء میتواند مستقل از کل وجود داشته باشد رابطه **EnvironmentalController** با **Cooler** یا **Heater**
- **Composition** (خط توپر با الماس پر): رابطه "جزء-کل" قوی. جزء بدون کل نمیتواند وجود داشته باشد مثال: **FoodItem** و **CaloricEquivalent**.

قابل رویت بودن کلاس ها

■ در C++ اعضا می توانند:

■ **Public** : توسط همه قابل دسترس باشند.

■ **Protected** : فقط توسط زیر کلاس ها، friend ها یا خود کلاس قابل دسترس هستند.

■ **Private**: فقط توسط خود کلاس یا friend هایش قابل دسترس باشند.

■ قابل دسترس بودن :

■ **Public (+)** : قابل مشاهده توسط هر جزئی که کلاس را می بیند.

■ **Protected (#)** : قابل مشاهده توسط اجزای دیگر داخل کلاس و زیر کلاسها

■ **Private (-)** : قابل مشاهده توسط اجزای دیگر در داخل کلاس

■ **Package (~)** : قابل مشاهده توسط اجزای داخل همان package

دیاگرام شی (object diagram)

- نمودار شی یک گراف است - مجموعه ای از گره ها (اشیاء) و یال ها (لینک ها)
- نمودار شی مجموعه ای از نمونه های کلاس ها و روابط آنها را در یک واحد زمانی مدل می کند - یک نمونه از کلاس دیاگرام است
- چند نمودار شی می تواند برای یک کلاس دیاگرام وجود داشته باشد.
- نمودار شی دید استاتیک یک سیستم را نشان می دهد - این نمودار شامل مجموعه ای از اشیا ، حالت های آنها و روابطشان است.

دیاگرام شی (object diagram)

- مشابه دیاگرام کلاس، یک خط افقی نام شی را از صفت ها و مقادیر صفت های شی مجزا می کند.
- اشیاء از طریق **link** ها با یکدیگر در ارتباطند.
- یک **link**، نمونه ای از یک **association** است.

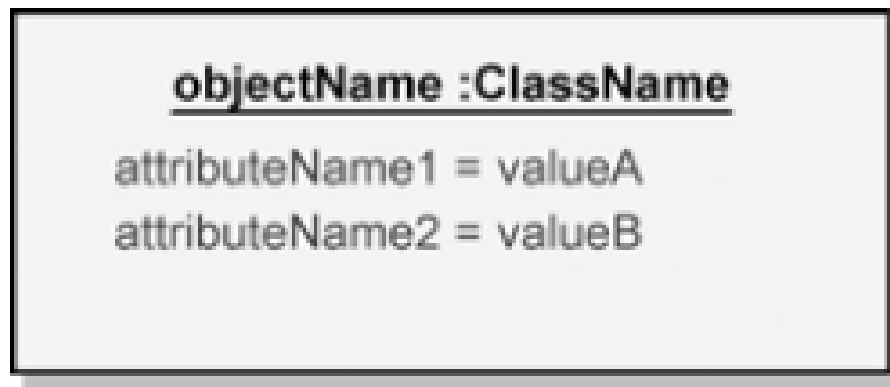
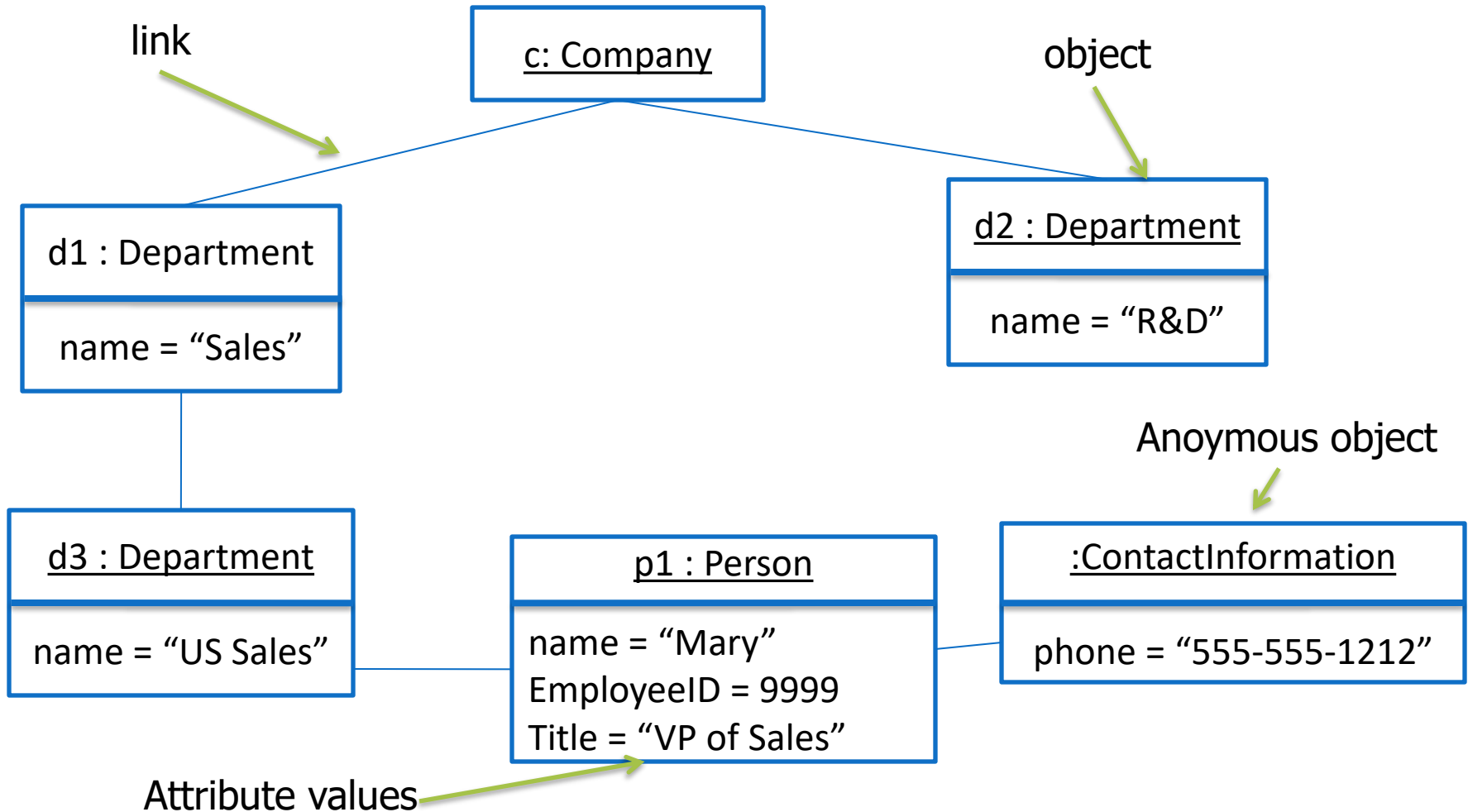


Figure 5–75 A Generic Object Icon

مثال نمودار شی



Package diagram

■ دیاگرام بسته (Package Diagram) چیست؟

■ دیاگرام بسته یک ابزار UML برای گروه‌بندی و سازماندهی عناصر سیستم (مانند کلاس‌ها، رابط‌ها، یا حتی دیاگرام‌های دیگر) در قالب بسته‌ها (Packages) است. هر بسته مانند یک "پوشه" عمل می‌کند و وابستگی‌های بین بسته‌ها را نشان می‌دهد.

■ مثال:

در یک سیستم دانشگاهی، بسته‌هایی مانند آموزش، مالی، و منابع انسانی می‌توانند وجود داشته باشند که هر کدام شامل کلاس‌ها و ماژول‌های مرتبط هستند.

- UserManagement: User, Profile, LoginService
- ProductCatalog: Product, Category, SearchService
- OrderProcessing: Order, Payment, Invoice

Package diagram

ملاک	دیاگرام بسته	دیاگرام کلاس
هدف اصلی	سازماندهی عناصر سیستم در سطح ماژولار	نمایش ساختار داخلی کلاس‌ها و روابط بین آنها
سطح جزئیات	سطح بالا (ماکرو)	سطح پایین (میکرو)
عناصر اصلی	بسته‌ها، وابستگی‌ها، رابط‌ها	کلاس‌ها، ویژگی‌ها، متدها، روابط (ارث‌بری، تجمیع، ...)
نمادها	 (مستطیل با زبانه) + فلش‌های وابستگی	 (مستطیل برای کلاس) + خطوط و فلش‌های رابطه
وابستگی‌ها	نشان می‌دهد کدام بسته به کدام بسته وابسته است مثلاً آموزش به مالی	نشان می‌دهد کدام کلاس به کلاس دیگر وابسته است مثلاً دانشجو به درس
کاربرد	طراحی معماری نرم‌افزار و مدیریت پیچیدگی	طراحی جزئیات داخلی سیستم

Package diagram

- **Package** مکانیزمی در **UML** برای گروه بندی است.

- اهداف **package**:

- گروه بندی معنایی اجزای به هم وابسته

- فراهم کننده فضای نام پنهان شده به طوری که نام ها باید منحصر به فرد باشند - برای دسترسی به جزئی در داخل یک فضای نام باید نام عنصر و نام فضای نام را مشخص کنید.

- مالک هر عنصر مدل یک **package** است

- عناصر **package** میدان دید دارند :

- **Public (+)** : عناصر برای پکیج های دیگر قابل رویت هستند.

- **Private (-)** : عناصر به طور کامل مخفی هستند.

Package diagram: Stereotypes

- در نمودار بسته‌ها (Package Diagram) در UML، از استریوتایپ‌ها (Stereotypes) برای مشخص کردن نوع و نقش بسته‌ها (Packages) یا روابط بین آن‌ها استفاده می‌شود. استریوتایپ‌ها نوعی برچسب معنایی هستند که با علامت `<<>>` نمایش داده می‌شوند.

◆ 1. `<<subsystem>>`

توضیح: نشان‌دهنده‌ی یک زیربخش (Subsystem) از سیستم است. یک بسته که شامل کلاس‌ها، رابط‌ها یا بسته‌های دیگر است که رفتار یا کارکردی خاص را ارائه می‌دهد.

کاربرد:

در سیستم بانکداری، ممکن است یک بسته‌ی `<<subsystem>> Authentication` داشته باشیم.

Package diagram: Stereotypes

◆ 2. <<framework>>

توضیح: نشان‌دهنده‌ی یک چارچوب نرم‌افزاری است که برای توسعه‌ی بخش‌های خاصی از سیستم استفاده می‌شود.

■ مثال کاربرد:

بسته‌ای به نام **UIFramework <<framework>>** برای کنترل ظاهر و رابط کاربری.

◆ 3. <<library>>

توضیح: بیانگر یک بسته‌ی کتابخانه‌ای است که شامل کدهای قابل استفاده‌ی مجدد می‌باشد.

■ مثال کاربرد:

MathUtils <<library>> که توابع ریاضی پایه را فراهم می‌کند.

Package diagram: Stereotypes

◆ 4. <<model>>

توضیح: این بسته شامل اجزای مدل سیستم است، مانند کلاس‌های دامنه (domain classes) یا نهادها.

مثال کاربرد:

OrderDomain <<model>> برای تعریف کلاس‌هایی مانند **Order** و **Customer**.

■ بسته‌ای که مدل‌های منطقی یا مفهومی سیستم را در خود جای می‌دهد.

◆ 5. <<utility>>

توضیح: بسته‌ای شامل توابع کمکی یا ابزارهای کاربردی است که در بخش‌های مختلف سیستم استفاده می‌شوند.

مثال کاربرد **StringHelpers <<utility>>**:

■ ابزارهایی کمکی مانند توابع پردازش رشته، تاریخ و زمان، و ... که در کل سیستم قابل استفاده هستند

◆ 6. <<service>>

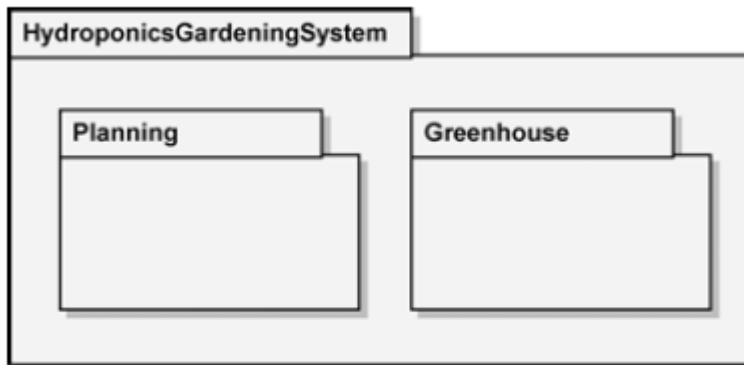
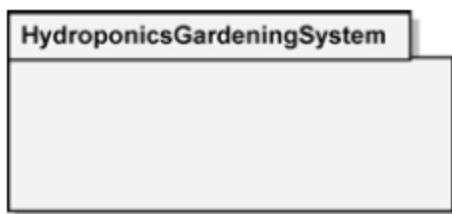
■ توضیح: بسته‌ای که سرویس‌های مشخصی را ارائه می‌دهد، مانند **API** یا سرویس‌های منطق تجاری. مثال

کاربرد **PaymentService <<service>>**:

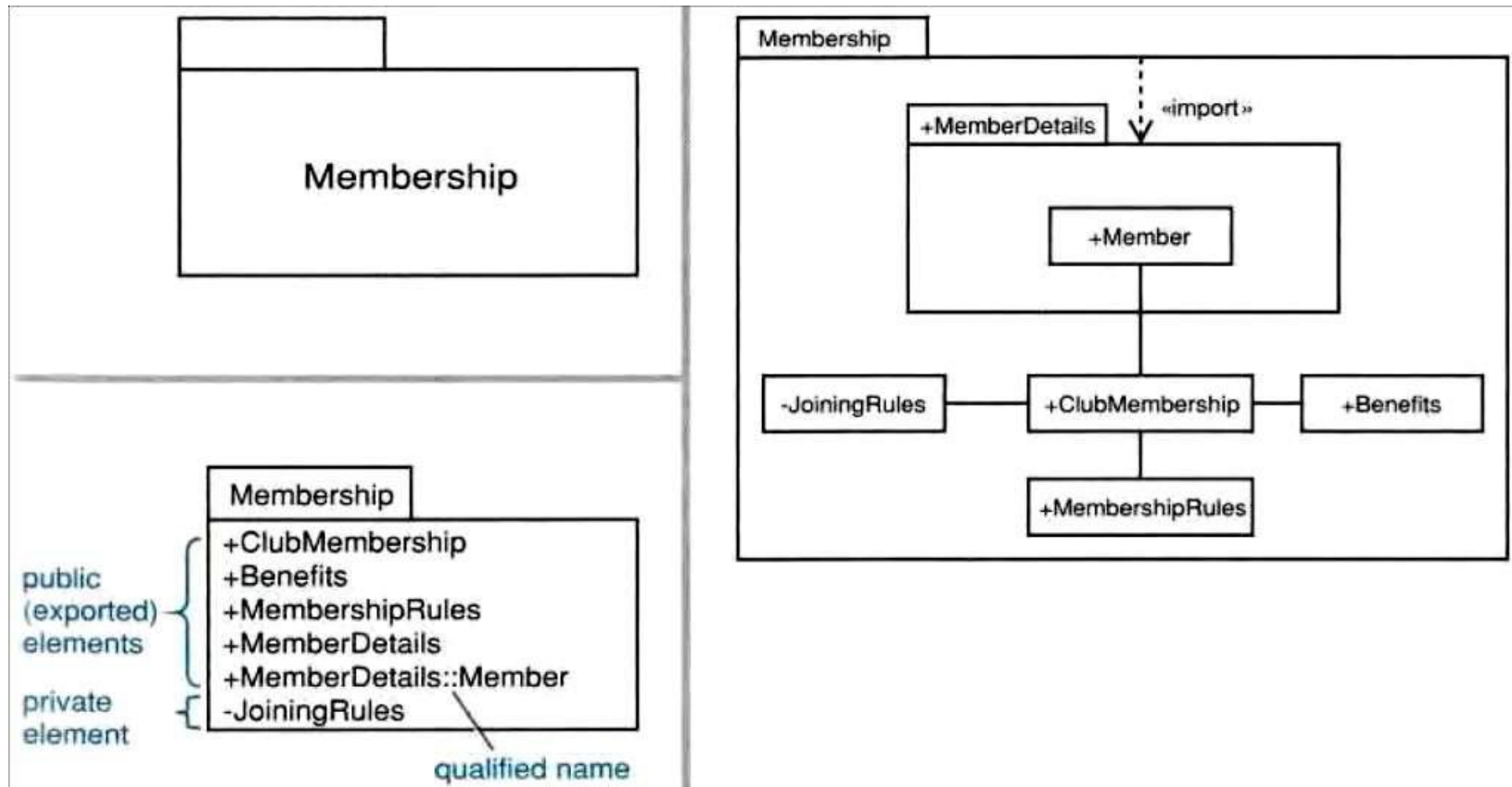
نمایانگر سرویس‌هایی است که عملیات خاصی را در سیستم انجام می‌دهند، مانند پرداخت، ورود، ارسال ایمیل و غیره.

Package diagram (cont.)

- نماد **Package** شامل یک چهارگوش به همراه نوار باریکی در بالای سمت چپ آن است.
- نماد **package** برای نمایش ساختار و محدوده اجزای مدل های مختلف مانند کلاس ها یا برای سازماندهی **use case** ها به کار میروند.
- آنها برای وضوح در یک سیستم بسیار بزرگ یا تقسیم کار استفاده میشوند.

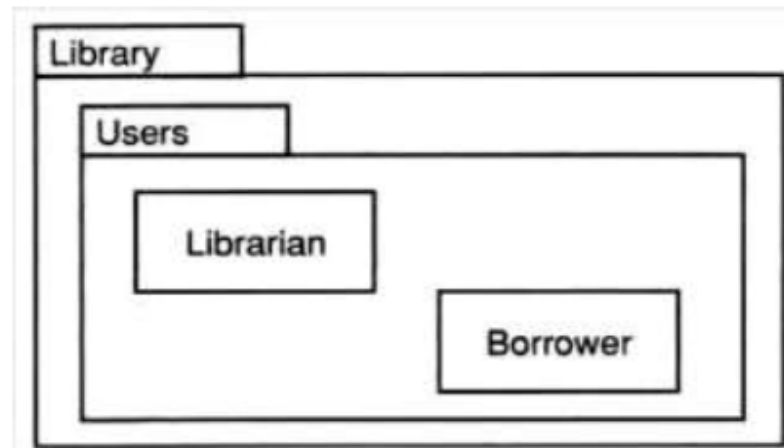
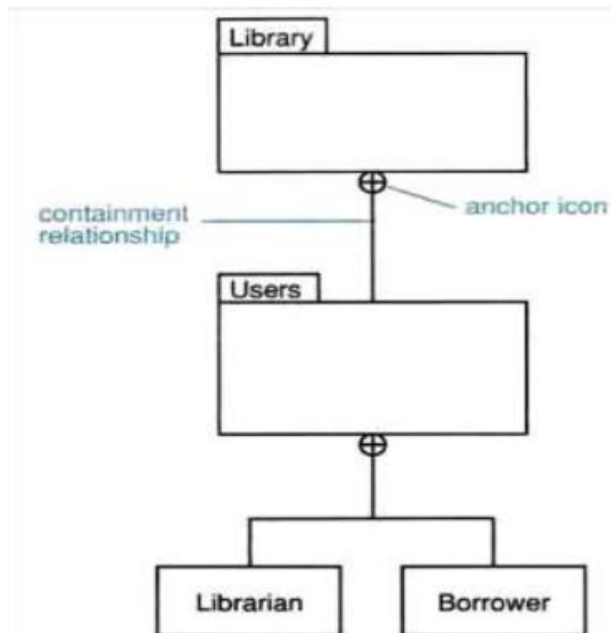


Package diagram (example)



Package های تو در تو

- پکیج داخلی می تواند تمام اعضای عمومی پکیج خارجی را ببیند.
- پکیج خارجی نمی تواند هیچکدام از اعضای پکیج داخلی را ببیند مگر اینکه یک وابستگی صریح با آنها داشته باشد (معمولاً `<<import>>` یا `<<access>>`)



Library::Users::Librarian

وابستگی package ها

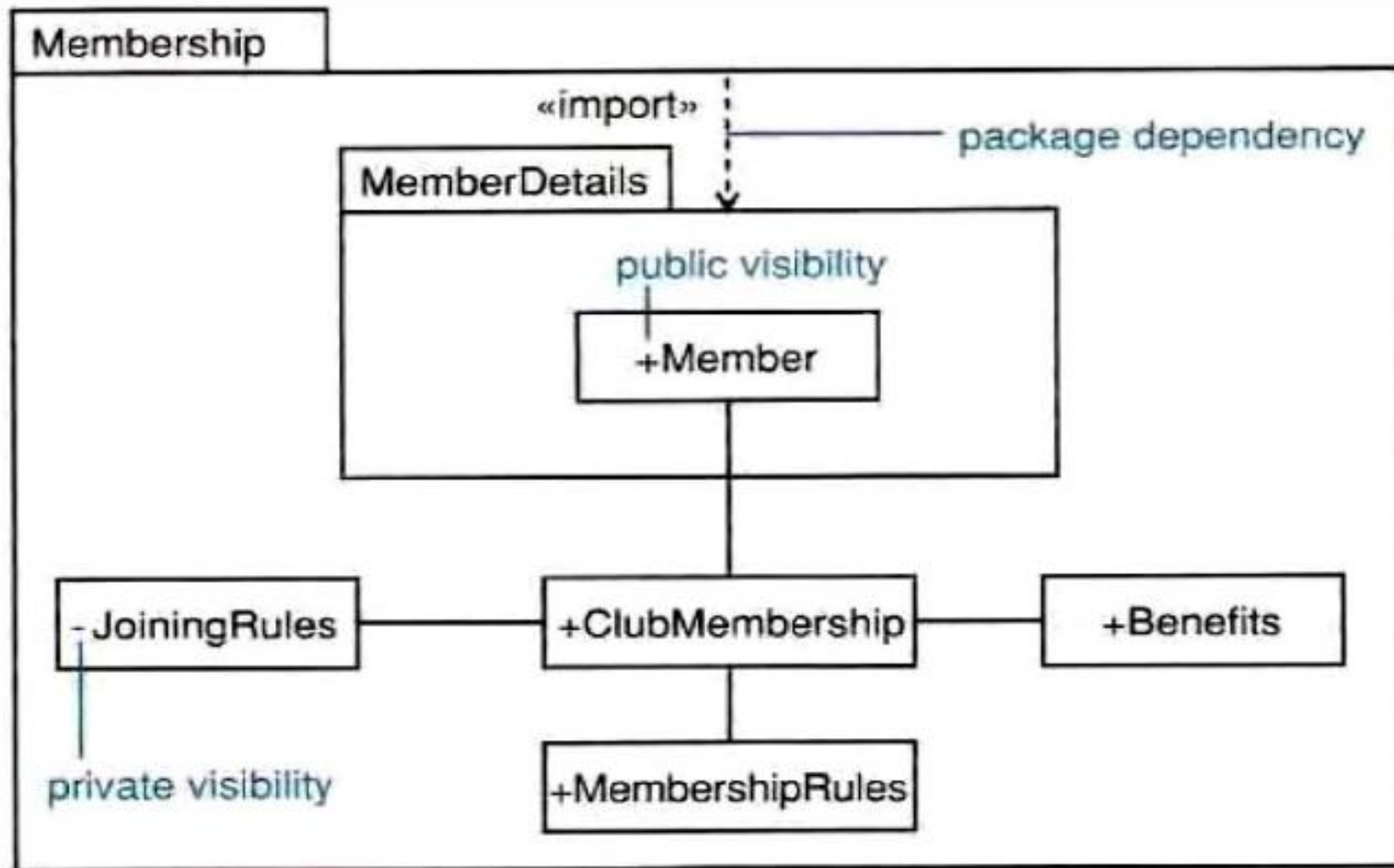
■ رابطه وابستگی بین package ها نشان می دهد که **client** package به **supplier package** وابسته است.

■ **<<use>> (default)** : یک عنصر در **client package** از یک عنصر عمومی در **supplier package** استفاده می کند.

■ **<<import>>** : عناصر عمومی **supplier namespace** به عنوان عناصر عمومی **client namespace** اضافه می شوند. عناصر در کلاینت می توانند به عناصر عمومی **supplier** دسترسی پیدا کنند.

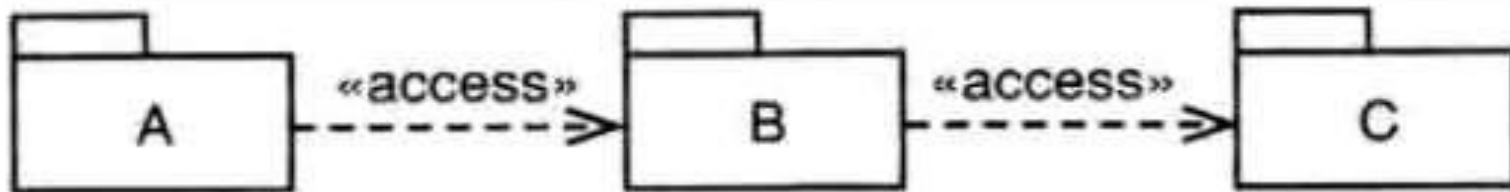
■ **<<access>>** : عناصر عمومی **supplier namespace** به عنوان عناصر خصوصی **client namespace** اضافه می شوند. عناصر در کلاینت میتوانند به همه عناصر عمومی در **supplier** دسترسی داشته باشند.

وابستگی package ها



رابطه تعدی

- تعدی : اگر A یک رابطه با B دارد و B هم با C در ارتباط است . بنابراین A با C در ارتباط است.
- `<<import>>` تعدی است اما `<<access>>` تعدی نیست.



Package Generalization

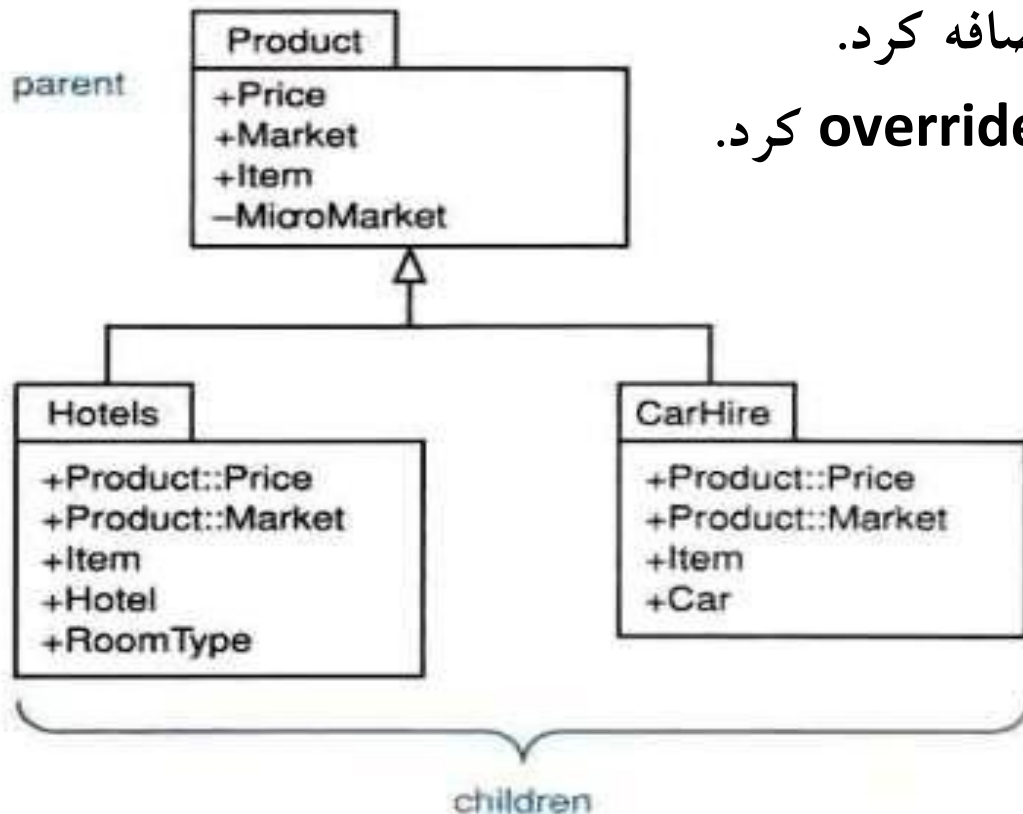
■ شبیه class generalization است.

■ Package های فرزند :

■ عناصر را از پکیج پدر به ارث می برند.

■ می توان عناصر جدید اضافه کرد.

■ می توان عناصر پدر را **override** کرد.



راه های انتخاب package ها

- راه های متفاوتی برای سازماندهی یک سیستم با package ها وجود دارد : توسط لایه های معماری، توسط زیرسیستم، توسط کاربران (برای usecase ها) و ...
- Package های خوب highly و loosely coupled و cohesive هستند.
- به این معنی که : باید تعاملات بیشتری بین اجزای یک package وجود داشته باشد و تعاملات بین package ها کمتر باشند.

راه‌های متفاوت سازماندهی سیستم با پکیج‌ها

■ سیستم‌های نرم‌افزاری را می‌توان با استفاده از پکیج‌ها به روش‌های مختلفی سازماندهی کرد. انتخاب روش مناسب به نیازهای سیستم و معماری آن بستگی دارد. برخی از رایج‌ترین روش‌ها عبارتند از:

■ سازماندهی بر اساس لایه‌های معماری (Layered Architecture):

○ سیستم به لایه‌های مجزا تقسیم می‌شود که هر لایه مسئولیت خاصی دارد.

○ مزیت: جداسازی وظایف و کاهش وابستگی بین لایه‌ها.

■ سازماندهی بر اساس زیرسیستم‌ها: (Subsystems)

○ سیستم به ماژول‌های مستقل تقسیم می‌شود که هر کدام یک قابلیت خاص را ارائه می‌دهند.

○ مزیت: توسعه و تست آسان هر زیرسیستم به صورت جداگانه.

راه‌های متفاوت سازماندهی سیستم با پکیج‌ها

■ سازماندهی بر اساس موارد استفاده: (Use Cases)

- پکیج‌ها بر اساس نیازهای کاربران یا سناریوهای استفاده (Use Cases) گروه‌بندی می‌شوند.
- مزیت: تمرکز بر نیازهای کاربر نهایی.

■ سازماندهی بر اساس فناوری: (Technology)

- پکیج‌ها بر اساس تکنولوژی مورد استفاده مانند API ها، دیتابیس، یا کتابخانه‌ها گروه‌بندی می‌شوند.

راه‌های متفاوت سازماندهی سیستم با پکیج‌ها

برای طراحی پکیج‌های کارآمد، دو اصل اساسی باید رعایت شود:
۱. **Loose Coupling**. وابستگی کم:

- معنی: پکیج‌ها تا حد امکان کمتر به یکدیگر وابسته باشند.
- مزایا:

- تغییرات در یک پکیج، تأثیر کمی بر پکیج‌های دیگر دارد.
- قابلیت نگهداری و توسعه سیستم افزایش می‌یابد.

• مثال:

- اگر پکیج Payment مستقل از پکیج Inventory باشد، تغییر در موجودی کالا، پرداخت‌ها را مختل نمی‌کند.

۲. **High Cohesion**. انسجام بالا

راه‌های متفاوت سازماندهی سیستم با پکیج‌ها

برای طراحی پکیج‌های کارآمد، دو اصل اساسی باید رعایت شود:
۲. **High Cohesion**. انسجام بالا

• معنی: عناصر داخل یک پکیج باید به شدت مرتبط و حول یک هدف مشترک متمرکز باشند.

• مزایا:

• خوانایی و درک کد بهبود می‌یابد.

• احتمال خطا کاهش می‌یابد.

• مثال:

• تمام کلاس‌های مرتبط با مدیریت کاربران مانند User، Role، Permission

در یک پکیج UserManager قرار می‌گیرند.

پیدا کردن package ها

زیرسیستم ها را بررسی کنید.

بررسی کلاس ها برای :

■ کلاسترهای منسجمی از کلاسهای به هم وابسته

■ **کلاستر (Cluster)** به مجموعه‌ای از کلاس‌هایی گفته می‌شود که تعامل زیادی با هم دارند. این کلاس‌ها باید در یک package قرار گیرند.

■ در بخش سفارش: کلاس‌های Order, OrderItem, Invoice, ShippingDetail

چون به هم مرتبط هستند، باید در یک package با عنوان OrderModule قرار گیرند.

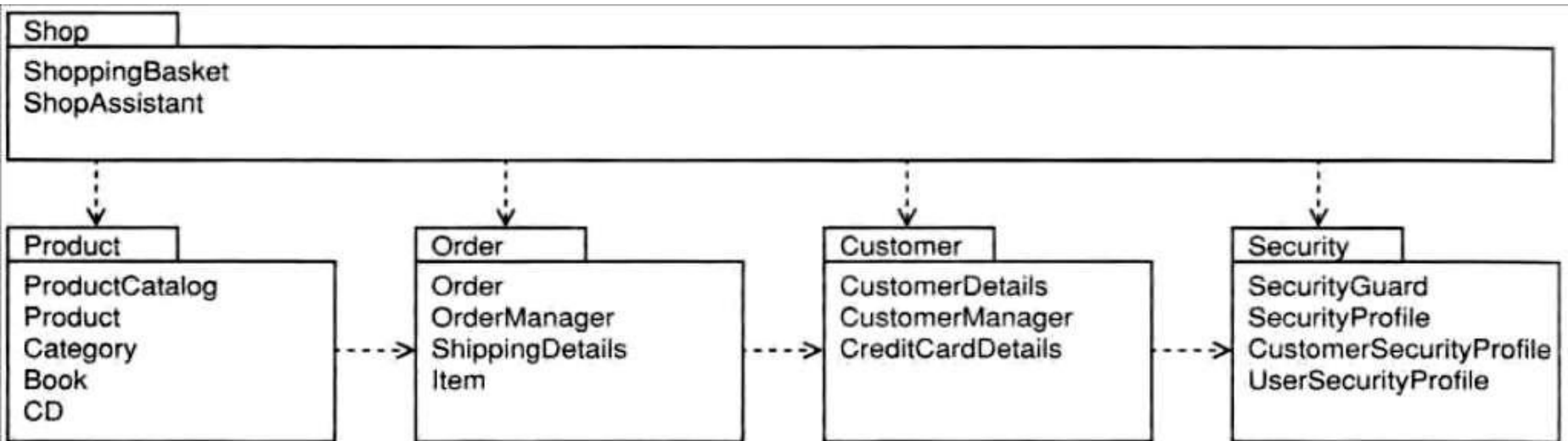
■ **ارث‌بری**: کلاس‌هایی که در یک ساختار ارث‌بری (سلسله‌مراتبی) قرار دارند معمولاً با هم وابستگی مفهومی دارند و می‌توانند در یک package قرار گیرند.

■ **کلاس‌های به هم وابسته** به ترتیب از طریق وراثت (inheritance) ، ترکیب (composition)، **تجمع (Aggregation)** و وابستگی (dependency)

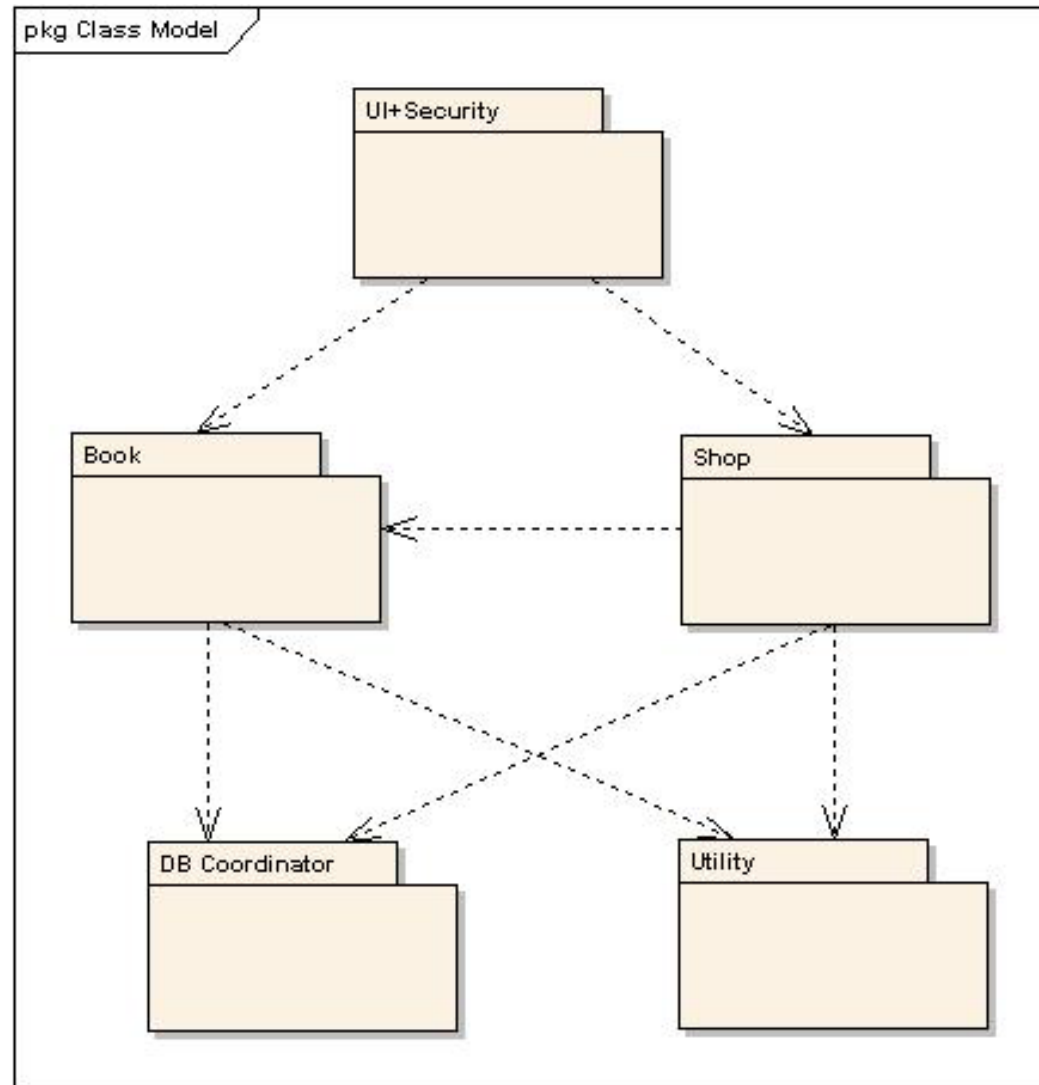
بررسی use case ها :

■ کلاسترهایی از use case ها که یک Actor یا پروسه را ساپورت می‌کنند باید در یک package قرار گیرند.

Package diagram (cont.)



Package diagram (cont.)



Activity diagrams

■ نمودار فعالیت یک نوع نمودار UML زبان مدل سازی یکپارچه است که جریان فعالیتها یا فرآیندهای یک سیستم را نشان می دهد. این نمودار شبیه به فلوچارت است، اما بیشتر بر گردش کار، فرآیندهای کسب و کار و رفتار سیستم تمرکز دارد تا صرفاً کنترل جریان.

■ اجزای اصلی نمودار فعالیت

1. **گره شروع** (حالت اولیه) - آغاز فرآیند را مشخص می کند.
2. **اقدامات/فعالیتها** - نشان دهنده مراحل یا وظایف در فرآیند.
3. **جریان کنترل (پیکانها)** - ترتیب فعالیتها را نمایش می دهد.
4. **گره تصمیم (لوزی)** - یک شرط انشعاب را نشان می دهد (مثلاً بله/خیر).
5. **گره ادغام** - چندین جریان را به یک مسیر برمی گرداند.
6. **گره فورک و جوین** - فعالیت های موازی را مدل می کند.
7. **گره پایان** (حالت نهایی) - پایان فرآیند را مشخص می کند.

نمودار فعالیت (Activity Diagram)

چه چیزی را نشان می دهد:

- شروع تا پایان فرآیند خرید در فروشگاه اینترنتی
- فعالیت هایی مانند: مشاهده محصولات، افزودن به سبد، پرداخت و دریافت تأییدیه
- نقاط تصمیم گیری:
 - آیا محصولی انتخاب شده؟
 - آیا سبد خرید خالی است؟
 - آیا پرداخت موفقیت آمیز بوده؟
- مسیرهای خطا: مثل پرداخت ناموفق یا سبد خرید خالی
- مسیر موفق: شامل ثبت سفارش و ارسال ایمیل تأییدیه

نمودار فعالیت (Activity Diagram)

کاربردها:  

- مستندسازی روندهای تجاری و کاربری
- طراحی یا تحلیل منطق کنترلی سیستم
- شناسایی شرایط تصمیم‌گیری، تکرارها و خطاها در یک فرآیند

UML Diagram Type

Structural Diagrams

Composite Structure Diagram

Deployment Diagram

Package Diagram

Profile Diagram

Class Diagram

Object Diagram

Component Diagram

Behavioral Diagrams

Activity Diagram

Use Case Diagram

State Machine Diagram

Interaction Diagram

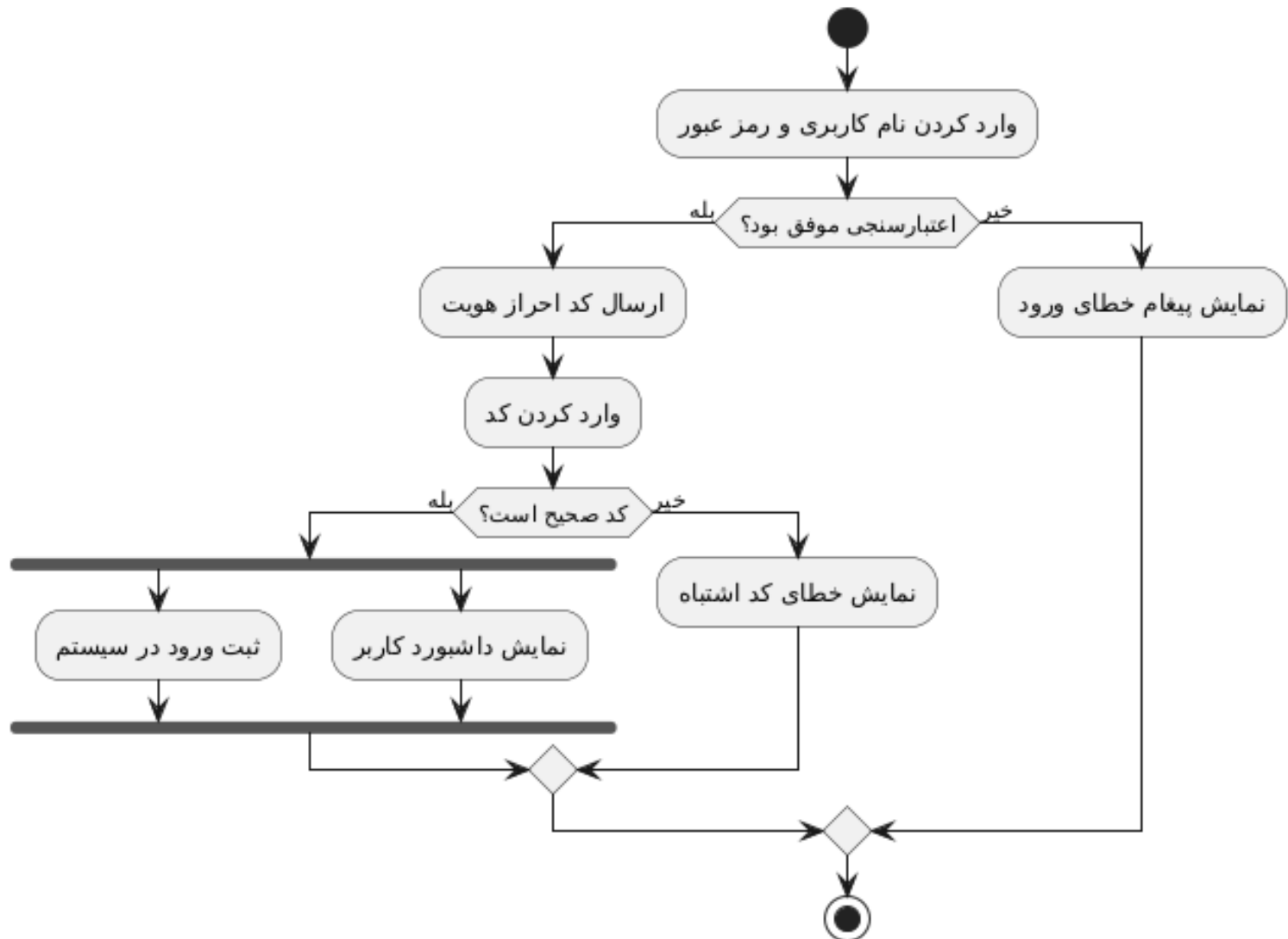
Sequence Diagram

Communication Diagram

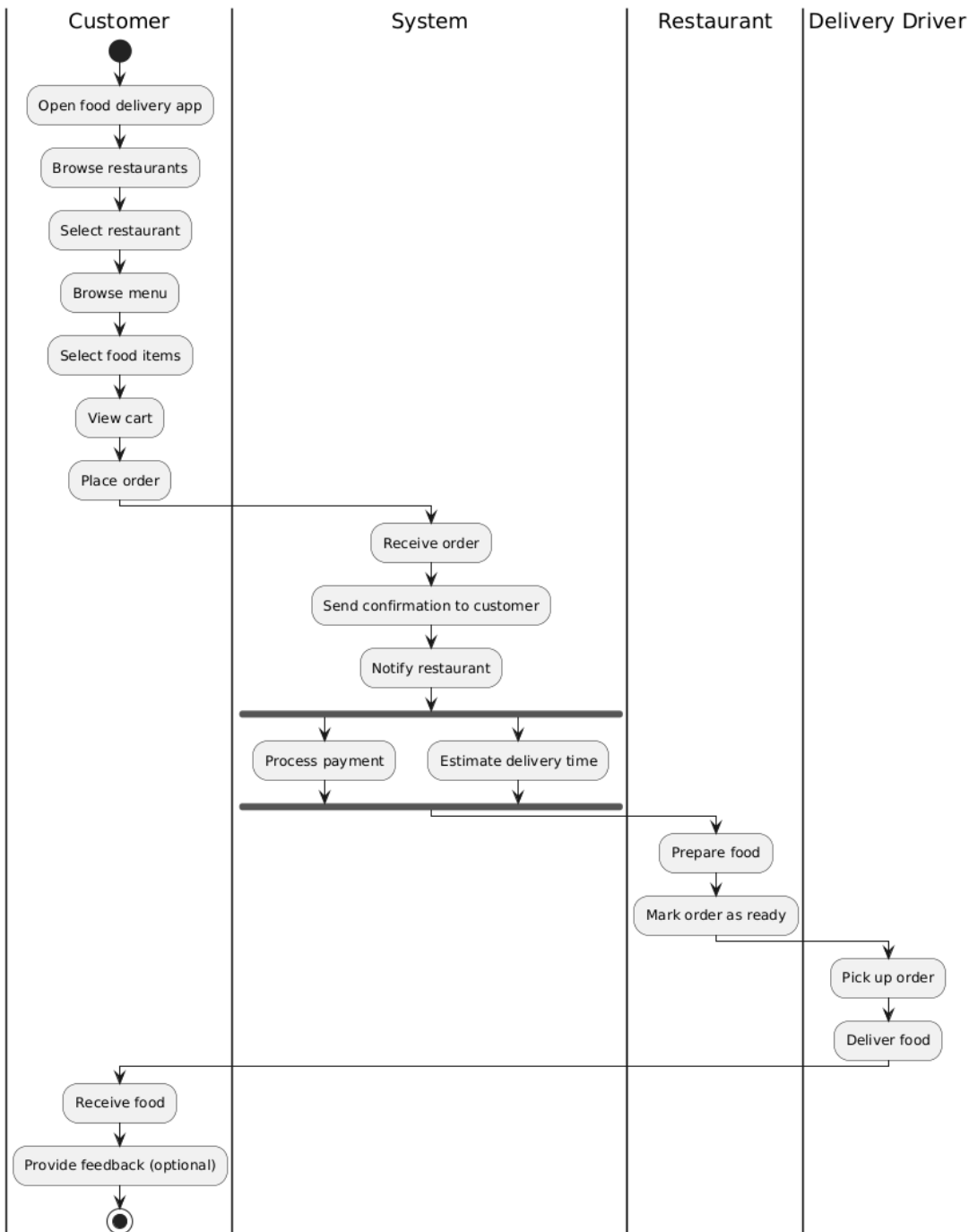
Interaction Overview Diagram

Timing Diagram

Activity diagrams

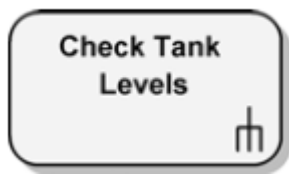


UML activity diagrams



Activity diagrams (ادامه..)

Action



- واحد اصلی رفتار در دیاگرام فعالیت هستند.
- فعالیت ها می توانند شامل چندین **action** باشند.

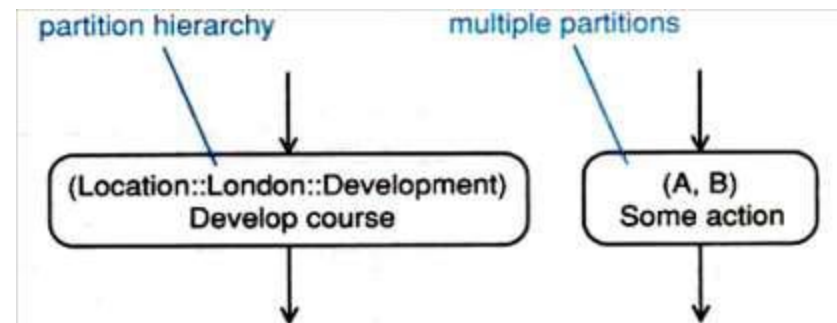
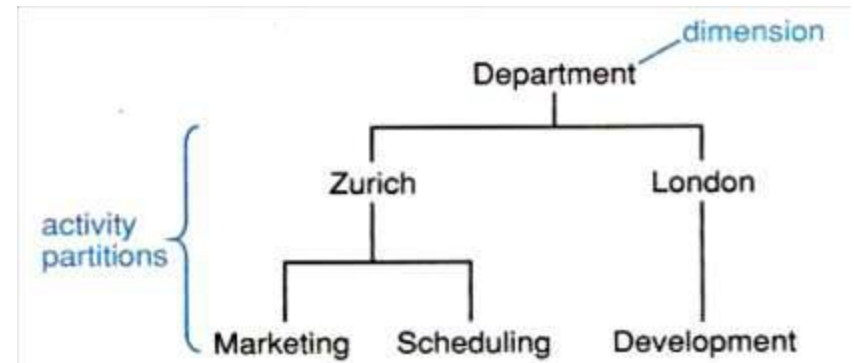
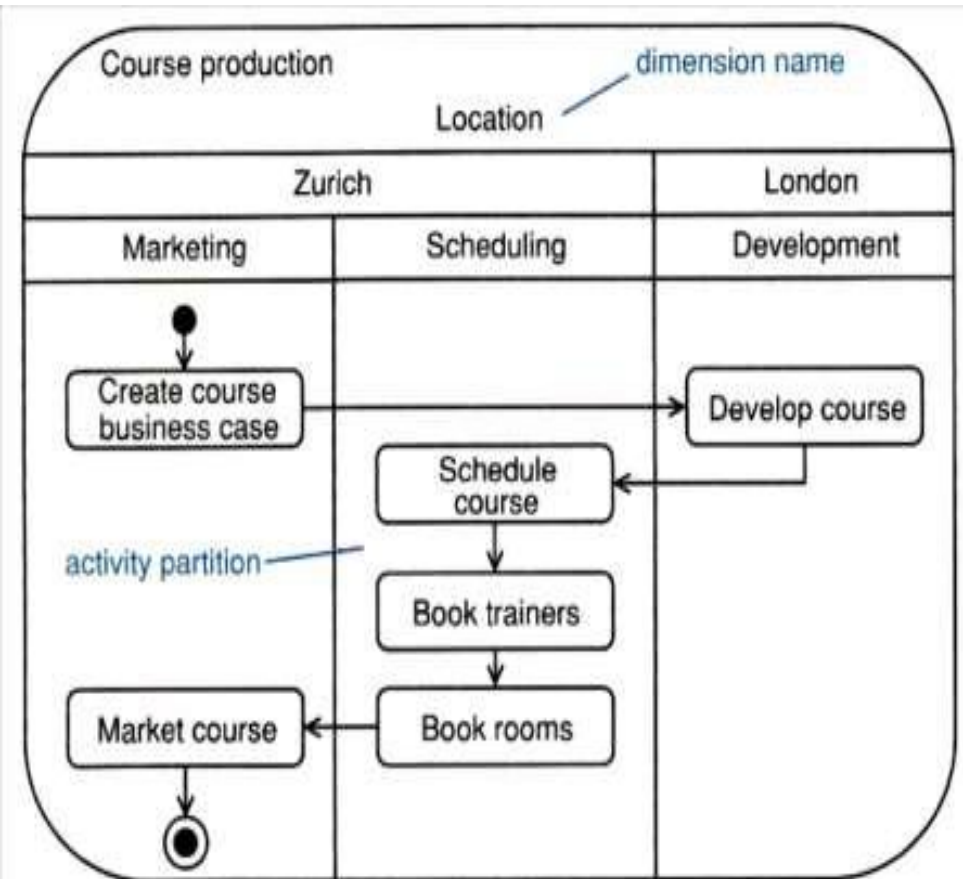
Starting and Stopping

- از آنجایی که دیاگرام فعالیت نشان دهنده جریان فرآیند است، جریان باید جایی شروع شده و جایی پایان یابد.
- نقطه شروع (**starting point- Initial point**) در جریان فعالیت با یک نقطه توپر نشان داده میشود.
- نقطه پایان (**stopping point- final point**)

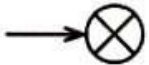
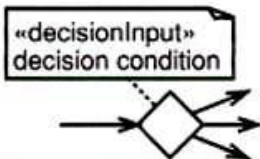
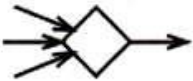
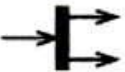
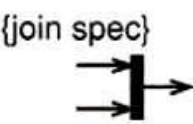


پارتیشن ها در activity diagram

- عناصر در دیاگرام فعالیت می توانند با استفاده از پارتیشن ها تقسیم بندی شوند.



گره های کنترلی

Syntax	Name	Semantics
	Initial node	Indicates where the flow starts when an activity is invoked
	Activity final node	Terminates an activity
	Flow final node	Terminates a specific flow within an activity – the other flows are unaffected
	Decision node	The output edge whose guard condition is true is traversed May optionally have a «decisionInput»
	Merge node	Copies input tokens to its single output edge
	Fork node	Splits the flow into multiple concurrent flows
	Join node	Synchronizes multiple concurrent flows May optionally have a join specification to modify its semantics

Final nodes

گره های کنترلی

معنی/توضیحات	ترجمه فارسی	عنوان (انگلیسی)
مشخص میکند جریان فعالیت هنگام فراخوانی از کجا شروع میشود	گره آغازین	Initial node
کل فعالیت را خاتمه میدهد	گره پایانی فعالیت	Activity final node
یک جریان خاص را درون فعالیت پایان میدهد - سایر جریانها تحت تأثیر قرار نمیگیرند	گره پایانی جریان	Flow final node
یال خروجی که شرط محافظ آن درست است، طی میشود. ممکن است «ورودی تصمیم» داشته باشد	گره تصمیم	Decision node
شرط تصمیمگیری برای انتخاب مسیر	«ورودی تصمیم»	«decisionInput»
توکنهای ورودی را به یال خروجی واحدش کپی میکند	گره ادغام	Merge node
جریان را به چند جریان همزمان تقسیم میکند	گره انشعاب	Fork node
جریانهای همزمان چندگانه را همگامسازی میکند. ممکن است مشخصه اتصال داشته باشد	گره اتصال	Join node
برای تغییر معناشناسی گره اتصال استفاده میشود (اختیاری)	مشخصه اتصال	Join specification

گره های merge و decision (ادامه...)

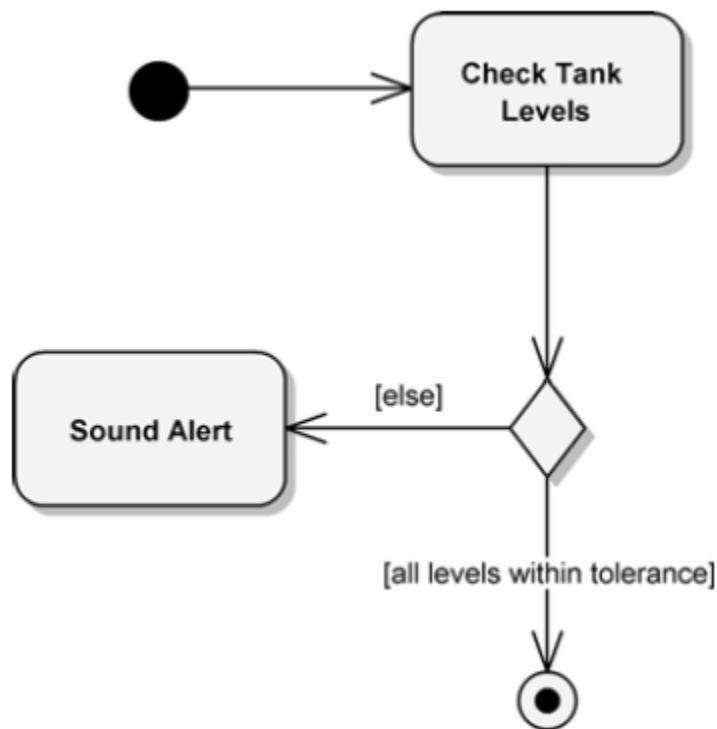


Figure 5–29 A Decision Node

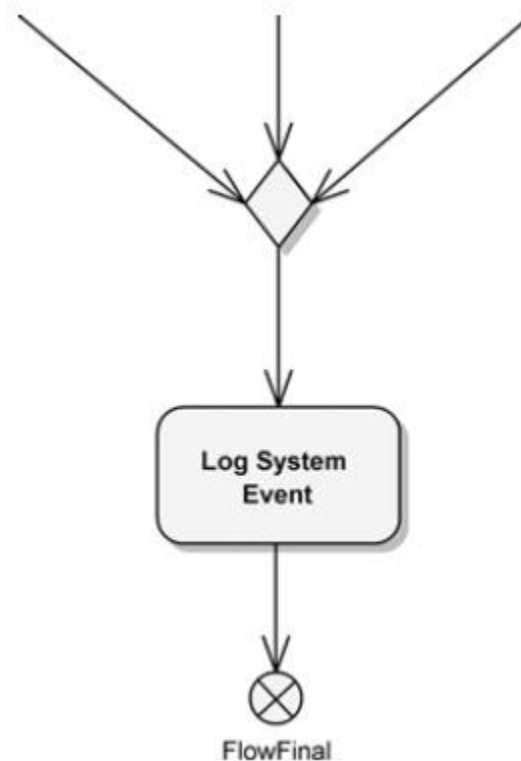
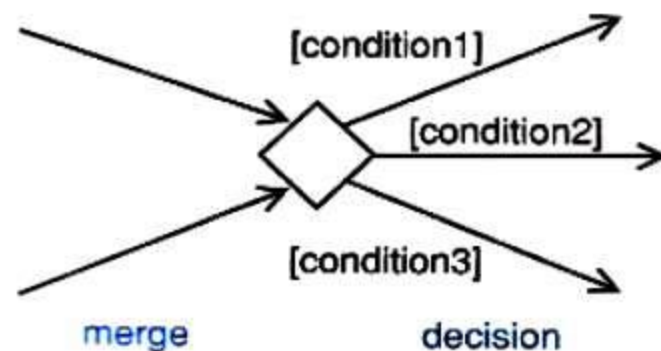
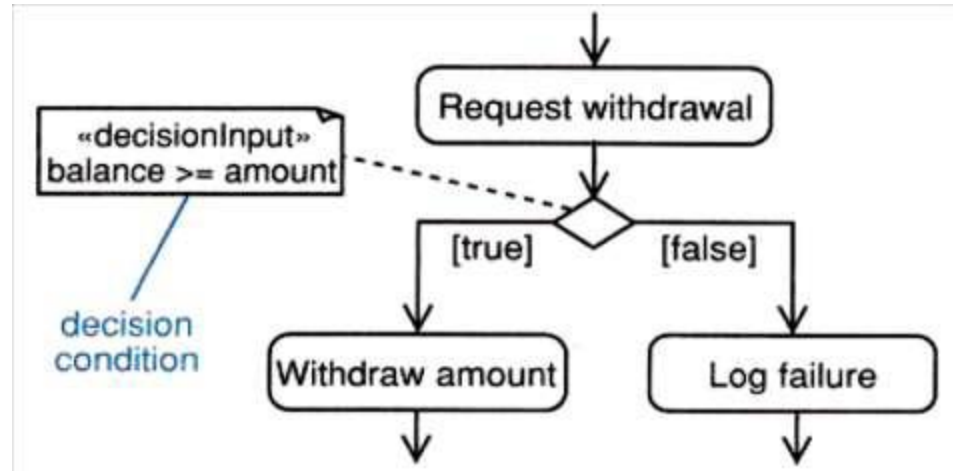
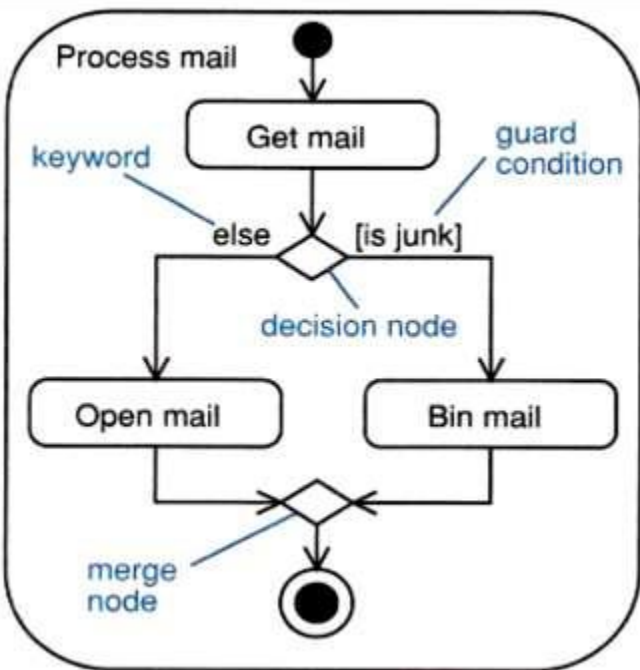
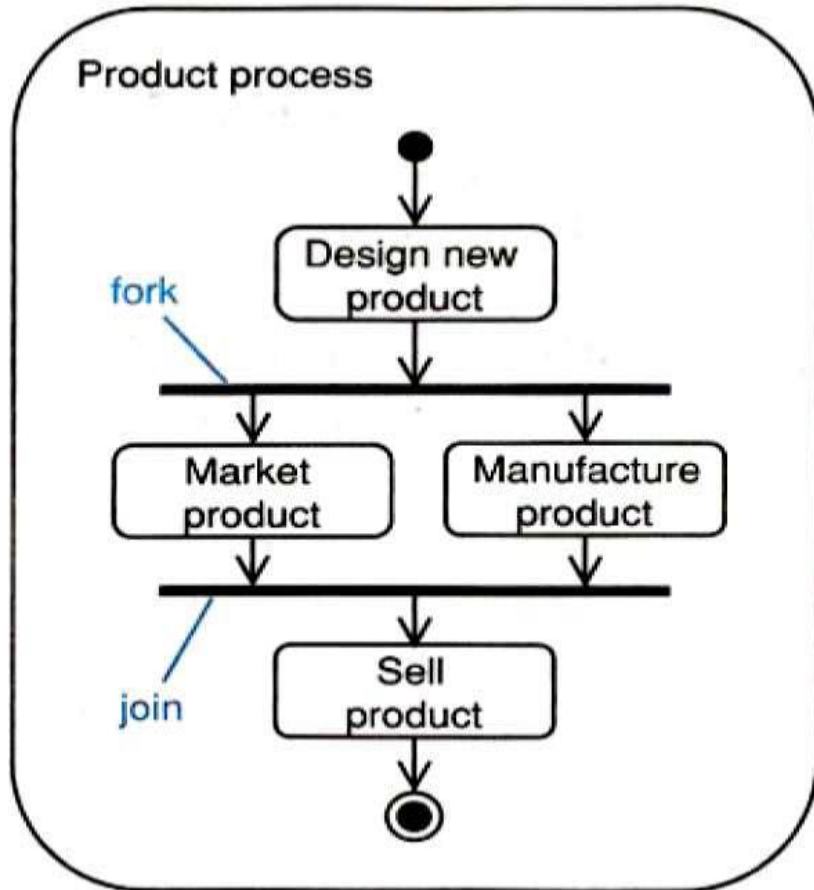


Figure 5–30 A Merge Node with a Flow Final Node

گره های decision و merge (ادامه...)

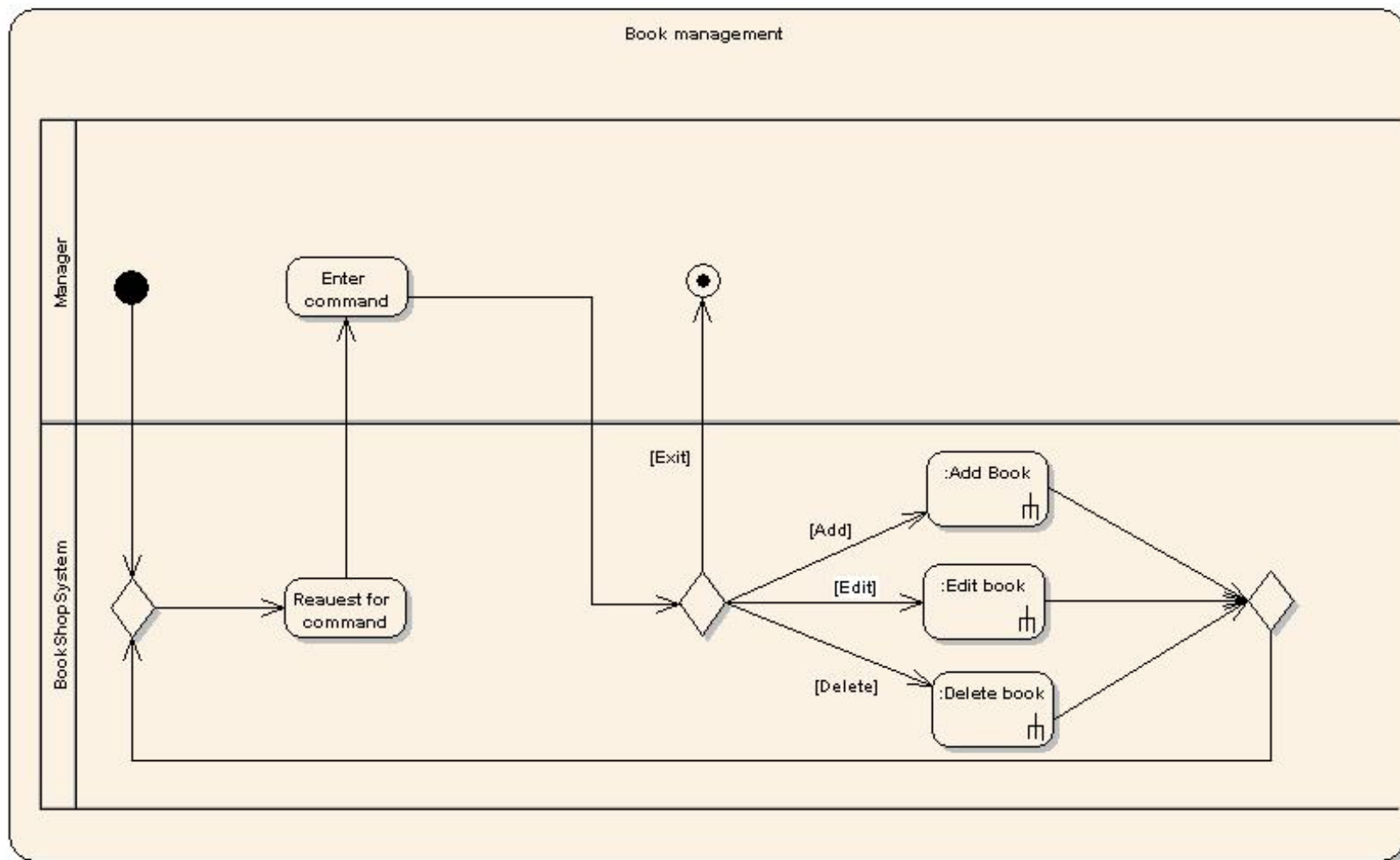


گره های کنترلی : Fork و Join



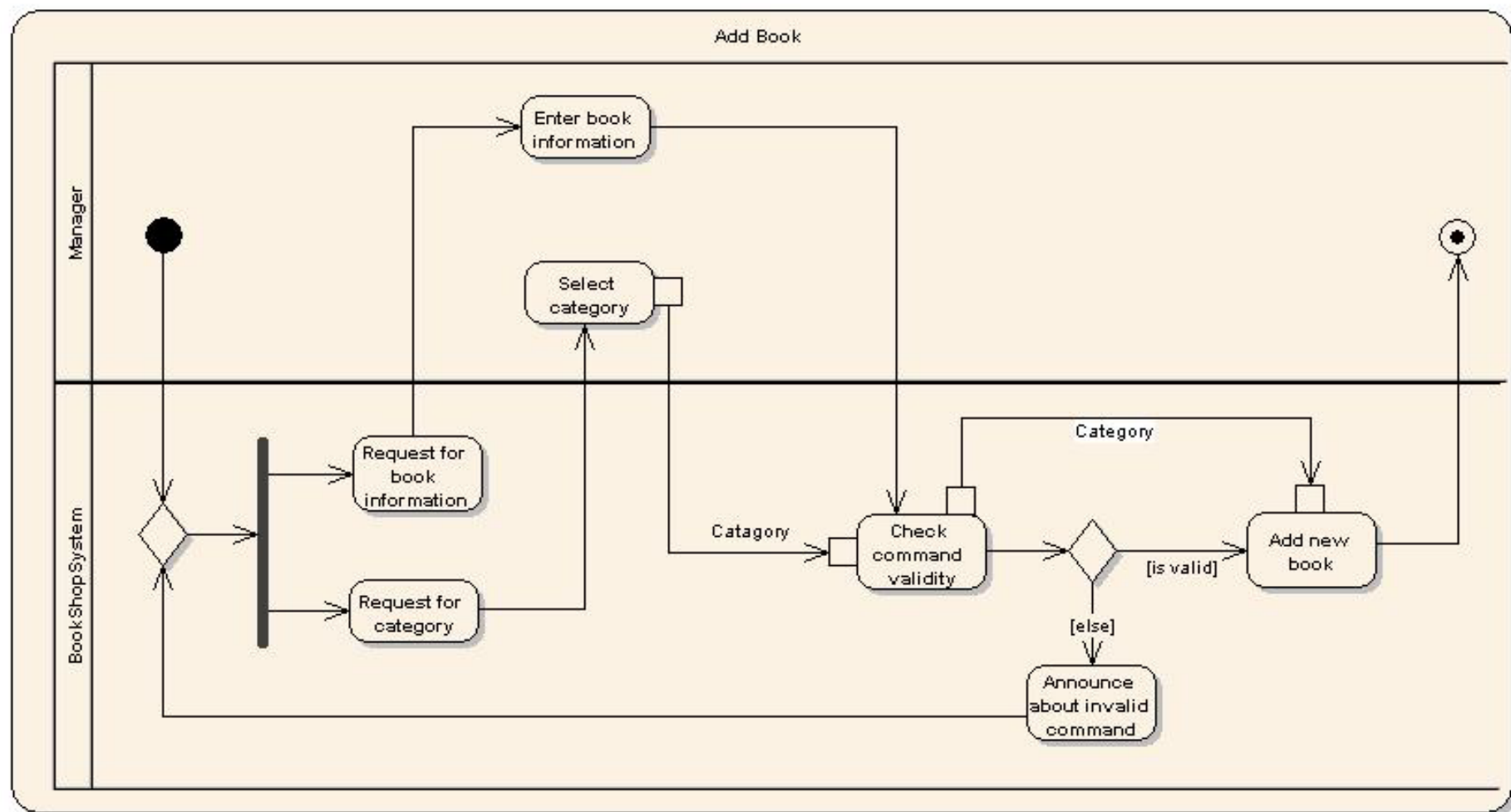
مثالی از activity diagram

act Book Management

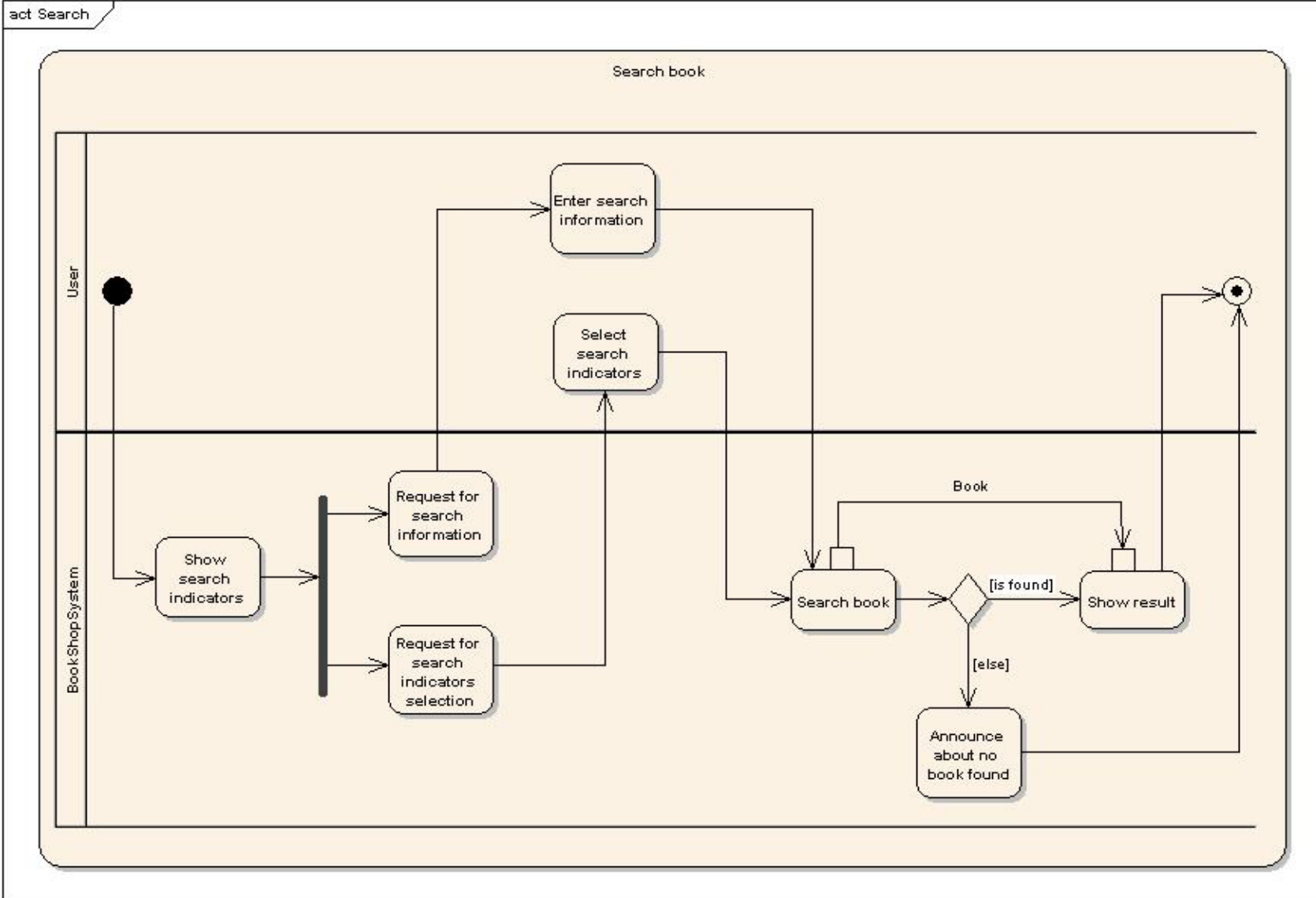


مثالی از activity diagram

act Add Book

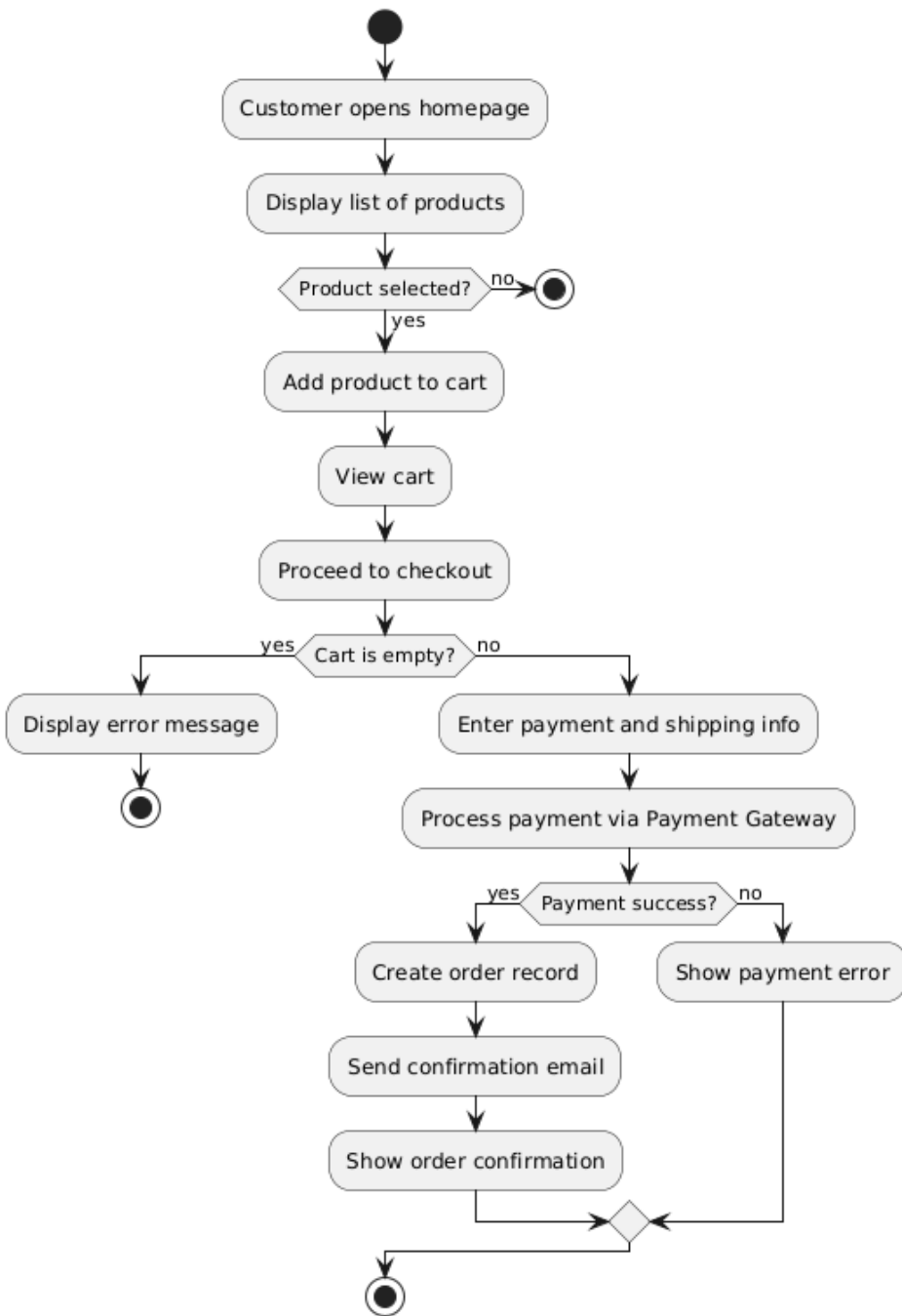


مثالی از activity diagram



Online Shop Purchase Flow - Activity Diagram

مثالی از activity diagram



UML Diagram Type

Structural Diagrams

Composite Structure Diagram

Deployment Diagram

Package Diagram

Profile Diagram

Class Diagram

Object Diagram

Component Diagram

Behavioral Diagrams

Activity Diagram

Use Case Diagram

State Machine Diagram

Interaction Diagram

Sequence Diagram

Communication Diagram

Interaction Overview Diagram

Timing Diagram

نمودار توالی (Sequence Diagram)

■ نمودار توالی برای مدل سازی تعامل بین اجزای سیستم در طول زمان استفاده می شود. تمرکز اصلی بر ترتیب زمانی پیام ها و ارتباطات بین اجزای مختلف مانند کاربر، پایگاه داده، خدمات پرداخت و غیره است.

چه چیزی را نشان می دهد: 🔍

- بازیگران (Customer, WebApp, PaymentGateway, ...)

- پیام ها و فراخوانی ها بین اجزا

- ترتیب اجرای عملیات از اولین درخواست کاربر تا دریافت تاییدیه نهایی

- نمایش روشن از چه کسی با چه کسی تعامل دارد و به چه ترتیبی

✓ کاربردها:

- طراحی و مستندسازی معماری سیستم و تعامل اجزا

- تحلیل و بررسی روند ارتباطات API و خدمات

- درک دقیق از ترتیب اجرای درخواست ها در سیستم

نمودار توالی (Sequence Diagram)

Scenario: Online Order System

■ Sequence Diagram shows:

- User → System → Payment Gateway → Database
- Exact order of messages like validate(), charge(), confirm()

■ Activity Diagram shows:

- Start → Select product → Checkout →
→ Decision: Payment successful?
 - Yes → Ship item
 - No → Retry / Cancel

نمودار توالی (Sequence Diagram)

ویژگی	نمودار فعالیت (Activity Diagram)	نمودار توالی (Sequence Diagram)
تمرکز اصلی	جریان کنترل و منطق تصمیم گیری	تعاملات بین اجزای سیستم در طول زمان
نمایانگر	فعالیت ها و مسیرهای کنترل	اشیاء، بازیگران و پیام ها
نمایش زمان	ضمنی و از طریق ترتیب گره ها	صریح و از بالا به پایین
کاربرد اصلی	مدل سازی فرآیندها و رفتارهای کاربر	مدل سازی تعاملات بین اجزای سیستم
ساختار بصری	گره ها (فعالیت) و یال ها (جریان)	خطوط عمر (lifelines) و پیام های افقی
مدیریت تصمیم گیری	به صورت if / else و شاخه های شرطی	نیازمند ساختارهای خاص برای شرط ها
نمونه کاربردی	روند خرید از دید کاربر و بررسی تصمیم ها	تعامل وب اپ با خدمات مختلف مثل پایگاه داده و پرداخت

Sequence diagrams (cont.)

- در دیاگرام های توالی، موجودیت ها (اشیا) در قسمت بالای دیاگرام به صورت افقی قرار می گیرند.
- خط چین عمودی، خط عمر (lifeline) نامیده می شود که زیر هر شی کشیده می شود. این خطوط، وجود شی را نشان می دهند.
- پیام ها (Messages) که مشخص کننده رخدادها و احضارها هستند به صورت افقی نشان داده میشوند.
 - پیام ها از فرستنده به گیرنده فرستاده می شوند.
 - اولین پیام در بالای دیاگرام و آخرین پیام در انتهای دیاگرام قرار می گیرد.

Sequence diagrams (cont.)

Use case: AddCourse

ID: 8

Brief description:
Add details of a new course to the system.

Primary actors:
Registrar

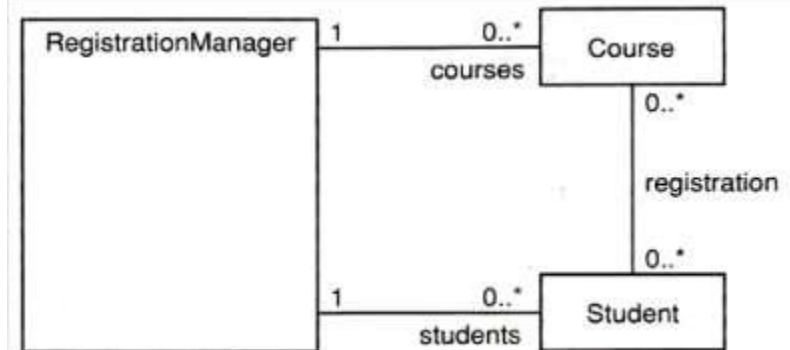
Secondary actors:
None.

Preconditions:
1. The Registrar has logged on to the system.

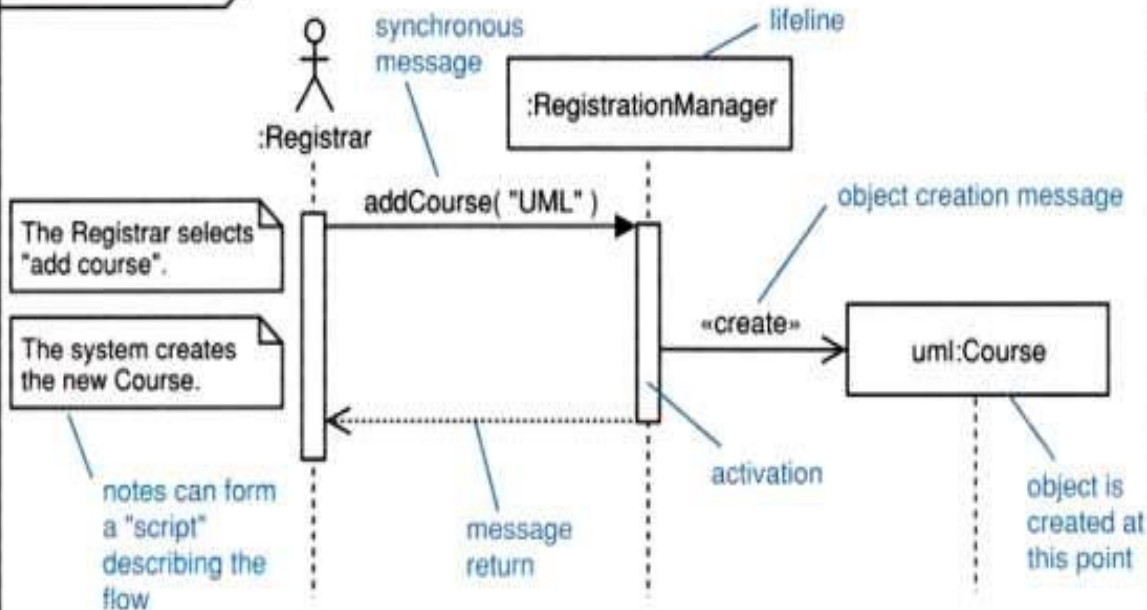
Main flow:
1. The Registrar selects "add course".
2. The Registrar enters the name of the new course.
3. The system creates the new course.

Postconditions:
1. A new course has been added to the system.

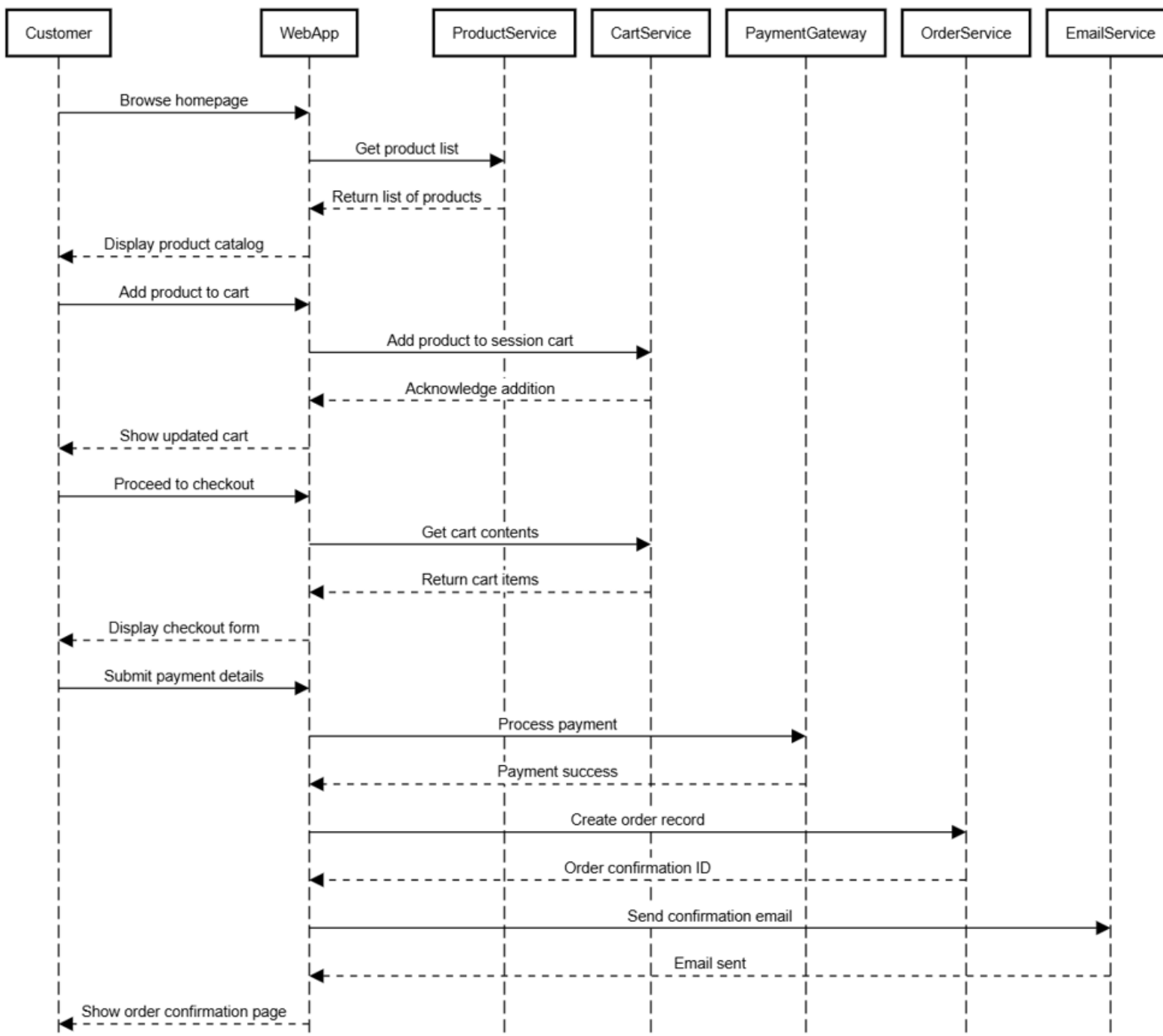
Alternative flows:
CourseAlreadyExists



sd AddCourse



Online Shop Purchase Flow



Sequence diagrams (cont.) – مثال ٢

Use case: DeleteCourse

ID: 8

Brief description:

Remove a course from the system.

Primary actors:

Registrar

Secondary actors:

None.

Preconditions:

1. The Registrar has logged on to the system.

Main flow:

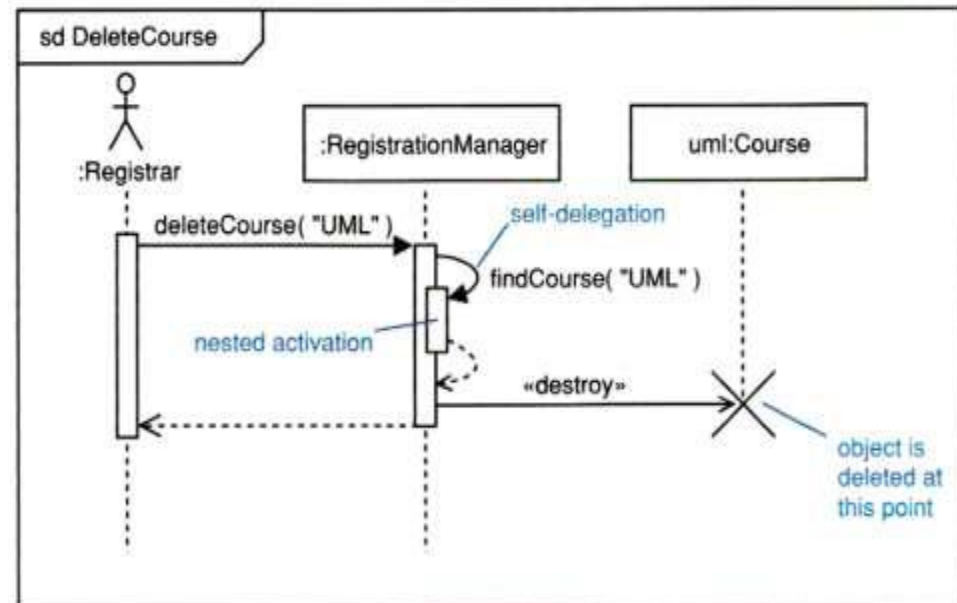
1. The Registrar selects "delete course".
2. The Registrar enters the name of the course.
3. The system deletes the course.

Postconditions:

1. A course has been removed from the system.

Alternative flows:

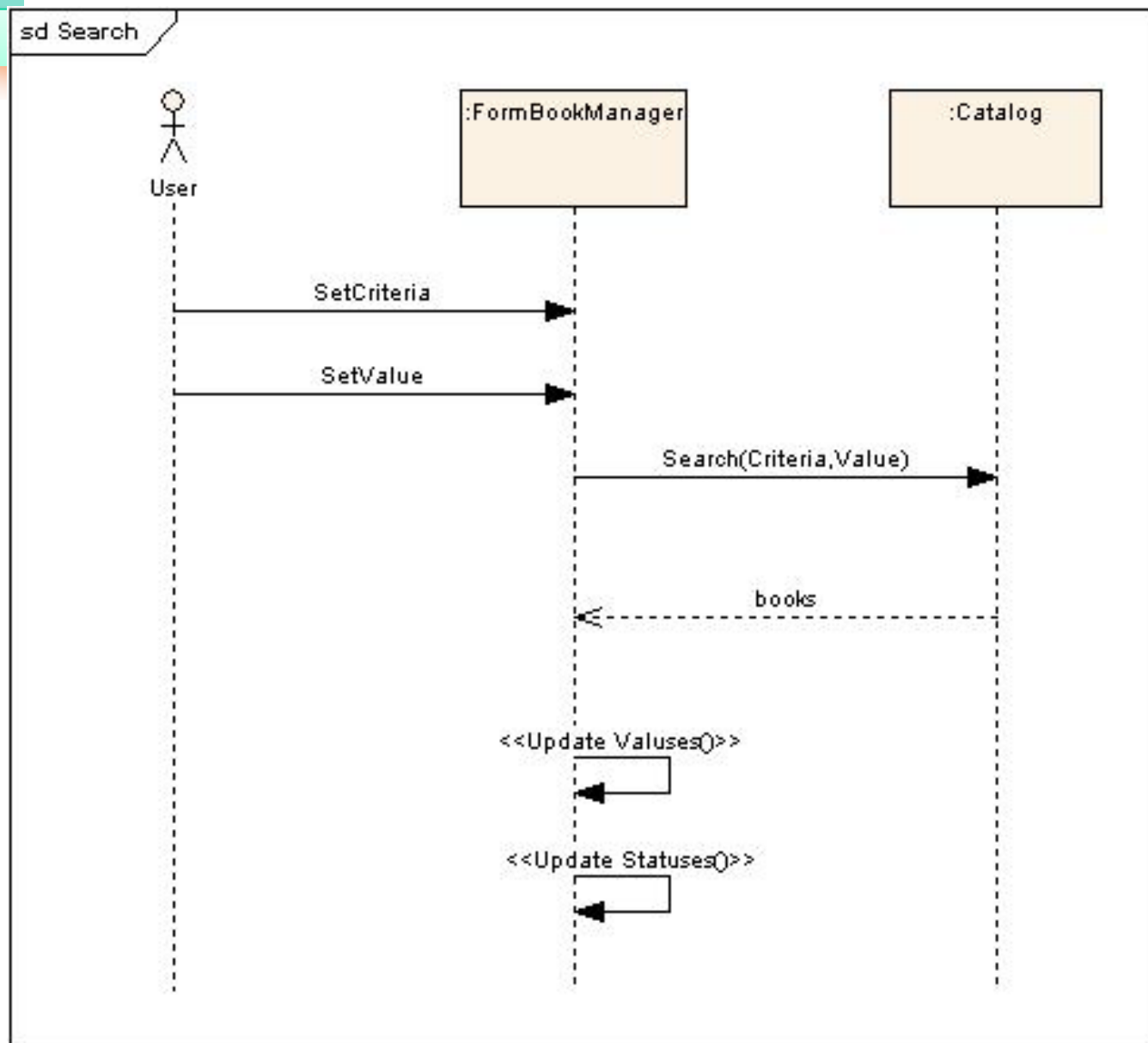
CourseDoesNotExist



مثال ۳

مورد کاربرد: جستجوی کتاب
شماره: ۳
توصیف اجمالی: کاربر اطلاعات جستجو را وارد می کند و سامانه کتاب هایی را که با این اطلاعات همخوانی دارند، به کاربر نشان می دهد.
عامل اصلی: کاربر
عامل فرعی: ندارد
شرایط اولیه: ندارد
روند اصلی: ۴. این مورد کاربرد وقتی آغاز می شود که کاربر از سامانه، درخواست جستجوی کتاب کند. ۵. سامانه شاخصه های جستجوی کتاب را به کاربر نشان می دهد و اطلاعات جستجو لازم را جهت جستجو را از کاربر می - خواهد. ۶. کاربر اطلاعات کتاب مورد نظر خود را وارد می کند. ۷. سامانه اطلاعات وارد شده را با اطلاعات کتاب ها مقایسه می کند و موارد همخوانی را می یابد. ۸. اگر کتابی همخوان با اطلاعات وارد شده پیدا شد ۸.۱. سامانه کتاب های پیدا شده را به کاربر نشان می دهد. ۹. و گرنه ۹.۱. سامانه به کاربر اطلاع می دهد که کتابی با اطلاعات وارد شده پیدا نشد.
شرایط نهایی: ندارد
روند جایگزین: ندارد

مثال ٣



انواع پیام ها

- پیام سنکرون (synchronous message) : با یک خط پر و یک پیکان توپر نشان داده می شود. (operation call)
- پیام آسنکرون (asynchronous message) : یک خط پر به همراه یک پیکان باز.
- پیام بازگشت (return message) : یک خط چین به همراه یک پیکان باز
- پیام مفقود (lost message) : پیامی که به مقصد نمی رسد. یک پیام سنکرون است که در یک نقطه پایان (نقطه سیاه) تمام می شود.
- پیام پیدا شده (found message) : پیامی که فرستنده آن مشخص نیست. پیام سنکرونی که در یک نقطه پایان شروع می شود.

انواع پیام ها

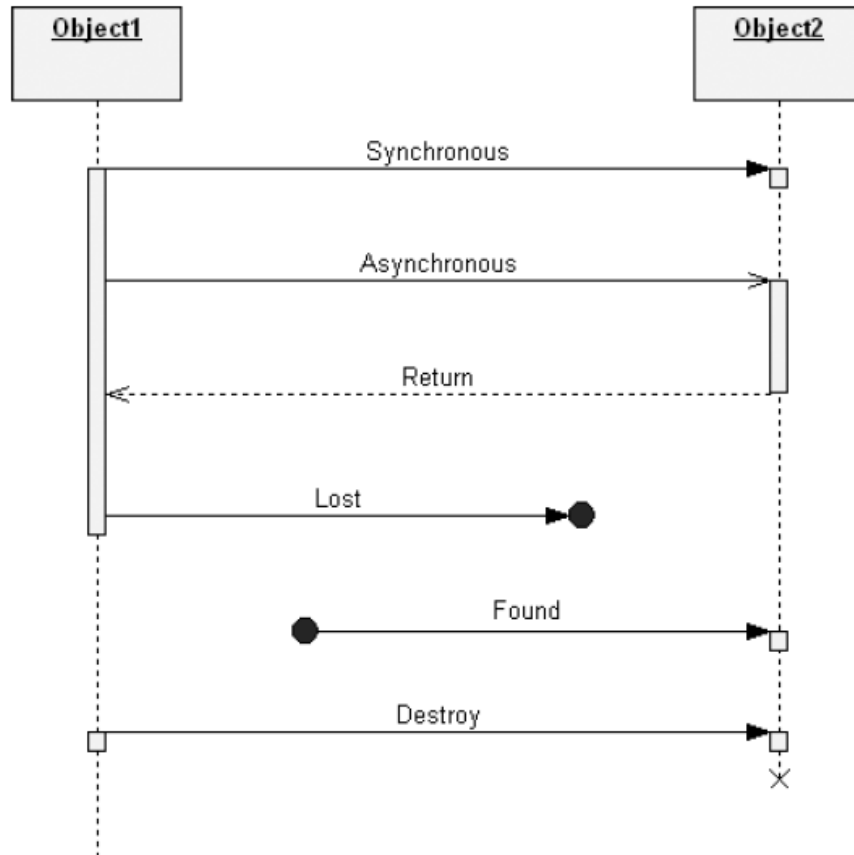


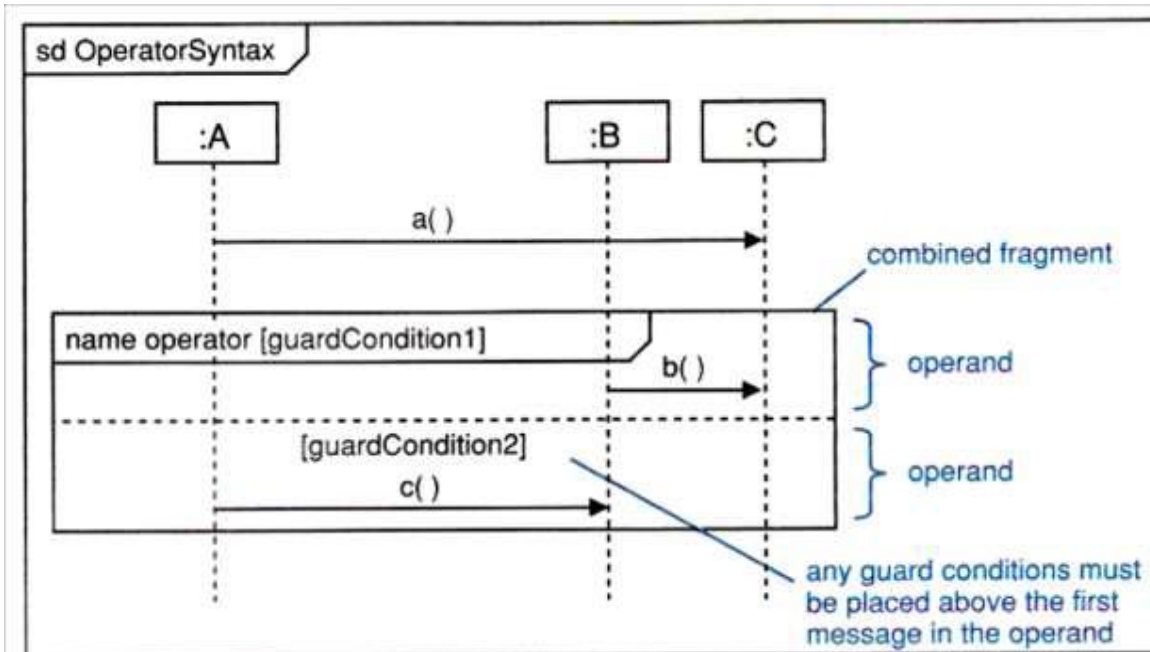
Figure 5-43 Notations for Types of Messages

- **پیام همگام (Synchronous)**: خط پیوسته با پیکان توپر نمایانگر یک فراخوانی مسدودکننده است که در آن فرستنده تا دریافت پاسخ منتظر می ماند.
- **پیام ناهمگام (Asynchronous)**: خط پیوسته با پیکان توخالی. فراخوانی غیرمسدودکننده که در آن فرستنده بدون انتظار ادامه می دهد.
- **پیام بازگشتی (Return)**: خط چین با پیکان توخالی. پاسخ به یک فراخوانی همگام قبلی.
- **پیام گمشده (Lost)**: پیکان پیام که به یک دایره سیاه کوچک ختم می شود. پیامی که ارسال شده اما هرگز دریافت نشده است.
- **پیام یافت شده (Found Message)**: پیامی که از یک دایره سیاه کوچک آغاز می شود. پیامی که از یک فرستنده نامشخص یا ناشناخته دریافت شده است.
- **تخریب (Destroy)**: علامت ضربدر (X) در انتهای خط حیات. نشان دهنده حذف یا نابودی شیء است.

قطعات ترکیبی

■ نواحی داخل یک sequence diagram با رفتارهای مختلف

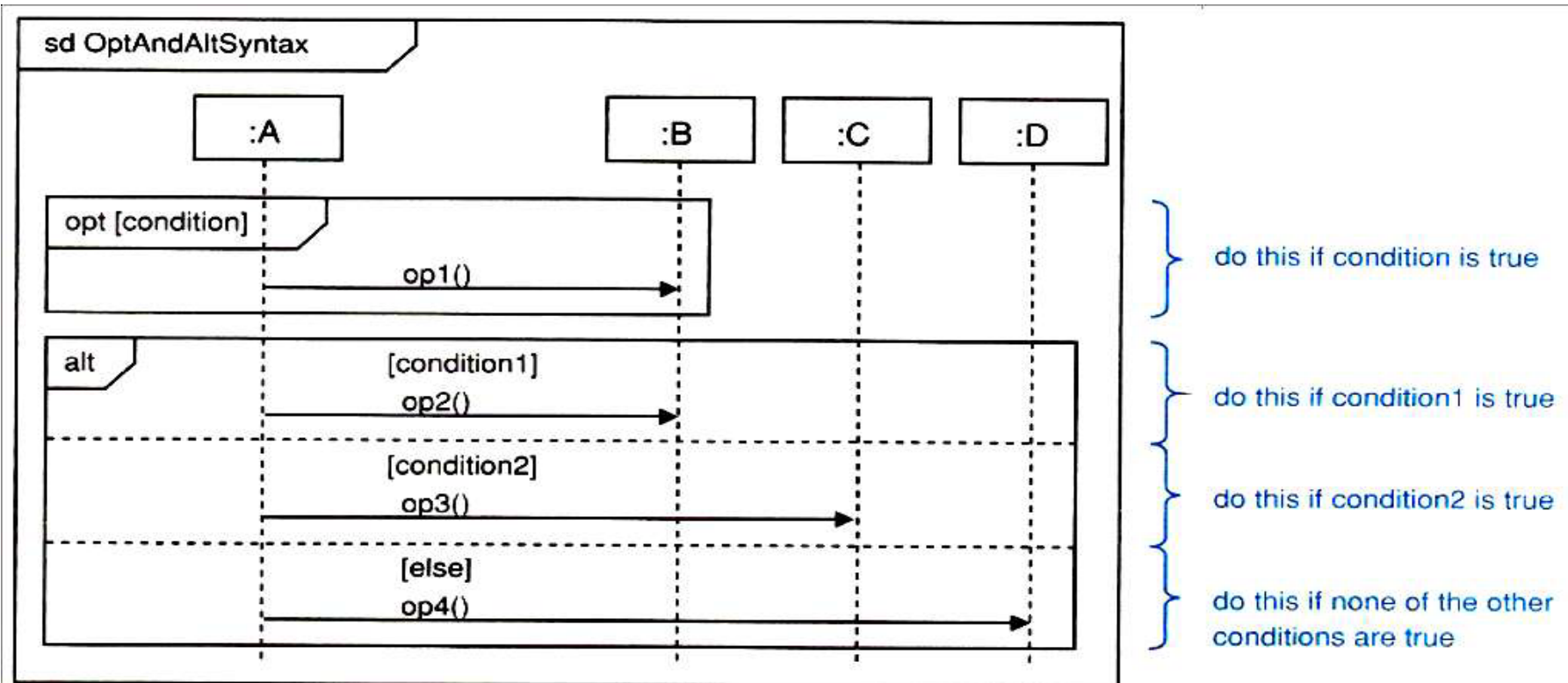
- **Operator** مشخص می کند که چگونه operand ها اجرا می شوند.
 - اپراتورها ساختارهای کنترلی هستند که رفتارهای شرطی یا تکراری را در نمودارهای توالی تعریف میکنند.
- **Guard condition** مشخص می کند که operand اجرا می شود یا نه.
- **Operand** شامل رفتار است: بخشی از یک اپراتور که شامل یک یا چند پیام است، اپرند نامیده میشود.



قطعات ترکیبی: *opt* and *alt* – Operators

■ **opt : Operand** می شود (مانند **if ... then**) اگر شرط برقرار باشد اجرا می شود

■ **alt : operand** می شود که شرط آن برقرار است اجرا می شود.



قطعات ترکیبی: *opt* and *alt* – Operators

اپراتور **opt** برای نشان دادن یک رفتار اختیاری به کار می رود؛ یعنی:
اگر شرط برقرار باشد، این بخش اجرا می شود، در غیر این صورت نادیده گرفته می شود.

در واقع مشابه یک **if** ساده بدون **else** است.
مثال:

سناریو: ورود کاربر با رمز اختیاری (OTP)
کاربر لاگین می کند

اگر احراز هویت دو مرحله ای فعال باشد، OTP ارسال می شود

opt [Two-Factor Enabled]

System → User: Send OTP

User → System: Enter OTP

اگر شرط برقرار نباشد یعنی **Two-Factor** غیر فعال باشد، این بخش اصلاً اجرا نمی شود.

قطعات ترکیبی: *opt* and *alt* – Operators

اپراتور *alt* (Alternative)

■ مفهوم

■ اپراتور *alt* برای نمایش چند مسیر جایگزین استفاده می شود:

■ فقط یکی از مسیرها بر اساس شرط اجرا می شود.

■ این دقیقاً معادل *if / else if / else* است.

■ سناریو: پرداخت آنلاین

***alt* [Payment Successful]**

System → User: Confirm Order

***else* [Payment Failed]**

System → User: Show Error

■ فقط یکی از این دو مسیر اجرا می شود:

■ اگر پرداخت موفق باشد - پس تأیید سفارش

■ اگر ناموفق باشد - پس نمایش خطا

قطعات ترکیبی: *opt* and *alt* Operators

Use case: ManageBasket

ID: 2

Brief description:

The Customer changes the quantity of an item in the basket.

Primary actors:

Customer

Secondary actors:

None.

Preconditions:

1. The shopping basket contents are visible.

Main flow:

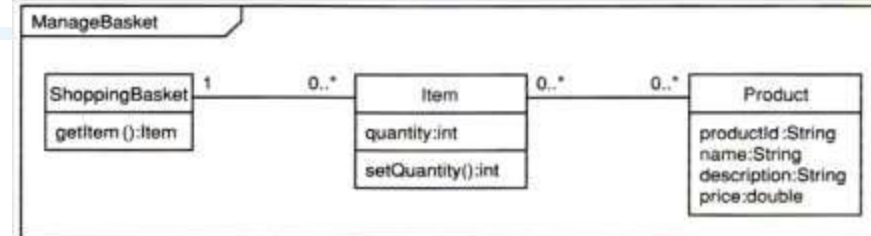
1. The use case starts when the Customer selects an item in the basket.
2. If the Customer selects "delete item"
 - 2.1 The system removes the item from the basket.
3. If the Customer types in a new quantity
 - 3.1 The system updates the quantity of the item in the basket.

Postconditions:

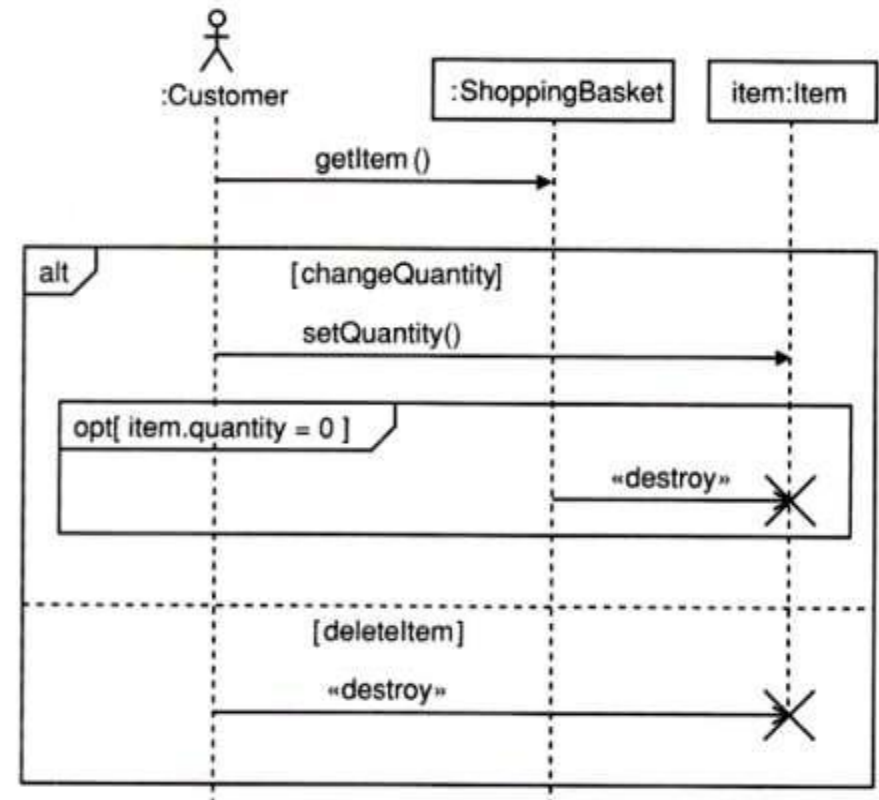
None.

Alternative flows:

None.



sd ManageBasket



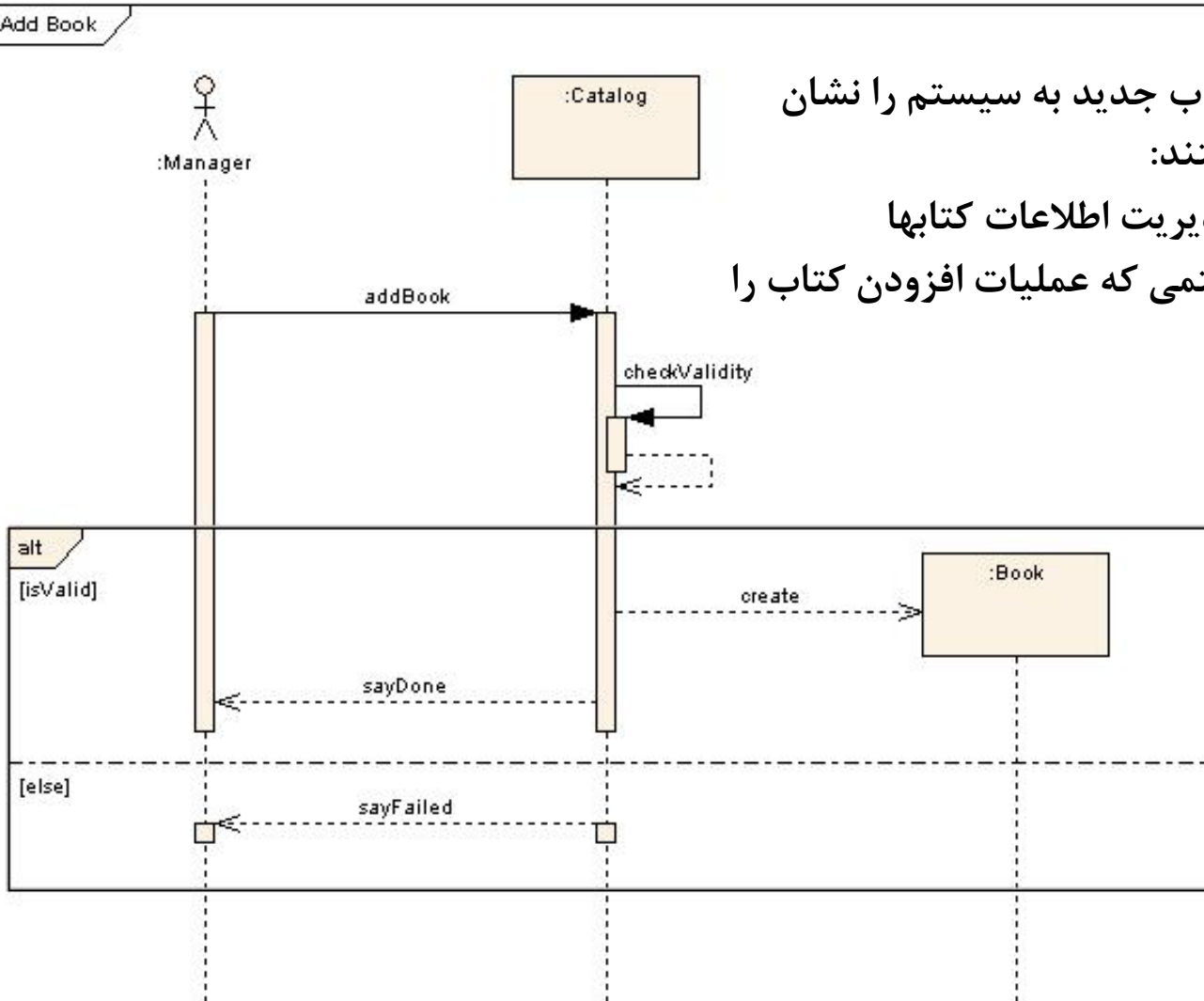
مثال ۴

مورد کاربرد: اضافه کردن کتاب
شماره: ۱۵
شماره پدر: ۱۴
توصیف اجمالی: مدیر می تواند کتاب جدید تعریف کند.
عامل اصلی: مدیر
عامل فرعی: ندارد
شرایط اولیه: مدیر باید وارد سامانه شده باشد.
روند اصلی: ۱. این مورد کاربرد وقتی آغاز می شود که مدیر درخواست اضافه نمودن کتابی را اعلام کند. ۲. سامانه از مدیر درخواست می کند اطلاعات کتاب جدید را وارد کند. ۳. مدیر اطلاعات مورد نظر خود را وارد می کند. ۴. سامانه از مدیر می خواهد دسته مورد نظر جهت اضافه نمودن کتاب را انتخاب کند. ۵. مدیر دسته مورد نظر خود را مشخص می کند. ۶. مدیر اطلاعات وارد شده را تأیید می کند. ۷. سامانه مجاز بودن اضافه نمودن کتاب را بررسی می کند. ۸. سامانه کتاب را اضافه می کند.
شرایط نهایی: کتابی به مجموعه کتاب ها اضافه می شود.
روند جایگزین: مجاز نبودن اضافه کردن کتاب جدید

مثال ۴

این نمودار فرآیند افزودن یک کتاب جدید به سیستم را نشان می‌دهد. دو شیء اصلی درگیر هستند:

۱. Catalog: مسئول مدیریت اطلاعات کتابها
۲. Manager: کاربر/سیستمی که عملیات افزودن کتاب را آغاز میکند



Operators – *loop* and *break* : قطعات ترکیبی

■ در زبان UML ، ساختارهای کنترل مانند حلقه‌ها (loop) و توقف‌ها (break) برای نمایش تکرار یا قطع اجرای پیام‌ها بین اشیاء استفاده می‌شوند.

■ loop – loop min, max [condition]

تکرار از حداقل min بار تا حداکثر max بار، در صورتی که شرط [condition] برقرار باشد.

مثال فروشگاه آنلاین:

بررسی وضعیت کالاها در سبد خرید یک کاربر (حداقل ۱ کالا و حداکثر ۵ کالا):

loop 1, 5 [hasNextItem()]

cart : getNextItem()

■ **inventory : checkStock(item)**

Operators – *loop* and *break* : قطعات ترکیبی

- **loop n, m – loop (m-n) times**

اجرای حلقه برای تعداد دفعات مشخص بین n تا m بار.

مثال فروشگاه آنلاین:

نمایش پیشنهادهای ویژه به کاربر هنگام ورود (بین ۲ تا ۴ پیشنهاد):

loop 2, 4

offerService : showSpecialOffer()

- **loop [booleanExpression] – loop while booleanExpression is true**

حلقه تا زمانی که شرط بولی برقرار باشد.

مثال فروشگاه آنلاین:

تا زمانی که کاربر در حال مرور محصولات است:

loop [user.isBrowsing()]

- **productService : displayNextProduct()**

Operators – *loop* and *break* : قطعات ترکیبی

- **loop 1, * [booleanExpression] – loop once then loop while booleanExpression is true**

حداقل یک بار اجرا می شود و سپس در صورت برقرار بودن شرط ادامه می یابد.

مثال فروشگاه آنلاین:

بارگیری صفحه پیشنهاد ویژه، و ادامه ی بارگیری در صورت موجود بودن پیشنهادهای بیشتر:

```
loop 1, * [hasMoreOffers()]
```

```
offerService : loadOffer()
```

- **break** اگر **guard condition** درست باشد، **operand** اجرا می شود، نه بقیه حلقه تکرار

در صورت برقرار بودن شرط نگهبان (**guard condition**) ، اجرای حلقه متوقف می شود و تنها همان بخش اجرا می شود.

مثال فروشگاه آنلاین:

اگر موجودی کالا صفر باشد، عملیات افزودن به سبد خرید متوقف می شود:

```
loop [itemsRemaining()]
```

```
item : checkStock()
```

```
break [stock == 0]
```

```
user : showOutOfStockMessage()
```

Operators – *loop* and *break* : قطعات ترکیبی

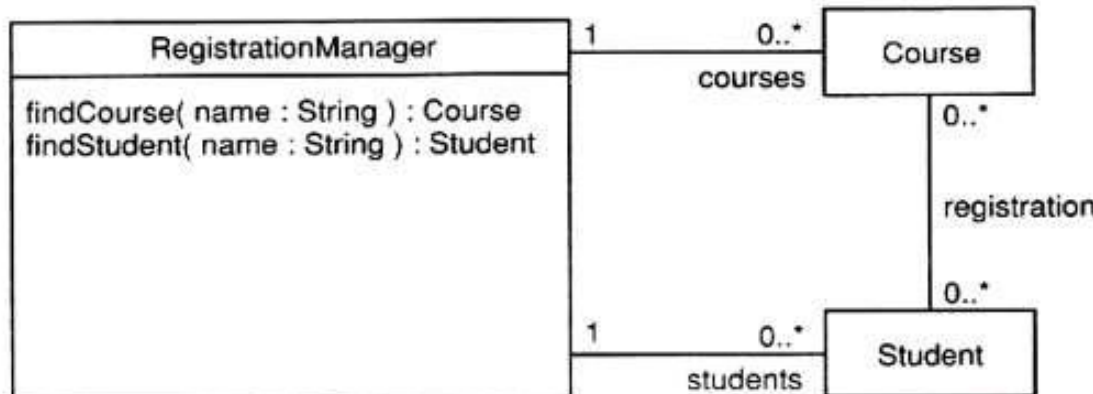
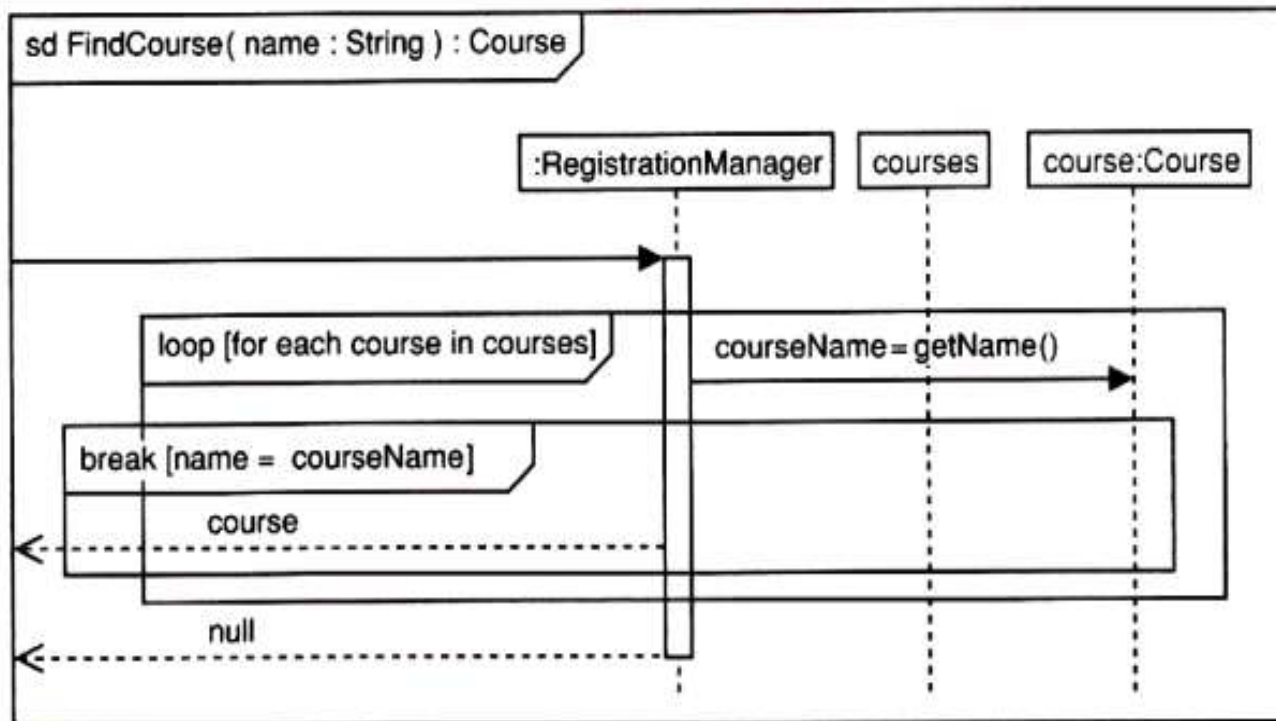
loop – loop min, max [condition] ■

- loop or loop * - loop forever;
- loop n, m – loop (m-n) times;
- loop [booleanExpression] – loop while booleanExpression is true;
- loop 1, * [booleanExpression] – loop once then loop while booleanExpression is true;

break : اگر guard condition درست باشد، operand ■

اجرا می شود، نه بقیه حلقه تکرار

Operators – *loop* and *break* : قطعات ترکیبی



نمودار ماشین حالات

این نمودار نشان می‌دهد یک شیء یا سیستم در هر لحظه در چه حالتی است و با وقوع چه رویدادی به حالت دیگر می‌رود.

هر سیستم:

در یک حالت **State** قرار دارد

با یک رویداد **Event** تغییر می‌کند

و به یک حالت جدید می‌رود

در یک سیستم عامل مانند **Linux**، وضعیت پردازنده‌ها (**Process States**) با ماشین حالت مدل می‌شود.
حالت‌ها:

■ **New**

■ **Ready**

■ **Running**

■ **Waiting**

■ **Terminated**

■ انتقال:

■ **Ready --(scheduler)--> Running**

■ **Running --(I/O request)--> Waiting**

■ **Waiting --(I/O complete)--> Ready**

نمودار ماشین حالات فروشگاه آنلاین

■ دیاگرام های ماشین حالت به منظور طراحی و فهم سیستم های مبتنی بر زمان استفاده می شوند. نمودار ماشین حالات رفتار یک موجودیت (در اینجا سفارش مشتری) را در طول چرخه ی حیات آن نشان می دهد.

■ این نمودار به ما کمک می کند تا متوجه شویم که یک سفارش در چه وضعیت هایی قرار می گیرد و چه رویدادهایی باعث تغییر حالت آن می شوند.

State Machine Diagrams

- ماشین های حالت در صنعت به دلیل استفاده در پردازش بلادرنگ شهرت دارند.
- وسایل پزشکی، سیستم های مالی، سیستم های کنترل و فرمان ماهواره مثال هایی هستند که در آنها دیاگرام های ماشین حالت نقش مهمی در درک چگونگی عملکرد سیستم در مقابل اتفاقات کلیدی بازی می کنند.
- یک دیاگرام ماشین حالت، رفتار را به صورت توالی یک سری حالت ها، رخدادهای راه اندازی شده و عملیات وابسته ای که ممکن است اتفاق افتد.
- دیاگرام های ماشین حالت توصیف کننده رفتار اشیاء هستند. ولی می توانند اجزای بزرگتر هر سیستم را نشان دهند.

حالات در سناریوی فروشگاه آنلاین:

ایجاد نشده (Idle)

- هنوز سفارشی ثبت نشده است.

در حال آماده سازی (Cart Created)

- مشتری کالایی را به سبد خرید اضافه کرده است.

در حال پرداخت (Awaiting Payment)

- مشتری فرایند پرداخت را آغاز کرده ولی هنوز پرداخت تکمیل نشده است.

پرداخت موفق (Paid)

- پرداخت با موفقیت انجام شده و سفارش ثبت شده است.

در حال ارسال (Shipping)

- سفارش بسته بندی و برای ارسال آماده شده است.

تحويل داده شده (Delivered)

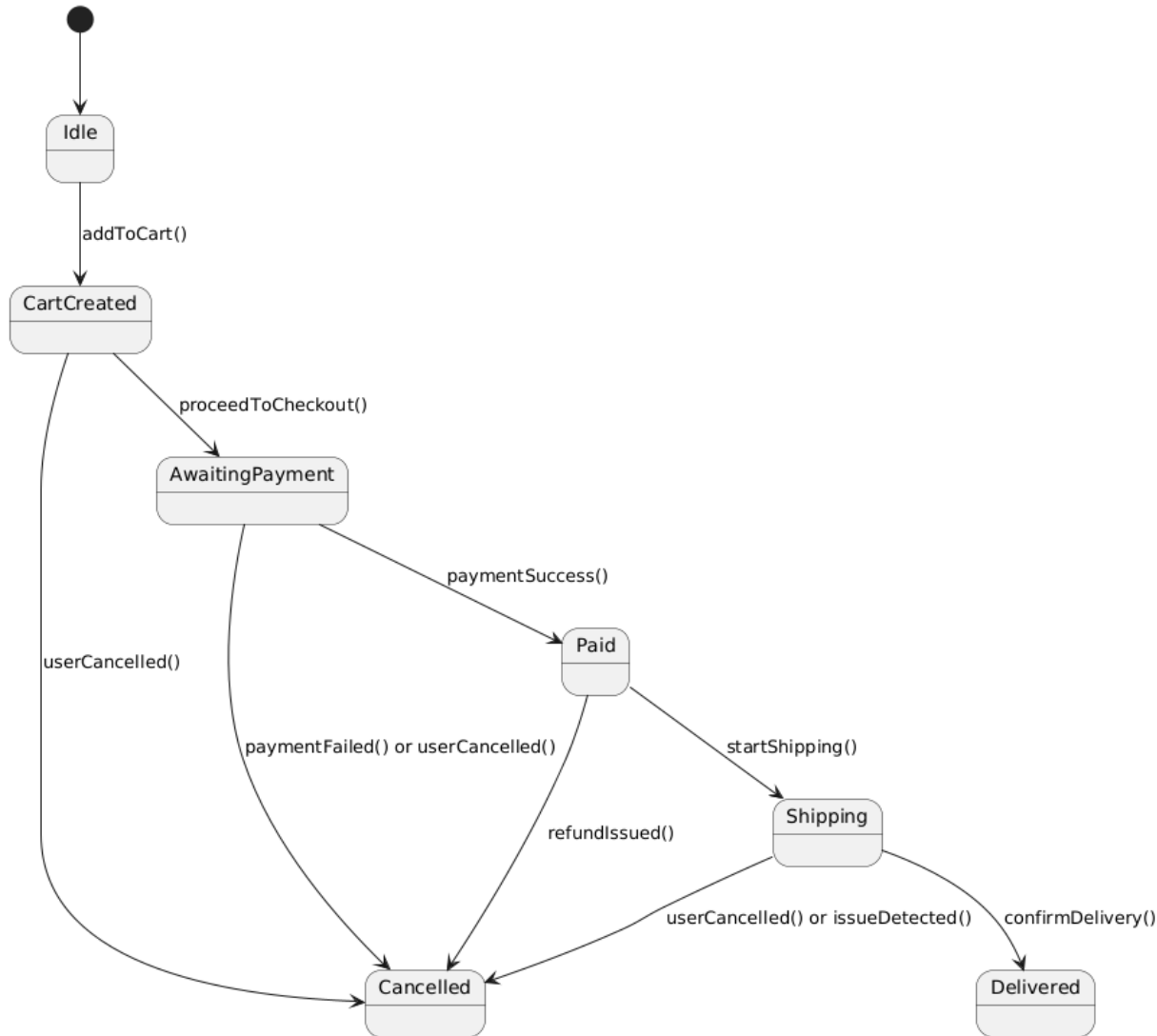
- سفارش با موفقیت به مشتری تحويل شده است.

لغو شده (Cancelled)

- سفارش توسط کاربر یا سیستم لغو شده است.

نمودار ماشین حالات فروشگاه آنلاین

State Machine - Order Lifecycle in Online Shop



State Machine Diagrams (cont.)

- حالت شی نمایش دهنده نتایج متراکم رفتار آن است.
- برای مثال، وقتی یک تلفن راه اندازی می شود،
 - در حالت **idle** قرار دارد و
 - آماده شروع به کار کردن (**initiate**)
 - یا پاسخگویی (**receive call**) است.
 - هنگامیکه گوشی تلفن را بر میداریم، تلفن در حالت شماره گیری (**dialing**) قرار دارد. در این حالت، تلفن زنگ نخواهد خورد،
 - اگر تلفن زنگ بخورد و گوشی برداشته شود، تلفن در حالت پاسخگویی قرار می گیرد و ما قادر به صحبت کردن با شخصی که تماس گرفته خواهیم بود.

State Machine Diagrams (cont.)

- هنگامی که یک شی در یک حالت مفروض قرار می گیرد، می تواند یکی از موارد زیر را انجام دهد:
 - اجرای یک فعالیت
 - انتظار برای یک رخداد
 - تکمیل یک شرط
 - انجام یک یا همه شرایط بالا
- در هر دیاگرام حالت، باید فقط یک حالت شروع (initiate state) وجود داشته باشد.

Initial State



State



State

Final State



State Machine Diagrams (cont.)

- حرکت بین حالت‌های مختلف **transition** نامیده می شود.
- هر **transition** دو حالت را به هم متصل می کند.
- یک حالت می تواند یک **transition** به خودش داشته باشد.



Figure 5–53 Transitions for the Duration Timer

State Machine Diagrams (cont.)

- رخدادهای خاصی باید اتفاق افتند تا یک **transition** انجام شود.
- این رخدادها روی **transition** نوشته می شوند.
- یک رخداد، اتفاقی است که ممکن است سبب تغییر حالت سیستم شود.

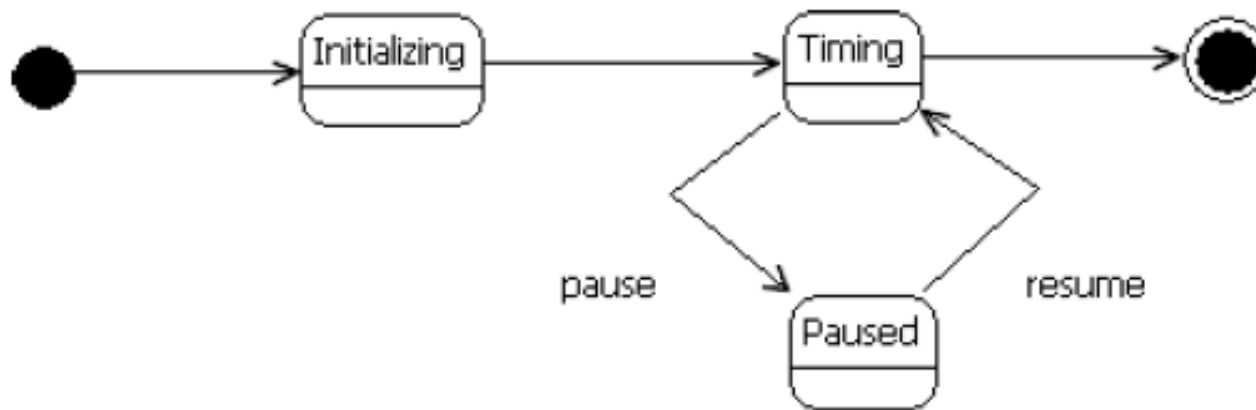


Figure 5–54 Additional States and Transition Events for the Duration Timer

State Machine Diagrams (cont.)

- فعالیت ها ممکن است با حالت ها همراه شوند.
- انجام یک فعالیت به محض ورود به حالت
- انجام یک فعالیت در حالی که در یک **state** هستیم.
- انجام یک فعالیت به محض خروج از یک حالت.

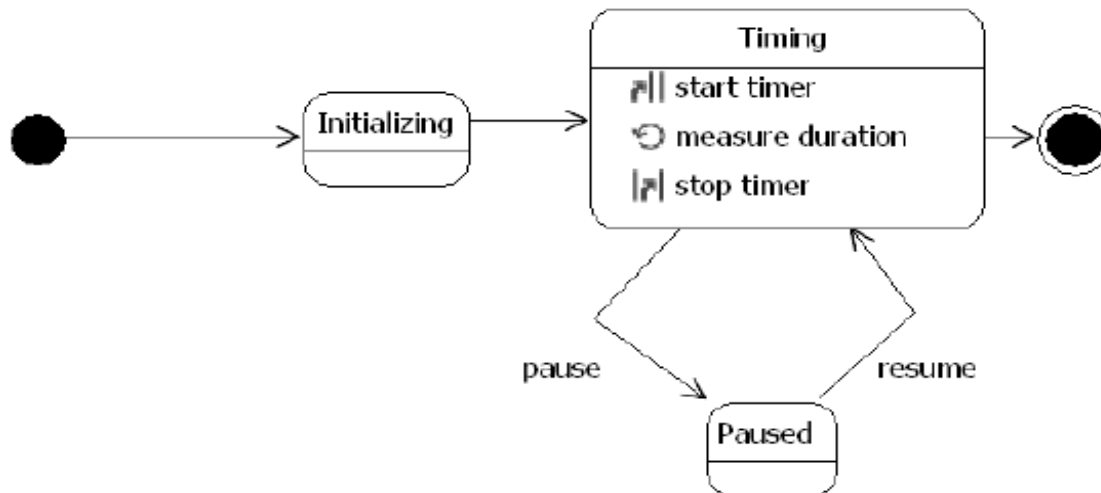


Figure 5–55 Entry, Do, and Exit Activities

State Machine Diagrams (cont.)

- شرط ها هم می توانند برای کنترل **transition** به کار روند.
- این شرایط به عنوان محافظ (**guard**) عمل می کنند، زیرا وقتی یک رخداد اتفاق می افتد، شرط می تواند اجازه دهنده **transition** باشد (اگر شرط درست باشد) یا به **transition** اجازه اجرا ندهد (اگر شرط نادرست باشد).
- راه دیگر کنترل رفتار **transition**، استفاده از **effect** ها است.
- یک **effect** رفتاری (فعالیت یا عمل) است که وقتی یک رخداد اتفاق می افتد، رخ می دهد.
- بنابراین هنگامی که یک رخداد **transition** اتفاق می افتد، **transition** اجرا شده و **effect** هم اتفاق می افتد.

State Machine Diagrams (cont.)

■ **Guard** یک شرط منطقی است که تعیین می کند آیا انتقال انجام شود یا نه.

■ نحوه نمایش : روی فلش می نویسیم : **event [condition]**

■ **Pending --(pay [amount >= price])--> Completed**

■ اگر پول کافی نباشد:

■ انتقال انجام نمی شود

■ سیستم در همان حالت می ماند

■ **Effect** اثر / عمل : یک عمل است که هنگام انجام انتقال اجرا می شود

■ انتقال اتفاق می افتد

■ همزمان یک کار هم انجام می شود

■ نحوه نمایش در : **event / action**

■ مثال ۱: دستگاه فروش

■ **Paid --(confirm / dispense item)--> Delivering**

■ وقتی **confirm** رخ دهد :

■ انتقال انجام می شود

■ همزمان کالا تحویل داده می شود

State Machine Diagrams (cont.)

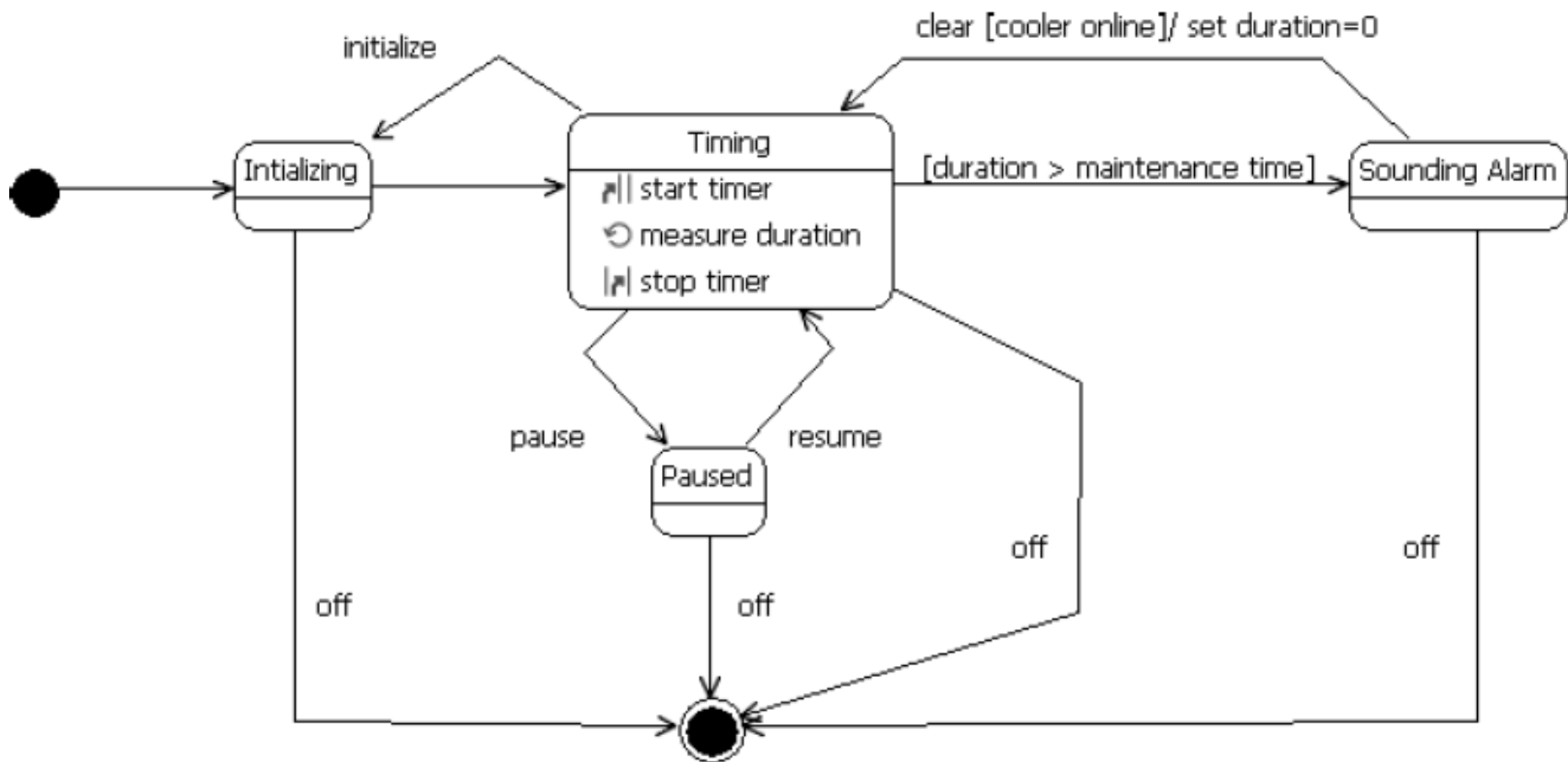


Figure 5–56 The Enhanced State Machine Diagram for the Duration Timer

State Machine Diagrams (cont.)

- ترتیب ارزیابی در حالت شرطی مهم است.
- حالت S با گذار T روی رخداد E با شرط C و effect A مفروض است. ترتیب زیر اعمال می شود:
 1. رخداد E اتفاق می افتد.
 2. شرط C ارزیابی می شود.
 3. اگر C درست باشد، transition T راه اندازی می شود و effect A احضار می گردد.
- به این معنی که اگر یک شرط به درستی ارزیابی نشود، حالت transition ممکن است راه اندازی نشود تا زمانی که رخداد دوباره اتفاق افتد و شرط دوباره ارزیابی شود.

State Machine Diagrams (cont.)

ترتیب ارزیابی در یک Transition

فرض کنید داریم:

■ $S \rightarrow (E [C] / A) \rightarrow S'$

- حالت فعلی - رخداد Event - شرط Guard - اثر Effect - همین انتقال - S' حالت مقصد
- تیب اجرای واقعی: رخداد E اتفاق می افتد. شرط C بررسی می شود. اگر درست باشد - انتقال اتفاق می افتد و effect یا اثر انجام می شود.
- مثال: سیستم ورود

■ Idle $\rightarrow$ (login [password correct] / grant access) $\rightarrow$ LoggedIn

- این رفتار نشان می دهد که:
- ✓ سیستم ها event-driven هستند
- نه condition-driven یعنی:
- شرط به تنهایی باعث تغییر نمی شود. حتماً باید رخداد هم باشد

State Machine Diagrams (cont.)

- در سیستم های پیچیده، دیاگرام های ماشین حالت، می توانند بسیار بزرگ باشند.
- امکان استفاده از حالت های تو در تو وجود دارد.
- در این حالت، دیاگرام ماشین حالت به صورت عمقی طراحی می شود.

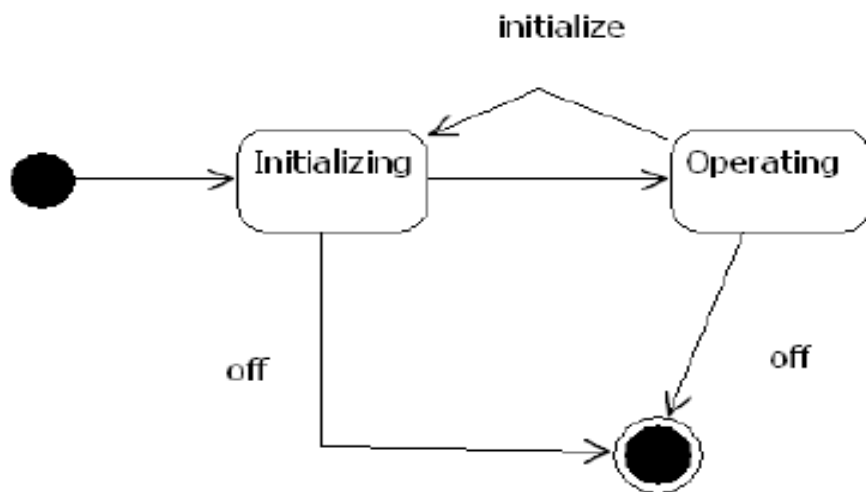


Figure 5–58 A Higher-Level View of the State Machine Diagram for the Duration Timer

State Machine Diagrams (cont.)

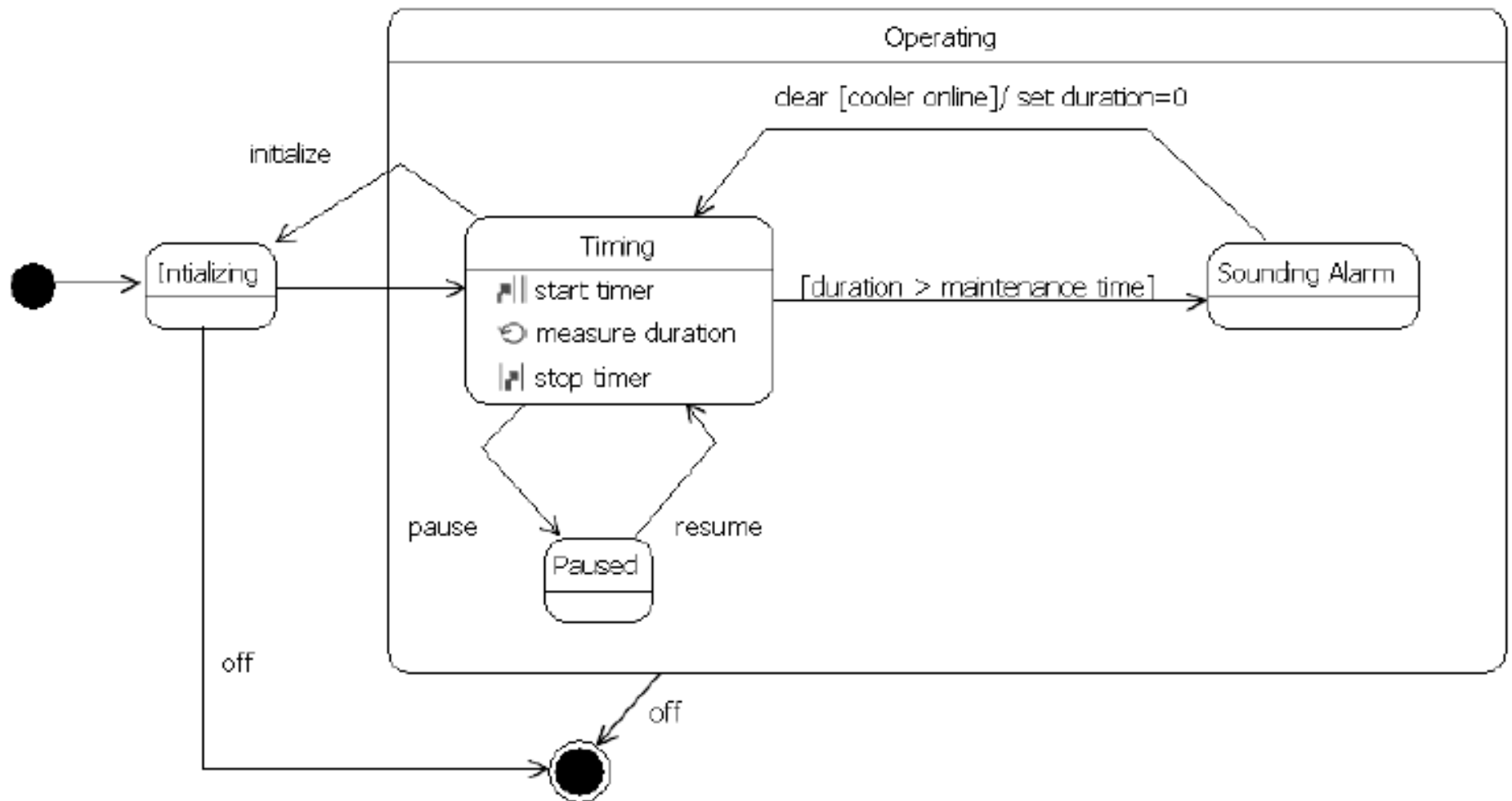
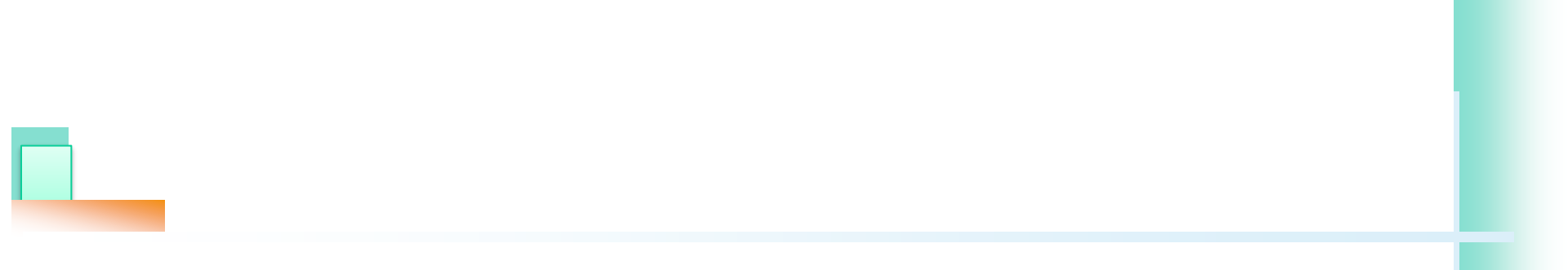


Figure 5–57 Composite and Nested States



■ پایان